

Projektarbeit

Stoneforge: Ein prozedural generiertes 2D-Spiel als Testumgebung für Reinforcement Learning

Spielentwicklung, Weltgenerator und die Generalisierung
eines rekurrenten Navigationsagenten

Vorgelegt von: Florian Merlau
Laurin Rößler

Studiengang: Masterstudiengang Informatik und Security (M.Sc.)

Betreuer: Prof. Dr. Carsten Lecon

Abgabedatum: 14. August 2026

Kurzfassung

Diese Projektarbeit verbindet zwei Themen, die auf den ersten Blick wenig miteinander zu tun haben: die prozedurale Erzeugung von Spielwelten und Reinforcement Learning unter partieller Beobachtbarkeit. Im ersten Teil bauen wir mit *Stoneforge* ein Spiel, dessen Generator reproduzierbare, nachweislich lösbare zweidimensionale Welten mit frei steuerbarer Zielentfernung erzeugt. Jede Karte ist lösbar. Jeder Seed reproduzierbar. Im zweiten Teil stellen wir eine unbequeme Frage: Bildet ein Agent, der nur einen kleinen lokalen Ausschnitt seiner Umgebung sieht, wirklich eine Navigationsstrategie, oder lernt er bloß die trainierten Karten auswendig? Wir testen ihn dort, wo es zählt: auf Welten, die er nie gesehen hat.

Der Generator erfüllt seine Anforderungen vollständig. Determinismus bis hin zu bitidentischen Trainingspfaden, 100 % Lösbarkeit über 3 150 geprüfte Seeds, eine Zielentfernung, die am tatsächlichen Laufweg gemessen exakt eingehalten wird. Der Agent, ein rekurrentes Verfahren mit internem Gedächtnis, erreicht über sieben unabhängige Läufe $65,7\% \pm 12,4$ auf dem Testset und $66,9\% \pm 12,8$ auf einem gesperrten Holdout. Die Holdoutschwelle ist erfüllt, die Testsetschwelle verfehlt.

Diese Zahlen tragen nicht. Eine ungelernete Kompassheuristik aus vier Zeilen Code erreicht auf derselben Aufgabe 92 % und ist bei vergleichbarer Erfolgsquote zugleich dreimal wegeffizienter. Die Erfolgsquote misst hier also nicht die Fähigkeit, um die es geht. Die Ursache liegt nicht im Agenten, sondern im Generator: Solange die Hindernisse aus unkorreliertem Rauschen stammen, entstehen kaum Sackgassen, und ohne Sackgassen erzwingt die Aufgabe kein Gedächtnis. Die zweite Forschungsfrage ist mit diesem Aufbau nicht entscheidbar.

Daraus folgt der übertragbare Beitrag. Die Schwierigkeit einer Lernaufgabe ist eine Eigenschaft der Weltgenerierung und gehört dort spezifiziert und geprüft, bevor ein Lernverfahren darauf angesetzt wird. Eine ungelernete Referenzpolitik leistet genau das, in Sekunden statt in Trainingsstunden.

Schlagwörter: Prozedurale Weltgenerierung, Spielentwicklung, Reinforcement Learning, POMDP, Generalisierung, RecurrentPPO, Curriculum Learning, Baseline-Vergleich

Eidesstattliche Erklärung

Wir versichern hiermit, dass wir die vorliegende Projektarbeit mit dem Thema „*Stoneforge*: Ein prozedural generiertes 2D-Spiel als Testumgebung für Reinforcement Learning“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen oder anderen Quellen entnommen sind, sind als solche kenntlich gemacht. Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt.

Die eingesetzten generativen KI-Werkzeuge sind auf der folgenden Seite in der „Erklärung zur Nutzung generativer KI“ vollständig ausgewiesen; die übrigen Hilfsmittel sind in Anhang 9 dokumentiert. Der vollständige Quellcode sowie das lückenlos geführte Experiment-Changelog liegen dem Projekt-Repository bei.

Ort, Datum

Florian Merlau

Ort, Datum

Laurin Rößler

Erklärung zur Nutzung generativer KI

Bei der Erstellung dieser Arbeit wurden generative KI-Werkzeuge als Hilfsmittel eingesetzt; Art und Umfang sind in der folgenden Tabelle offengelegt. Wir verantworten alle daraus übernommenen Inhalte vollumfänglich selbst, haben sie inhaltlich nachvollzogen und gegen die Primärdaten des Projekt-Repositories verifiziert. **Nicht** eingesetzt wurde generative KI zur Erzeugung von Messergebnissen: Sämtliche berichteten Zahlen stammen aus eigenen Trainings- und Evaluationsläufen und sind im Experiment-Changelog sowie über Seeds, Modell-Checkpoints und Konfigurationsdateien reproduzierbar (Anhang 9).

Übersicht der verwendeten KI-Werkzeuge

Werkzeug	Einsatzform	Teile der Arbeit	Bemerkungen
Claude (Opus 5), Anthropic	Vorschläge zur Prüfung des Manuskripts auf Rechen-, Zahlen- und Konsistenzfehler sowie zur Formulierung des Diagnoseskripts <code>reward_bilanz.py</code>	Kapitel 4–7, Anhänge A und B	Jeder Vorschlag wurde von uns gegen Quellcode, <code>eval_history.json</code> und <code>CHANGELOG.md</code> geprüft und erst nach eigener Verifikation übernommen; drei Beanstandungen erwiesen sich dabei als unzutreffend und wurden verworfen. Die Messwerte des Diagnoseskripts stammen aus eigenen Läufen.
Claude Code (Opus 4.8), Anthropic	Textentwürfe und -überarbeitungen nach unseren Vorgaben (Gliederung, Kürzung, sprachlicher Stil); Vorschläge für Literaturbelege; Anpassungen am Dokumentlayout	Kurzfassung, Kapitel 1–3 und 5, Abschnitte in Kapitel 7–8, Deckblatt	Struktur, Aussagen und alle berichteten Ergebnisse legten wir fest; Claude lieferte Formulierungs- und Kürzungsvorschläge sowie Belegvorschläge, die wir gegen <code>references.bib</code> und die Originalquellen geprüft haben. Layoutänderungen (Logo, Farben, Abstände) sind rein optisch und ohne inhaltliche Wirkung.

Ort, Datum

Florian Merlau

Ort, Datum

Laurin Rößler

Inhaltsverzeichnis

Kurzfassung	ii
Eidesstattliche Erklärung	iii
Erklärung zur Nutzung generativer KI	iv
Abbildungsverzeichnis	vii
Tabellenverzeichnis	viii
Abkürzungs- und Symbolverzeichnis	ix
1 Einleitung	1
1.1 Motivation	1
1.2 Die Aufgabe in Kürze	1
1.3 Problemstellung und Forschungsfrage	1
1.4 Zielsetzung und Erfolgskriterien	2
1.5 Aufbau der Arbeit	3
2 Grundlagen	5
2.1 Prozedurale Weltgenerierung	5
2.1.1 Begriff, Seed und Determinismus	5
2.1.2 Algorithmen der prozeduralen Weltgenerierung	5
2.2 Lernen unter partieller Beobachtbarkeit	7
2.2.1 Reinforcement Learning und der Markov-Entscheidungsprozess	7
2.2.2 Partielle Beobachtbarkeit (POMDP)	7
2.2.3 Wertbasierte Verfahren: Q-Learning und DQN	8
2.2.4 Policy-Gradient- und Actor-Critic-Verfahren	9
2.2.5 Proximal Policy Optimization (PPO)	9
2.2.6 Rekurrente Politiken (LSTM)	10
2.3 Prozedurale Welten im Reinforcement Learning	10
2.4 Steuerung des Lernprozesses	11
2.4.1 Reward Shaping (PBRs)	11
2.4.2 Curriculum Learning	11
2.5 Messen und Bewerten	12
2.5.1 Metriken einer Navigationsaufgabe	12
2.5.2 Streuung, Konfidenz und die Zahl der Läufe	12
2.5.3 Referenzpolitiken als Bezugspunkt	13
3 Einordnung in verwandte Arbeiten	15
4 Implementierung des Spiels Stoneforge	17
4.1 Technologie-Stack	17
4.2 Die Bausteine: Tile, Chunk und Welt	17
4.3 Der Weltgenerator	18
4.4 Exit-Platzierung und garantierte Lösbarkeit	26
4.5 Grafischer Client und Zuschaumodus	28
5 Implementierung der Lernumgebung	30
5.1 Was der Agent davon sieht: POMDP-Charakter	30
5.2 Umgebung, Beobachtung und Aktionen	30
5.3 Die Schnittstelle: was bei <code>reset()</code> und <code>step()</code> tatsächlich passiert	31

5.4	Reward-Design	32
5.5	Trainings-Pipeline	34
5.6	Monitoring und Werkzeuge	34
6	Methodik	36
6.1	Algorithmus	36
6.2	Curriculum	36
6.3	Evaluationsprotokoll	37
6.4	Methodische Fallstricke der Evaluation	38
6.5	Herleitung der Erfolgsschwellen	39
6.6	Versuchsaufbau	40
7	Evaluation	41
7.1	Bewertung der Plattform (Z1)	41
7.2	Zentrale Gegenüberstellung: Gedächtnis statt lokalem Orakel	42
7.3	Endergebnis (v12, $n = 7$)	42
7.4	Ungelernte Referenzpolitiken und die Grenzen der Erfolgsquote	46
7.5	Der Det/Stoch-Gap: Weglängen-Analyse und POMDP-Einordnung	50
7.6	Versuche zur Schließung des Det/Stoch-Gaps	51
8	Fazit und Ausblick	54
8.1	Ergebnis	54
8.2	Beiträge	55
8.3	Einschränkungen	55
8.4	Ausblick	56
9	Anhang: Reproduzierbarkeit	58
9.1	Eingesetzte Hilfsmittel	59
10	Anhang: Entwicklungsverlauf von der Baseline zum LSTM-Agenten	60
10.1	Etappe 1 (Mai 2026): Baselines mit BFS-Orakel	60
10.2	Etappe 2 (Juni 2026): Kernbeitrag, Navigation ohne Orakel	60
10.3	Etappe 3 (13.–25.06.): Fehlkonfigurierte Hyperparameter und Root-Cause-Analyse	61
10.4	Etappe 4 (06.07.2026): Umgebungsrevision v11 und Performance	61
10.5	Etappe 5 (07.07.2026): Erster v11-Gesamtlauf, Monitoring-Redesign und Seed-Replay-Analyse	62
10.6	Etappe 6 (07.07.2026, nachmittags/abends): Systematische Fehlersuche vom Eval-Artefakt zur Batch-Größen-Korrektur	64
10.7	Umgebungsrevision v11 (06.07.2026)	65
10.8	Performance-Analyse: Training beschleunigen	67
10.8.1	CPU schlägt Apple-GPU (MPS)	67
10.8.2	Batch-Größe: vermeintliche Validierung (06.07.), am Folgetag widerlegt	68
10.9	Zeitleiste und Lessons Learned	68
	Literaturverzeichnis	71

Abbildungsverzeichnis

1	Agent-Umwelt-Interaktion im MDP	7
2	MDP gegenüber POMDP	8
3	Partielle Beobachtbarkeit am Beispiel Stoneforge	8
4	Wirkung des PPO-Clippings	9
5	Rekurrente Politik über die Zeit	10
6	Hash-Rauschen gegenüber interpoliertem Wertrauschen	20
7	Die Generator-Pipeline an einer echten Welt	24
8	Pipeline der Weltgenerierung je Chunk	25
9	Benutzeroberfläche des spielbaren C++-Clients	29
10	Luftlinien- gegenüber BFS-Potential an einer Sackgasse	33
11	Architektur: gemeinsamer Simulationskern für Training und Client	35
12	Lernkurve des historischen Referenzlaufs	43
13	Endergebnis v12 mit Streuung über sieben Läufe	44
14	Trainingsverläufe der sieben v12-Läufe	45
15	Erfolgsquote nach Startdistanz zum Exit	50
16	Entwicklung über die Modellgenerationen	61
17	Lernkurve des ersten v11-Gesamtlaufs	62
18	Live-Monitoring-Dashboard	63
19	Eval-Cap-Artefakt	64
20	Batch-Größen-Forensik	66
21	Wirkung der BFS-basierten Exit-Platzierung	67
22	Training-FPS nach Device und Batch-Größe	67
23	Validierungslauf batch_size=64	68

Tabellenverzeichnis

1	Arbeitsteilung	4
2	Technologie-Stack.	17
3	Belegung einer Chunk-Zeile Feld für Feld	23
4	Biomabhängige Schwellwerte der Basisbelegung	23
5	Tile-Zusammensetzung der Eval-Welten	24
6	Aktive Mechanismen der Weltgenerierung	25
7	Regelsweep der zellulären Glättung	26
8	Aufbau der Observation (Env v11)	30
9	Reward-Komponenten (Env v11)	33
10	Curriculum in vier Phasen	37
11	Gemeinsame Basiskonfiguration aller Läufe	40
12	Bewertung der Plattform gegen die Anforderungen Z1	41
13	Gegenüberstellung: Gedächtnis gegenüber lokalem BFS-Orakel	42
14	Endergebnis der v12-Läufe über sieben Seeds	43
15	Gating-Bilanz der sieben v12-Läufe	44
16	Ungelernte Referenzpolitiken gegenüber dem trainierten Agenten	47
17	Erfolgsquote als Funktion des Episodenbudgets	48
18	Bilanz der dichten Reward-Terme	48
19	Varianten zur Schließung des Det/Stoch-Gaps	51
20	Temperatur-Sweep der Evaluation	53
21	Systemumgebung	58
22	Root-Cause-Analyse der fehlgeschlagenen Läufe v7–v9	62
23	Zeitleiste der RL-Entwicklung	69

Abkürzungs- und Symbolverzeichnis

Kürzel	Bedeutung
A2C	Advantage Actor-Critic, synchrone Variante von A3C
approx_kl	Maß für die Stärke einer Politikänderung pro Update
BFS	<i>Breadth-First Search</i> , Breitensuche; liefert den kürzesten begehbaren Weg
BPTT	<i>Backpropagation Through Time</i> , Lernverfahren rekurrenter Netze
CNN	<i>Convolutional Neural Network</i> , faltendes neuronales Netz
DQN	<i>Deep Q-Network</i> , wertbasiertes RL-Verfahren
DRQN	<i>Deep Recurrent Q-Network</i> , DQN mit Gedächtnis
EV	<i>Explained Variance</i> , Gütemaß der gelernten Wertfunktion
GAE	<i>Generalized Advantage Estimation</i> , Schätzung des Vorteils
HARKing	<i>Hypothesizing After the Results are Known</i>
LSTM	<i>Long Short-Term Memory</i> , rekurrente Netzarchitektur (Gedächtnis)
MDP	<i>Markov Decision Process</i> , Markow-Entscheidungsprozess
MLP	<i>Multilayer Perceptron</i> , vorwärtsgerichtetes Netz ohne Gedächtnis
MPS	<i>Metal Performance Shaders</i> , GPU-Backend von Apple
PBRs	<i>Potential-Based Reward Shaping</i> , politikerhaltende Belohnungsformung
PLR	<i>Prioritized Level Replay</i> , priorisierte Level-Wiederholung
POMDP	<i>Partially Observable MDP</i> , teilweise beobachtbarer MDP
PPO	<i>Proximal Policy Optimization</i> , das hier verwendete RL-Hauptverfahren
RL	<i>Reinforcement Learning</i> , bestärkendes Lernen
SB3	<i>Stable-Baselines3</i> , verwendete RL-Bibliothek
SR	<i>Success Rate</i> , Erfolgsquote: Anteil gelöster Welten
γ	Diskontfaktor: Gewicht künftiger Belohnungen (hier 0,999)
ϵ	Clipping-Grenze von PPO (Abschnitt 2.2.5)
ε	Zufallsanteil der Vergleichspolitik (Abschnitt 7.4)
η	Pfادهffizienz: kürzester Weg geteilt durch tatsächliche Schritte
$\pi(a \mid s)$	Politik: Wahrscheinlichkeit, im Zustand s die Aktion a zu wählen
$\Phi(s)$	Potentialfunktion des Reward Shapings
σ	Standardabweichung über Trainingsläufe

1 Einleitung

1.1 Motivation

Videospiele sind eine Standardumgebung für Reinforcement Learning (RL): Die Regeln stehen fest, Erfolg ist messbar, und man kann beliebig oft von vorn anfangen [1]. Genau diese Messbarkeit täuscht allerdings leicht. Wenn eine KI ein Level fehlerfrei durchspielt, kann das zweierlei heißen: Sie hat gelernt zu spielen. Oder sie hat dieses eine Level auswendig gelernt. Von außen sieht beides gleich aus.

Auseinanderhalten lässt sich das nur mit Levels, die der Agent nie gesehen hat. Cobbe et al. zeigen es an Zahlen: Trainiert man Agenten auf einer festen Menge von Levels, schnellen die Werte dort nach oben; auf neuen Levels brechen sie ein [2]. Diese Lücke zwischen Trainings- und Testleistung zählt zu den offenen Problemen des Feldes [3]. Das übliche Gegenmittel: *prozedurale Generierung*. Die Welt entsteht für jeden Durchlauf neu aus einer Zufallszahl, Auswendiglernen bringt dann nichts mehr [2, 4].

Damit hängt aber alles daran, welche Welten der Generator überhaupt erzeugt. Kirk et al. sagen es in ihrer Übersichtsarbeit direkt: Ein rein prozeduraler Ansatz beim Benchmark-Entwurf bringt die Generalisierungsforschung nicht voran [3]. Bei den verbreiteten Umgebungen ändert sich daran wenig: Ihre Schwierigkeit lässt sich nur grob wählen, nicht für jede einzelne Welt [2, 5].

Deshalb bauen wir beides selbst: ein eigenes 2D-Spiel und einen Agenten, der darin den Ausgang finden soll, ohne dass ihm jemand den Weg vorrechnet. Wer die Welt selbst erzeugt, kann sie schrittweise schwerer machen und beobachten, wie weit eine gelernte Strategie wirklich trägt. Und weil Agent und Mensch dasselbe Spiel spielen, bleibt es am Ende nicht bei einer Prozentzahl: Wir können dem Agenten beim Spielen zusehen.

1.2 Die Aufgabe in Kürze

Der Agent startet in der Mitte einer Welt, die er noch nie gesehen hat, und soll ihren Ausgang finden. Das Spiel dahinter heißt *Stoneforge* und läuft rundenbasiert ab. Aus einer Zufallszahl, dem *Seed*, entsteht eine Landschaft aus quadratischen Feldern: Wiese, Wald, Fels, Wasser. Gleicher Seed, gleiche Welt; anderer Seed, völlig andere Welt. Pro Zug geht der Agent ein Feld nach oben, unten, links oder rechts. Mehr kann er nicht.

Schwierig wird es dadurch, dass er die Welt nicht überblickt. Er sieht nur einen kleinen Ausschnitt um sich herum, der Rest ist schwarz. Die Richtung zum Ausgang kennt er, wie bei einem Kompass, aber nicht den Weg dorthin. Steht eine lange Felswand dazwischen, muss er selbst herausfinden, ob er sie nördlich oder südlich umgeht, und sich merken, wo er schon war. Abbildung 3 auf Seite 8 zeigt beides nebeneinander: die Welt, wie sie aussieht, und das, was der Agent davon sieht.

Der Agent lernt über Belohnung. Ausgang erreicht: großer Pluspunkt. Jeder Schritt: kleiner Abzug. Nach Millionen Versuchen verhält er sich so, dass am Ende möglichst viel übrig bleibt. Am Schluss zählen wir, wie viele von 50 unbekannten Welten er schafft.

1.3 Problemstellung und Forschungsfrage

In diesem Aufbau stecken zwei Probleme, und sie hängen zusammen.

Das erste betrifft den Agenten. Weil er nur einen Ausschnitt sieht, sehen zwei völlig verschiedene Orte für ihn gleich aus. Er muss aus dem, was er bisher gesehen hat, auf seine Lage schließen. Diese Arbeit könnte man ihm abnehmen und ihm den kürzesten Weg vorrechnen, ein „Orakel“. Dann löst er die Aufgabe, hat aber nichts gelernt, was ohne dieses Orakel funktioniert. Die Aufgabe ist also erst dann richtig gestellt, wenn dieser Ausweg ausgeschlossen ist.

Das zweite betrifft den Generator. Er soll abwechslungsreiche Welten liefern, sonst testet man keine Generalisierung. Er soll lösbare Welten liefern, sonst ist die Erfolgsquote wertlos. Und er soll angeben können, wie schwer eine Welt ist, sonst lässt sich das Training nicht steuern. Diese Forderungen ziehen in verschiedene Richtungen, und schon der letzte Punkt ist offen: Woran misst man die Schwierigkeit einer erzeugten Welt überhaupt?

Wie schwer die Aufgabe für den Agenten ist, entscheidet also der Generator. Daraus folgen zwei Fragen:

F1 *Wie lässt sich ein prozeduraler Weltgenerator so konstruieren, dass er abwechslungsreiche, nachweislich lösbare Welten mit präzise steuerbarer Schwierigkeit erzeugt und dabei als reproduzierbare Versuchsumgebung für Reinforcement Learning taugt?*

F2 *Kann ein Reinforcement-Learning-Agent in solchen, nur teilweise einsehbaren 2D-Welten eine Navigationsstrategie lernen, die auch auf unbekannten Welten funktioniert, ohne dass ihm der Weg zum Ziel vorberechnet wird?*

F2 legt sich absichtlich nicht auf eine Architektur fest. Ob der Agent dafür ein Gedächtnis braucht, ist Teil der Frage und nicht ihre Voraussetzung.

1.4 Zielsetzung und Erfolgskriterien

Daraus folgen zwei Sorten von Kriterien. Die erste sagt, wann die Umgebung brauchbar ist, die zweite, wann der Agent gut genug ist. Die zweite ist nur sinnvoll, wenn die erste erfüllt ist.

Z1: Anforderungen an Spiel und Generator. Diese Kriterien sind erfüllt oder nicht. Wir weisen sie an der jeweils angegebenen Stelle nach und führen sie in Abschnitt 7.1 zusammen.

- **Determinismus:** Gleicher Seed, gleiche Welt und gleicher Episodenverlauf, auch wenn Weltausschnitte in anderer Reihenfolge oder parallel erzeugt werden (Abschnitt 4.3).
- **Lösbarkeit:** Jede erzeugte Welt hat einen begehbaren Weg vom Start zum Ziel, nachgewiesen über alle verwendeten Seeds (Abschnitt 4.4).
- **Steuerbare Schwierigkeit:** Die Zielentfernung lässt sich vorgeben und wird eingehalten, gemessen am tatsächlichen Laufweg statt an der Luftlinie (Abschnitt 10.7).
- **RL-Tauglichkeit:** Anbindung an die Gymnasium-Schnittstelle, schnell genug für Läufe über Millionen Schritte auf einem normalen Rechner (Abschnitt 4.1).
- **Spielbarkeit:** Dieselbe Simulation ist für Menschen spielbar, und man kann einem trainierten Agenten zusehen (Abschnitt 4.1).

Z2: Schwellen für den Agenten.

- **Testset A** (Seeds 7000–7049): $\geq 70\%$ Erfolgsquote,
- **Holdout B** (Seeds 8000–8049): $\geq 60\%$ Erfolgsquote,
- pro Konfiguration mindestens 3 Trainingsläufe (Mittelwert $\pm$ Standardabweichung).

Die beiden Schwellen unter Z2 standen vor den Auswertungsläufen fest. Wir haben sie also nicht nachträglich passend gemacht. Woher sie kommen, steht in Abschnitt 6.5. Z1 ist anders entstanden: Das sind Konstruktionsziele, die sich während der Entwicklung geschärft haben; die steuerbare Schwierigkeit etwa bekam ihre heutige Form erst, nachdem sich das Luftlinienmaß als irreführend herausgestellt hatte.

Vorgehen. Die Welt entsteht in Stufen. Ein grobes Landschaftsfeld legt fest, welches Biom wo liegt; daraus belegen wir jedes einzelne Feld, wobei der Anteil an Hindernissen vom Biom abhängt. Die Erreichbarkeit des Ziels prüfen wir dabei nicht nachträglich, wir stellen sie durch die Konstruktion sicher: Eine Breitensuche wählt den Ausgang unter den Feldern, die vom Start aus begehbar erreichbar sind und die gewünschte Weglänge haben. Der Generator speichert nichts, jeder Ausschnitt hängt allein an Seed und Koordinaten. Dieselbe Simulation ist über eine Standardschnittstelle an den Lernstack angebunden und bleibt zugleich für Menschen spielbar.

Auf dieser Umgebung trainieren wir den Agenten mit einem rekurrenten Verfahren, das ein internes Gedächtnis besitzt. Er lernt stufenweise: erst auf kurzen Wegen zum Ziel, dann auf zunehmend längeren. Wir messen auf zwei Mengen von Welten, die im Training nie vorkommen und auch für die Auswahl des Modells gesperrt sind. Da einzelne Trainingsläufe im Reinforcement Learning stark schwanken können [6], beruht jede berichtete Zahl auf mehreren unabhängigen Läufen. Zusätzlich tritt der Agent gegen ungelernte Vergleichspolitiken an, die dieselbe Aufgabe unter demselben Protokoll lösen. Die Einzelheiten stehen in den Kapiteln 4, 5 und 6.

Eine Lücke in dieser Liste bestimmt am Ende das Ergebnis der Arbeit: Z1 verlangt lösbare Welten mit einstellbarer Entfernung, aber nicht, dass sie *schwer genug* sind. Die Umgebung kann also alle fünf Kriterien erfüllen und für F2 trotzdem untauglich sein. Genau das ist eingetreten (Abschnitt 7.4); ein passendes sechstes Kriterium für Folgearbeiten steht in Abschnitt 8.

1.5 Aufbau der Arbeit

Die Arbeit folgt dem üblichen Ablauf: erst die Begriffe, dann der Versuchsaufbau, dann die Messungen, zuletzt deren Bewertung. Durchgehend steht dabei zuerst das Spiel und dann der Agent, weil das eine die Voraussetzung des anderen ist. Die Implementierung belegt deshalb zwei Kapitel: Kapitel 4 behandelt das Spiel mit Simulationskern, Weltgenerator, Lösbarkeitsgarantie und Client, Kapitel 5 die darauf aufsetzende Lernumgebung mit POMDP-Charakter, Beobachtungsraum, Reward-Design, Trainings-Pipeline und Monitoring. Beide sind Bestandteile derselben Implementierung und keine getrennten Vorhaben; Kapitel 7 bewertet in gleicher Reihenfolge erst die Plattform und dann den Agenten. Zwei Anhänge ergänzen den Hauptteil: einer zur Reproduzierbarkeit, einer zum chronologischen Entwicklungsverlauf mit den gescheiterten Versuchen. Dass dieser zweite Anhang existiert, ist kein Nebenschauplatz, denn mehrere zentrale Erkenntnisse ergaben sich erst aus dem Vergleich fehlgeschlagener mit funktionierenden Läufen.

Zur Versionsbezeichnung. Die Kürzel *v11* und *v12* sind nötig, weil Ergebnisse verschiedener Entwicklungsstände nicht vergleichbar sind: Ändert sich die Weltgenerierung, erzeugt derselbe Seed eine andere Welt. *v11* ist die überarbeitete Umgebung, auf der alle Endergebnisse beruhen (Abschnitt 10.7), *v12* die darauf aufbauende Trainingskonfiguration (Tabelle 11).

Zur Arbeitsteilung. Die Arbeit entstand zu zweit; Tabelle 1 ordnet die Beiträge den Kapiteln zu. „Schwerpunktmäßig“ heißt, dass Entwurf und Umsetzung überwiegend bei der genannten Person lagen. Beide Teile hängen enger zusammen, als die Aufteilung vermuten lässt: Mehrere der wichtigsten Befunde betreffen weder den Generator noch den Agenten allein, sondern ihr Zusammenspiel.

Beitrag	Schwerpunkt	Kapitel
Spielkonzept, C++-Simulationskern, Spiellogik	Laurin Rößler	4, 4.1
Prozedurale Weltgenerierung und Lösbarkeitsgarantie	Laurin Rößler	2.1.2, 4.3
Grafischer Spiel-Client mit KI-Zuschaumodus	Laurin Rößler	4.1
Gym-Anbindung, Beobachtungsraum, Reward-Design	Florian Merlau	2.4.1, 5
Curriculum-Training, Hyperparameter, Fehleranalysen	Florian Merlau	6, 10
Evaluationsprotokoll, Referenzpolitiken, Auswertung	Florian Merlau	6.3, 7
Live-Monitoring, Werkzeuge, Experiment-Changelog	gemeinsam	4.1, 9

Tabelle 1: Arbeitsteilung nach Beiträgen und Kapiteln.

2 Grundlagen

Dieses Kapitel legt die Begriffe fest, mit denen die folgenden Kapitel arbeiten. Zwei Themen behandeln wir zuerst getrennt, weil sie sich getrennt verstehen lassen: die prozedurale Erzeugung von Spielwelten als Verfahren der Spieleentwicklung (Abschnitt 2.1) und das Reinforcement Learning unter partieller Beobachtbarkeit als Lernparadigma (Abschnitt 2.2). Abschnitt 2.3 führt beide zusammen und begründet, warum prozedural erzeugte Welten das geeignete Mittel sind, um die Generalisierung einer gelernten Politik zu prüfen. Darauf bauen die letzten beiden Abschnitte auf: die Steuerung des Lernprozesses (Abschnitt 2.4) sowie die Metriken und das statistische Handwerkszeug, mit denen wir das Ergebnis bewerten (Abschnitt 2.5). Eine vollständige Einführung in das RL-Gebiet liefern Sutton und Barto [7], in die prozedurale Inhaltserzeugung Shaker et al. [8].

2.1 Prozedurale Weltgenerierung

2.1.1 Begriff, Seed und Determinismus

Prozedurale Inhaltserzeugung (*Procedural Content Generation*, PCG) erzeugt Spielinhalte wie Karten, Level oder Gegenstände algorithmisch statt von Hand [8, 9]. Der Entwickler entwirft kein Level, sondern ein Regelwerk, das die Welt bei Bedarf berechnet. Das spart Speicher und Autorenaufwand, weil nur der Algorithmus abgelegt wird und nicht das fertige Level, und es liefert praktisch unbegrenzte Vielfalt [8].

Gesteuert wird die Erzeugung über einen *Seed*, den Startwert des Zufallszahlengenerators. Derselbe Seed erzeugt reproduzierbar dieselbe Welt, ein anderer Seed eine andere. Diese Reproduzierbarkeit ist keine technische Nebensache. Sie ist die Grundlage jedes kontrollierten Experiments mit prozeduralen Welten: Eine über ihre Seeds festgelegte Menge von Welten lässt sich jederzeit exakt wiederherstellen, womit eine feste, benannte Stichprobe definiert ist. Wie wir diese Eigenschaft nutzen, um Trainings- und Testmenge sauber zu trennen, zeigt Abschnitt 2.3.

2.1.2 Algorithmen der prozeduralen Weltgenerierung

Der vorige Abschnitt klärt, *was* prozedurale Generierung ist. Dieser klärt, *wie* eine begehbare Welt entsteht. Die Darstellung folgt dem für gitterbasierte Generatoren üblichen dreistufigen Aufbau aus Basisbelegung, Formgebung und Validierung [8] und bereitet die konkrete Umsetzung in Abschnitt 4.3 vor. Einen Überblick über das Feld geben Shaker et al. [8] sowie Togelius et al. [9].

Gitterbasierte Generatoren bauen die Welt selten am Stück, sondern in quadratischen Kacheln fester Größe, sogenannten *Chunks*. Ein Chunk entsteht erst, wenn der Agent ihn betritt. Hängt seine Berechnung ausschließlich von Seed und Koordinaten ab, spielt die Reihenfolge der Erzeugung keine Rolle und die Welt braucht keinen gespeicherten Zustand.

Rauschbasierte Basisbelegung. Die erste Stufe belegt jede Gitterzelle deterministisch aus einer Rauschfunktion und übersetzt deren Wert über Schwellwerte in Materialklassen. Das ist eines der Standardverfahren gitterbasierter PCG [8]. Zwei Varianten sind zu unterscheiden, und der Unterschied entscheidet über die Spielbarkeit. *Hash-Rauschen* berechnet den Wert jeder Zelle unabhängig aus ihren Koordinaten und dem Seed. Das Ergebnis ist unkorreliertes weißes Rauschen, aus dem ein Schwellwert verstreute *Einzelhindernisse* schneidet. *Wert-* oder *Gradientenrauschen* wertet dagegen nur ein gröberes Stützgitter aus und interpoliert dazwischen, klassisch mit Perlins Glättungsfunktion $3t^2 - 2t^3$ [10]. Benachbarte Zellen korrelieren dadurch, und es entstehen *zusammenhängende* Formationen. Verschiebt man die Eingabekoordinaten vor der Auswertung selbst durch ein Rauschfeld (*Domain Warping*), verlieren die Regionen ihre am Stützgitter ablesbare Rechteckigkeit. Beide Verfahren

sind pro Seed reproduzierbar und kommen ohne gespeicherten Zustand aus, taugen also für theoretisch unbegrenzte Welten.

Zelluläre Automaten als Formgebung. Rein rauschbasierte Belegungen wirken häufig zerfranst, weshalb meist eine Glättungsstufe über einen zellulären Automaten folgt. Er bewertet jede Zelle iterativ anhand der Zahl belegter Nachbarn, üblicherweise in der *Moore-Nachbarschaft* aus den acht direkt angrenzenden Zellen (horizontal, vertikal und diagonal). Geburts- und Überlebensregeln (*Birth/Survival Rules*) formen daraus fließende, natürlich wirkende Höhlen- und Klippenstrukturen: Eine freie Zelle wird belegt, sobald die Zahl belegter Nachbarn eine Geburtsschwelle erreicht; eine belegte Zelle bleibt nur belegt, solange sie eine Überlebensschwelle erfüllt. Wenige Iterationen genügen, um verstreutes Rauschen in zusammenhängende Strukturen zu überführen. Johnson et al. [11] demonstrieren das Verfahren an echtzeitfähig erzeugten Höhlenleveln.

Graphentheoretische Erreichbarkeitsanalyse. Prozedurale Welten haben ein Kernproblem: die Spielbarkeit (*Playability*), also die Vermeidung unlösbarer Level-Zustände (*Deadlocks*). Für die Analyse der Konnektivität lesen wir das Zellgitter als ungerichteten Graphen, in dem begehbare Zellen die Knoten und Nachbarschaftsbeziehungen die Kanten bilden. Welche Nachbarschaft dabei richtig ist, folgt aus dem Bewegungsmodell: Bei ausschließlich achsenparalleler Bewegung gilt die Vierer-Nachbarschaft (*von Neumann*) und nicht die für die Glättung verwendete Moore-Nachbarschaft, denn eine über die Diagonale hergestellte Verbindung könnte der Agent gar nicht begehen. Die Breitensuche (*Breadth-First Search*, BFS) traversiert den Graphen ebenenweise vom Startknoten aus [12]. Interessiert dabei nur die erreichbare Menge und nicht die Distanz, spricht man von *Flood-Fill*. Die BFS beantwortet damit zwei Fragen zugleich: ob ein Zielknoten überhaupt erreichbar ist und wie der kürzeste Pfad auf einem ungewichteten Gitter verläuft [12]. Beide Eigenschaften kehren später wieder (Abschnitte 2.4.1 und 2.5.1).

Heuristische Sicherheitsnetze und Metriken. Spielbarkeit sicherzustellen und unspielbare Level zu reparieren ist ein eigenständiger Baustein der PCG-Praxis [8]. Weil die exakte graphenbasierte Validierung bei großen Weltausschnitten teuer wird, kombiniert man sie häufig mit billigeren Prüfungen. Als Vorabschätzung dient die *Manhattan-Distanz*, die Distanz zweier Punkte auf einem orthogonalen Gitter unter achsenparalleler Bewegung:

$$D_{\text{Manhattan}} = |x_1 - x_2| + |y_1 - y_2|. \quad (1)$$

Sie ist in konstanter Zeit berechenbar und unterschätzt den tatsächlichen Laufweg, sobald Hindernisse dazwischenstehen. Als Ersatz für eine Erreichbarkeitsprüfung taugt sie deshalb nicht, wohl aber als untere Schranke und als Abbruchkriterium. Scheitert die Prüfung, greift ein zweites Sicherheitsnetz: Eine pragmatische *Fallback-Generierung* (*Carving*) räumt entlang direkter Geometrieachsen einen Korridor von Start zu Ziel frei, typischerweise erst in der einen, dann in der anderen Achse.

Beide Mittel wirken, aber sie sind nicht folgenlos. Ein gecarvter Korridor ist per Konstruktion eine gerade, hindernisfreie Linie und damit eine strukturelle Auffälligkeit, die das erzeugte Weltbild systematisch von einer gleichmäßig verteilten Hindernisstruktur wegzieht. Welche Verfahren der vorliegende Generator implementiert und welche in der berichteten Konfiguration tatsächlich laufen, führt Abschnitt 4.3 auf. Was eine solche Auffälligkeit für eine lernende Politik bedeutet, greift die Evaluation auf (Abschnitt 7).

2.2 Lernen unter partieller Beobachtbarkeit

2.2.1 Reinforcement Learning und der Markov-Entscheidungsprozess

Reinforcement Learning (RL) beschreibt Lernen durch Interaktion: Der Agent wählt in einem Zustand eine Aktion, kassiert eine Belohnung (Reward) und landet in einem Folgezustand [7]. Formal ist das ein Markov-Entscheidungsprozess (MDP), ein Tupel (S, A, P, R, γ) mit Zustandsmenge S , Aktionsmenge A , Übergangswahrscheinlichkeit $P(s' | s, a)$, Rewardfunktion $R(s, a)$ und Diskontfaktor $\gamma \in [0, 1)$. Gesucht ist eine Politik $\pi(a | s)$, die den erwarteten diskontierten Return $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$ maximiert. Die Zustandswertfunktion $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$ gibt den erwarteten Return ab einem Zustand an. Der Diskontfaktor γ gewichtet spätere Rewards geringer; wir arbeiten mit $\gamma = 0,999$.

Abbildung 1 zeigt diesen Kreislauf. Er ist die Grundfigur des gesamten Verfahrens. Alles, was wir in dieser Arbeit trainieren, ist eine Variante dieser Schleife: Die Umgebung heißt *Stoneforge*, die Aktionen sind die vier Bewegungsrichtungen, der Reward stammt aus Abschnitt 5.4.

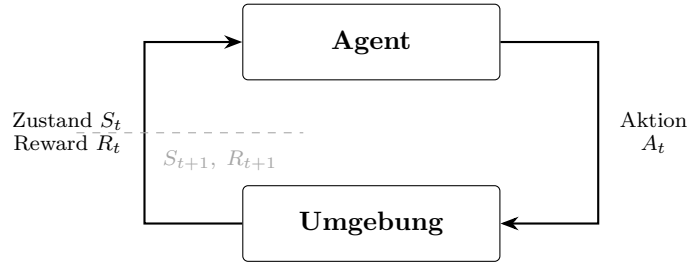


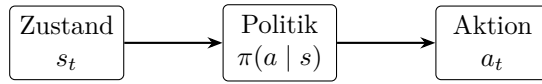
Abbildung 1: Die Agent-Umwelt-Interaktion im Markov-Entscheidungsprozess; die gestrichelte Linie markiert den Übergang zum nächsten Zeitschritt. Eigene Darstellung in Anlehnung an Sutton und Barto [7, Abb. 3.1].

2.2.2 Partielle Beobachtbarkeit (POMDP)

Sieht der Agent nicht den vollständigen Zustand, sondern nur eine Beobachtung, liegt ein *Partially Observable* MDP (POMDP) vor [13]. Für die Beobachtung gilt die Markov-Eigenschaft dann nicht mehr: Dieselbe lokale Beobachtung kann zu ganz verschiedenen globalen Zuständen gehören. Eine reaktive Politik, die nur die aktuelle Beobachtung nutzt, ist deshalb im Allgemeinen nicht optimal. Abhilfe schafft ein internes Gedächtnis, das vergangene Beobachtungen zu einem *Belief-State* verdichtet [14]. Abbildung 2 stellt beide Fälle gegenüber: Im MDP greift die Politik direkt auf den Zustand zu, im POMDP schiebt sich eine Beobachtungsfunktion dazwischen, und die Politik muss aus der Vorgeschichte erst rekonstruieren, wo sie vermutlich steht.

Eine zweite Konsequenz trägt diese Arbeit weit: Unter partieller Beobachtbarkeit kann eine *stochastische* gedächtnislose Politik jede deterministische gedächtnislose deutlich übertreffen [15]. Auf die Reichweite dieser Aussage kommt es an. Sie gilt für *gedächtnislose* Politiken. Ist der Belief-State hinreichend genau rekonstruiert, existiert wieder eine optimale deterministische Politik. Ein Leistungsabstand zwischen Sampling- und Argmax-Auswertung einer *rekurrenten* Politik ist damit gerade kein notwendiges Merkmal der Aufgabe, sondern ein Indikator für einen unvollständigen Belief-State. Genau so prüfen wir ihn in der Evaluation (Abschnitt 7). *Stoneforge* ist ein POMDP, denn der Agent sieht nur einen lokalen 15×15 -Ausschnitt (siehe Abschnitt 5); Abbildung 3 stellt den Unterschied schematisch dar. Neben diesem unmittelbar sichtbaren lokalen Ausschnitt gibt es eine zweite, weniger offensichtliche Quelle partieller Beobachtbarkeit. Sie entsteht erst aus der Forderung nach Generalisierung

MDP: vollständig beobachtbar



POMDP: nur Ausschnitt sichtbar

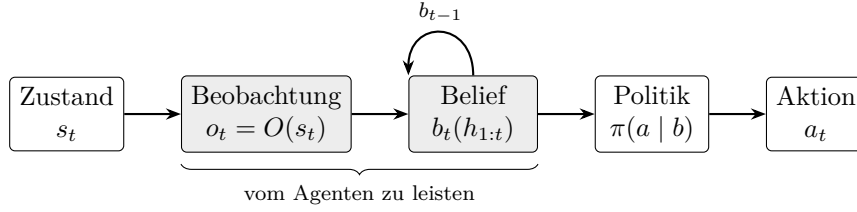


Abbildung 2: Vollständige (oben) gegenüber partieller Beobachtbarkeit (unten). Grau hinterlegt sind die beiden Schritte, die im POMDP zusätzlich anfallen: die Beobachtung $o_t = O(s_t)$ und die Bildung eines *Beliefs* aus ihr und der Vorgeschichte. Eigene Darstellung; Formalismus nach Kaelbling et al. [13].

auf unbekannte Welten und wird in Abschnitt 2.3 behandelt, wo beide Themen zusammenlaufen.

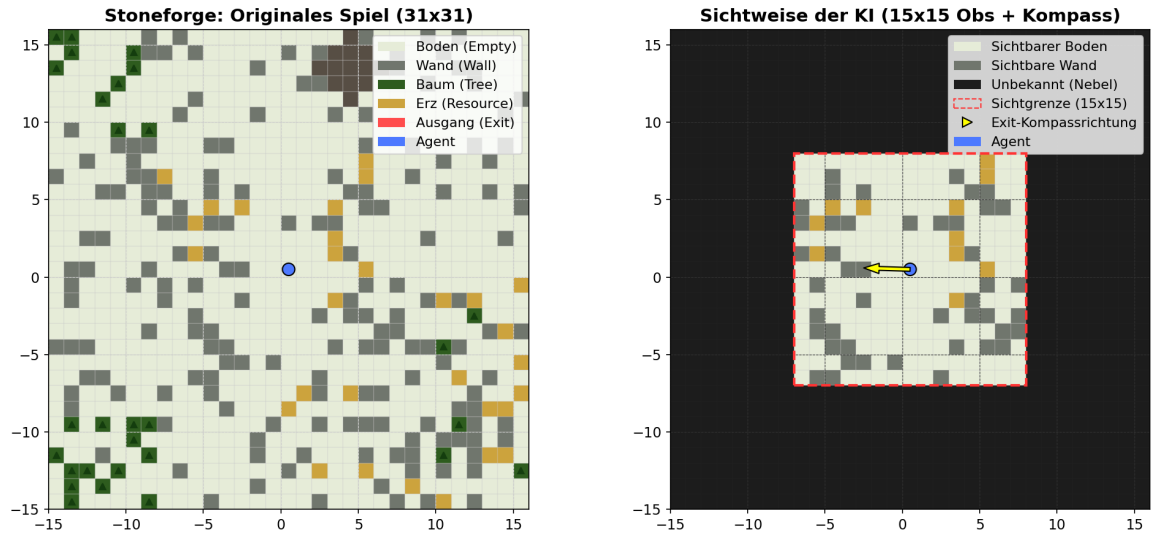


Abbildung 3: Partielle Beobachtbarkeit am Beispiel Stoneforge. Links der vollständige Weltzustand (31×31 -Ausschnitt), rechts die Beobachtung des Agenten: nur das rot umrandete 15×15 -Sichtfeld ist bekannt. Der Pfeil zeigt die Kompassrichtung zum Exit, nicht den Weg.

2.2.3 Wertbasierte Verfahren: Q-Learning und DQN

Wertbasierte Verfahren lernen die Aktionswertfunktion $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$ und leiten die Politik daraus ab, indem sie die Aktion mit dem höchsten Q -Wert wählen. Q-Learning aktualisiert Q über die Bellman-Gleichung. Deep Q-Networks (DQN) approximieren Q mit einem neuronalen Netz und stabilisieren das Training über einen Replay-Buffer und ein Target-Netz [1]. DQN verarbeitet nur diskrete Aktionsräume. Warum wir es früh eingesetzt und später zu PPO gewechselt haben, begründet Abschnitt 6.

2.2.4 Policy-Gradient- und Actor-Critic-Verfahren

Policy-Gradient-Verfahren optimieren die Politik π_θ direkt über den Gradienten des erwarteten Returns. Actor-Critic-Verfahren stellen ihnen eine gelernte Wertfunktion (Critic) zur Seite, die als Baseline die Varianz der Gradientenschätzung senkt. Der Vorteil (Advantage) $A(s, a) = Q(s, a) - V(s)$ misst, wie viel besser eine Aktion gegenüber dem Durchschnitt ist; die Generalized Advantage Estimation (GAE) schätzt ihn über mehrere Zeitschritte mit einem Parameter λ [16], im Projekt $\lambda = 0,95$. Advantage Actor-Critic (A2C) ist die synchrone Variante von A3C [17]. Wie gut der Critic gelernt hat, verrät die *explained variance* (EV): der Anteil der Varianz der tatsächlichen Returns, den die Wertschätzung erklärt. $EV \approx 1$ steht für eine verlässliche Wertfunktion, $EV \approx 0$ dafür, dass der Critic praktisch nichts gelernt hat. Diese Größe ist unsere wichtigste Diagnosemetrik des Trainings.

2.2.5 Proximal Policy Optimization (PPO)

PPO ist das Hauptverfahren dieser Arbeit [18]. Es ist ein Actor-Critic-Verfahren, das die Politik pro Update nur begrenzt verändert, um Instabilität durch zu große Schritte zu vermeiden. Dazu schneidet es das Wahrscheinlichkeitsverhältnis $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{alt}}}(a_t | s_t)$ in der Zielfunktion ab (*clipping*):

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)] .$$

Abbildung 4 zeigt die Wirkung des Clippings an einem einzelnen Zeitschritt. War die Aktion besser als erwartet ($A_t > 0$), lohnt es sich, ihre Wahrscheinlichkeit zu erhöhen, aber nur bis zum Faktor $1 + \epsilon$; darüber verläuft die Zielfunktion flach und der Anreiz für einen größeren Schritt fällt weg. War sie schlechter als erwartet ($A_t < 0$), gilt dasselbe spiegelbildlich bei $1 - \epsilon$. Genau das verhindert die zu großen Aktualisierungen, an denen einfache Policy-Gradient-Verfahren scheitern.

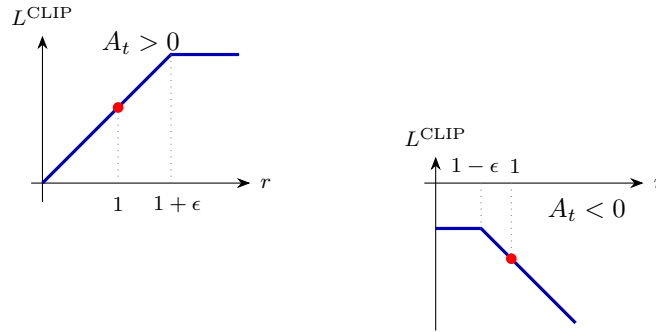


Abbildung 4: Ein einzelner Term der abgeschnittenen Zielfunktion L^{CLIP} über dem Wahrscheinlichkeitsverhältnis $r_t(\theta)$, links für positiven, rechts für negativen Advantage. Der rote Punkt markiert den Startpunkt der Optimierung ($r = 1$). Eigene Darstellung in Anlehnung an Schulman et al. [18, Abb. 1].

Hinzu kommt ein Entropie-Term, der Exploration fördert; sein Gewicht (`ent_coef`) steuert, wie stochastisch die Politik bleibt. Trainiert wird in Zyklen: Erst sammeln die parallelen Umgebungen einen Rollout von n_{steps} Schritten, dann optimiert PPO diesen in Minibatches der Größe `batch_size` über n_{epochs} Durchläufe. Wir verwenden die Implementierung aus Stable-Baselines3 [19].

Eine gelernte stochastische Politik lässt sich auf zwei Arten ausführen: *stochastisch*, indem die Aktion aus der Verteilung $\pi(a | s)$ gezogen wird (Sampling), oder *deterministisch* über die stets wahrscheinlichste Aktion (Argmax). Dasselbe trainierte Modell kann in beiden Betriebsarten unterschiedlich gut abschneiden. Den Abstand zwischen beiden Messwerten nennen wir

durchgängig **Det/Stoch-Gap**. Er ist einer der Hauptgegenstände dieser Arbeit, weil er sich als groß und hartnäckig erwiesen hat (Abschnitt 7.6). Anschaulich gesprochen: Der Agent löst die Aufgabe zuverlässiger, wenn er würfelt, als wenn er stets seine eigene beste Aktion wählt. Dass das unter partieller Beobachtbarkeit möglich ist, folgt aus Abschnitt 2.2.2. Dass es bei einer Politik *mit* Gedächtnis trotzdem erklärungsbedürftig bleibt, ebenfalls.

2.2.6 Rekurrente Politiken (LSTM)

Alle bisher genannten Verfahren laufen auch mit einem gewöhnlichen vorwärtsgerichteten neuronalen Netz, einem *Multilayer Perceptron* (MLP). Ein solches Netz bildet jede Beobachtung unabhängig auf eine Aktion ab und besitzt kein Gedächtnis. Es kann nicht unterscheiden, ob es eine Stelle zum ersten oder zum zehnten Mal sieht. Unter partieller Beobachtbarkeit ist das eine harte Einschränkung (Abschnitt 2.2.2); die Bezeichnung *MLP* steht im Folgenden durchgängig für diese gedächtnislose Variante.

Um in einem POMDP ein Gedächtnis bereitzustellen, erweitern wir die Politik um ein rekurrentes Netz. Long Short-Term Memory (LSTM) ist eine rekurrente Architektur, die Information über viele Zeitschritte hinweg hält [20]. Ihr Zustand approximiert den Belief-State und fasst die bisher gesehenen Beobachtungen zusammen. Rekurrentes Netz und wertbasiertes RL wurden als DRQN kombiniert [14]; wir nutzen die policy-gradient-Variante RecurrentPPO (PPO mit LSTM-Politik) aus *sb3-contrib*, mit einer LSTM-Größe von 256.

Wie das Gedächtnis wirkt, zeigt Abbildung 5 in der über die Zeit „entrollten“ Darstellung: Der verborgene Zustand h_t wandert von Schritt zu Schritt weiter und sammelt Information, die in der einzelnen Beobachtung fehlt. Für Stoneforge ist das genau die Information „hier war ich schon“ bzw. „diese Wand bin ich bereits nach Norden abgelaufen“.

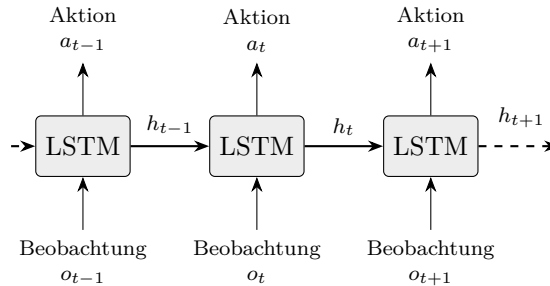


Abbildung 5: Über die Zeit entrollte rekurrente Politik: Neben der aktuellen Beobachtung o_t erhält jeder Zeitschritt den verborgenen Zustand h_{t-1} des Vorschriffs. Eigene Darstellung; rekurrente Politiken im RL nach Hausknecht und Stone [14].

Trainiert wird ein solches Netz per *Backpropagation Through Time* (BPTT): Das Netz wird über eine Sequenz fester Länge entrollt, der Gradient läuft über alle Zeitschritte zurück. Die Sequenzlänge ist damit ein Hyperparameter mit zwei Seiten. Zu kurze Sequenzen begrenzen, wie weit zurück ein Zusammenhang überhaupt gelernt werden kann; zu lange Sequenzen lassen Gradienten verschwinden und bremsen das Training. Dazu kommt die Frage, mit welchem verborgenen Zustand eine Sequenz startet. Kapturowski et al. zeigen für rekurrente Verfahren, dass die Behandlung dieses Zustands die Leistung bei sonst unveränderter Architektur erheblich verschiebt, und schlagen unter anderem eine Einlaufphase (*Burn-in*) vor [21]. Bei der Auswertung stellt sich dieselbe Frage: Wer den Zustand zwischen Episoden nicht zurücksetzt, misst eine andere Politik als die trainierte.

2.3 Prozedurale Welten im Reinforcement Learning

Die beiden vorangegangenen Abschnitte behandeln zwei eigenständige Themen: die prozedurale Erzeugung von Welten und das Lernen einer Politik unter partieller Beobachtbarkeit.

Hier laufen sie zusammen.

Der Ausgangspunkt ist ein bekanntes Problem des maschinellen Lernens. Trainiert und prüft man ein Modell stets auf denselben Daten, lernt es womöglich die Antworten auswendig (*Overfitting*), statt eine übertragbare Regel zu bilden. Im Reinforcement Learning entspricht ein einzelnes festes Level solchen festen Daten; die Politik memoriert dann die konkrete Karte, ohne allgemeine Navigation zu erlernen [2, 3]. Bekommt dagegen jede Episode eine neue, per Seed erzeugte Welt, entzieht sich die Aufgabe dem Auswendiglernen: Der Agent sieht jede einzelne Karte zu selten, um sie zu behalten, und muss eine Strategie bilden, die über die Trainingswelten hinaus trägt. Prozedurale Generierung ist deshalb ein etabliertes Mittel, um Generalisierung im RL zu prüfen und Overfitting auf einzelne Level zu verhindern [2, 4].

Die Seed-Reproduzierbarkeit aus Abschnitt 2.1.1 liefert dafür das Werkzeug: Sie erlaubt disjunkte Seed-Mengen und damit eine strenge Trennung von Trainings- und Testmenge. Die Evaluation auf trainingsfremden Seeds (Testset A und Holdout B, Abschnitt 6.3) folgt genau diesem Prinzip. Gemessen wird nicht, wie gut eine Politik gelernte Welten wiederfindet, sondern wie gut sie auf nie gesehene überträgt.

Die Verbindung reicht tiefer als bis zum Versuchsaufbau. Die Forderung, auf unbekannten Welten zu funktionieren, erzeugt selbst partielle Beobachtbarkeit (Abschnitt 2.2.2): Der Politik ist zur Laufzeit unbekannt, in welcher der möglichen Welten sie steckt. Ghosh et al. zeigen, dass daraus auch dann ein POMDP wird, wenn jede einzelne Welt für sich vollständig beobachtbar wäre, und nennen das *epistemic POMDP* [22]. Die Unsicherheit liegt hier über der Identität der Welt und nicht über dem Zustand innerhalb einer bekannten Welt. Mehr Gedächtnis über die laufende Episode löst sie deshalb nicht auf. Auch die optimale Politik eines epistemic POMDP ist im Allgemeinen stochastisch, was für die spätere Untersuchung des Det/Stoch-Gaps zählt. Prozedurale Generierung und Lernen unter partieller Beobachtbarkeit sind für diese Arbeit also keine zufällig kombinierten Bausteine. Die Generalisierungsanforderung verknüpft sie ursächlich.

2.4 Steuerung des Lernprozesses

2.4.1 Reward Shaping (PBRs)

Zusätzliche Belohnungen können das Lernen beschleunigen, dürfen die optimale Politik aber nicht verändern. Potential-Based Reward Shaping (PBRs) leistet genau das: Der Zusatzreward hat die Form $F(s, s') = \gamma \Phi(s') - \Phi(s)$ mit einer Potentialfunktion Φ . Ng et al. zeigen, dass diese Form policy-invariant ist, die optimale Politik also unangetastet lässt [23]. Bei uns hängt Φ an der BFS-Distanz zum Exit: Die Breitensuche (BFS) liefert die Länge des kürzesten begehbaren Wegs vom Agenten zum Exit in Schritten. Anders als die Luftlinie berücksichtigt sie Wände und Umwege.

2.4.2 Curriculum Learning

Curriculum Learning trainiert mit ansteigender Aufgabenschwierigkeit statt in zufälliger Reihenfolge [24]. Wir erhöhen die Exit-Distanz stufenweise; der Übergang zwischen den Stufen läuft leistungsbasiert, eine Stufe gilt ab einer Ziel-Erfolgsquote als bestanden. Davon zu unterscheiden ist die Steuerung auf Ebene einzelner Welten. *Prioritized Level Replay* (PLR) wiederholt gezielt jene Level, an denen eine Politik noch am meisten lernen kann, statt sie gleichverteilt zu ziehen, und verbessert so Sample-Effizienz und Generalisierung [25]. Auf die Auswahlrichtung kommt es an: Wiederholt werden Level mit hohem Lernpotenzial, gerade nicht die bereits gemeisterten.

2.5 Messen und Bewerten

Die bisherigen Abschnitte behandeln, wie Welten entstehen und wie Politiken lernen. Dieser behandelt, woran sich beides beurteilen lässt. Das ist kein Nebenschauplatz: Zwei der drei Befunde dieser Arbeit betreffen nicht das Verhalten des Agenten, sondern die Frage, was eine Messung überhaupt aussagt.

2.5.1 Metriken einer Navigationsaufgabe

Erfolgsquote. Die naheliegende Metrik ist der Anteil gelöster Welten. Formal ist sie der Schätzwert $\hat{p} = k/n$ eines Bernoulli-Parameters: Jede Welt ist ein Versuch, der gelingt oder nicht. Daraus folgen zwei Eigenschaften unmittelbar. Erstens ist die Erfolgsquote nur zusammen mit dem Abbruchkriterium definiert, denn ob eine Welt als gelöst zählt, hängt davon ab, wie lange der Agent suchen darf. Das Episodenlimit gehört damit zur Metrik und nicht zu ihrem Umfeld. Zweitens ist $\hat{p}$ bei kleinem n ungenau. Wie ungenau, beziffert Abschnitt 2.5.2.

Pfادهffizienz. Die Erfolgsquote sagt nichts darüber, *wie* eine Welt gelöst wurde. Dafür dient die Pfادهffizienz

$$\eta = \frac{d^*(s_0)}{T}, \quad (2)$$

das Verhältnis aus kürzestem Weg d^* vom Startzustand und tatsächlich benötigten Schritten T . Es gilt $0 < \eta \leq 1$; $\eta = 1$ bedeutet optimale Navigation, $\eta = 0,05$ einen zwanzigfachen Umweg. Zwei Feinheiten sind zu beachten, sonst liest man sie als Rechenfehler.

Zum einen macht es einen Unterschied, ob über die *Quotienten* der Einzelepisoden gemittelt oder der Quotient der Mittelwerte gebildet wird. Beide Größen fallen auseinander, sobald die Schrittzahl schief verteilt ist. Da $1/T$ konvex in T ist, liefert die Jensen-Ungleichung

$$\mathbb{E}\left[\frac{d^*}{T}\right] \geq \frac{d^*}{\mathbb{E}[T]}, \quad (3)$$

Das Mittel der Quotienten liegt also systematisch höher. Welche Variante berichtet wird, gehört in den Text.

Zum anderen ist η nur auf erfolgreichen Episoden definiert. Vergleicht man zwei Politiken mit unterschiedlicher Erfolgsquote, bewertet man die schwächere auf der leichteren Teilmenge. Die Verzerrung arbeitet also zugunsten der schwächeren Politik. Wer das beim Interpretieren vergisst, liest eine Stärke, wo keine ist.

Betriebsarten. Eine gelernte stochastische Politik lässt sich auf zwei Arten ausführen: durch Ziehen aus $\pi(a | s)$ oder durch Wahl der wahrscheinlichsten Aktion (Argmax). Dazwischen liegt die temperaturskalierte Auswertung, die die Logits vor dem Softmax durch einen Parameter T teilt; $T \rightarrow 0$ ergibt Argmax, $T = 1$ das ungeänderte Sampling. Dieselbe Politik kann in beiden Betriebsarten unterschiedlich abschneiden, weshalb jede Angabe die gemessene Betriebsart nennen muss. Dass ein solcher Unterschied unter partieller Beobachtbarkeit theoretisch zu erwarten ist, folgt aus Abschnitt 2.2.2.

2.5.2 Streuung, Konfidenz und die Zahl der Läufe

Die Messunsicherheit dieser Experimente hat zwei voneinander unabhängige Quellen. Wer sie vermischt, misst Unsinn.

Unsicherheit innerhalb eines Laufs. Prüfen wir ein festes Modell auf n Welten, ist die Erfolgsquote ein Bernoulli-Anteil mit Standardfehler $\sqrt{p(1-p)/n}$. Für $n = 50$ und den ungünstigsten Fall $p = 0,5$ ergibt das ein 95 %-Konfidenzintervall von rund ± 14 Prozentpunkten. Unterschiede unterhalb dieser Größe lassen sich aus einer einzelnen Messung nicht interpretieren, egal wie sorgfältig sie erhoben wurde.

Unsicherheit zwischen Läufen. Zwei Trainingsläufe derselben Konfiguration unterscheiden sich allein durch Zufallsinitialisierung und Reihenfolge der Erfahrungen. Henderson et al. zeigen, dass diese Streuung im Deep Reinforcement Learning so groß ausfällt, dass Aussagen aus wenigen Läufen systematisch in die Irre führen. Sie fordern deshalb Mittelwert und Standardabweichung über mehrere unabhängige Läufe und ausdrücklich *nicht* den besten Lauf [6]. Für die Aggregation über m Läufe gilt die *Stichproben*-Standardabweichung,

$$s = \sqrt{\frac{1}{m-1} \sum_{i=1}^m (x_i - \bar{x})^2}, \quad (4)$$

mit dem Nenner $m-1$ statt m (Bessel-Korrektur), denn die Läufe sind eine Stichprobe aus allen möglichen Läufen und nicht die Grundgesamtheit. Das zugehörige Konfidenzintervall folgt bei kleinem m der t -Verteilung mit $m-1$ Freiheitsgraden, nicht der Normalverteilung.

Vergleich zweier Gruppen. Wollen wir zwei Chargen von Läufen vergleichen, etwa um zu prüfen, ob ein Unterschied eine systematische Ursache hat oder bloß Streuung ist, eignet sich der Welch-Test. Er ist die Variante des t -Tests, die ungleiche Varianzen und ungleiche Gruppengrößen zulässt, und damit die konservative Wahl, solange über die Varianzen nichts bekannt ist. Ein nicht signifikanter Ausgang belegt keine Gleichheit. Er besagt nur, dass die Daten einen Unterschied nicht belegen.

Vorabfestlegung von Schwellen. Formuliert man Erfolgskriterien erst nach Kenntnis der Ergebnisse oder passt sie nachträglich an, verlieren sie ihre Aussagekraft. Kerr nennt dieses Vorgehen *HARKing* (*Hypothesizing After the Results are Known*) [26]. Das Gegenmittel ist billig: die dokumentierte Festlegung vor der Messung. Allgemeiner fassen Pineau et al. die Anforderungen an reproduzierbare Berichte in einer Checkliste zusammen, die neben der Zahl der Läufe auch den durchsuchten Hyperparameterbereich und die Trennung der Datenmengen umfasst [27].

2.5.3 Referenzpolitiken als Bezugspunkt

Ein Messwert ohne Bezugspunkt ist nicht interpretierbar. Ob eine Erfolgsquote von 65 % eine gelernte Fähigkeit ausdrückt, lässt sich nur beantworten, wenn bekannt ist, was ohne Lernen erreichbar wäre. Dafür dienen *ungelernte Referenzpolitiken*, die dieselbe Beobachtung erhalten und unter demselben Protokoll laufen. Zwei Typen sind zu unterscheiden.

Eine **Zufallspolitik** wählt gleichverteilt aus dem Aktionsraum und markiert die absolute Untergrenze. Sie beantwortet, ob überhaupt etwas gelernt wurde, nicht aber, ob das Gelernte über triviale Strategien hinausgeht.

Eine **Heuristik** nutzt einen Teil der Beobachtung nach einer fest programmierten Regel, ohne Parameter und ohne Training. Sie ist der schärfere Bezugspunkt, weil sie ausdrückt, welcher Anteil des Ergebnisses schon ohne Lernen zu haben ist. Übertrifft eine solche Heuristik den trainierten Agenten, liegt das Problem nicht beim Agenten, sondern bei der Aufgabenstellung: Sie verlangt die Fähigkeit gar nicht, deren Nutzen wir messen wollen.

Damit bekommt die Referenzpolitik eine zweite Rolle, die über den Vergleich hinausgeht. Sie taugt als *Kalibrierinstrument*, das die Schwierigkeit einer Umgebung vor jedem Training bestimmt, in Sekunden statt in Trainingsstunden. Kirk et al. argumentieren in dieselbe

Richtung, wenn sie einen rein prozeduralen Ansatz beim Benchmark-Entwurf als hinderlich für den Fortschritt der Generalisierungsforschung bezeichnen: Ohne Instrument, das trennt, warum ein Verfahren gelingt oder scheitert, sagt der Messwert allein wenig [3].

3 Einordnung in verwandte Arbeiten

Stoneforge liegt strukturell zwischen drei etablierten Benchmarks: MiniGrid (teilbeobachtbare Grid-Navigation, PPO+LSTM als Standard-Baseline) [28], Crafter (prozedural generiertes 2D-Survival-Spiel mit Mining und Crafting) [29] und Procgen (Generalisierung über prozedurale Level) [2]. Prozedurale Generierung als Mittel, Generalisierung zu erzwingen, hat darüber hinaus eine eigene Forschungstradition [9, 4].

Weltgenerierung und der Preis des eigenen Generators. Die Verfahren, auf denen der hier verwendete Generator beruht, sind etabliert: rauschbasierte Basisbelegung, zelluläre Automaten zur Formgebung und graphenbasierte Erreichbarkeitsprüfung sind Standardbausteine der prozeduralen Inhaltserzeugung [8, 11]. Der Beitrag dieser Arbeit liegt insofern nicht in einem neuen Generierungsverfahren, sondern in ihrer Kombination zu einer Umgebung, die zugleich *spielbar* und als Messinstrument *kontrollierbar* ist.

Diese Kontrollierbarkeit ist der Grund, keinen der genannten Benchmarks zu verwenden. MiniGrid, Crafter und Procgen liefern fertige, gut untersuchte Generatoren, deren Parametrisierung aber nicht Gegenstand des Experiments ist: Wandanteil, Korreliertheit der Hindernisse und das Maß, in dem die Aufgabenschwierigkeit angegeben wird, lassen sich dort nicht gezielt verstellen. Der Preis eines eigenen Generators ist die fehlende Einbettung in publizierte Vergleichswerte (siehe unten); der Gewinn zeigt sich in Abschnitt 7.4, wo sich die entscheidende Frage der Arbeit als eine Frage an den Generator und nicht an den Agenten herausstellt, und in Abschnitt 8, wo die Antwort darauf ein Eingriff in die Weltgenerierung ist, der mit einem fremden Generator gar nicht möglich wäre.

Lösbarkeitsgarantie im Vergleich der Benchmarks. Wie die genannten Umgebungen ihre Spielbarkeit (*Playability*) sichern, unterscheidet sich grundlegend und ist für die Einordnung des eigenen Ansatzes aufschlussreich. MiniGrid erzeugt seine Aufgaben nach festen Platzierungsregeln, die Start, Ziel und Hindernisse so anordnen, dass ein lösbares Layout *konstruktiv* entsteht; eine nachgelagerte Erreichbarkeitsprüfung entfällt [28]. Procgen steuert die Schwierigkeit über einen Verteilungsmodus („easy“/„hard“) und die Zahl der Trainingslevel; Lösbarkeit ist dort eine Eigenschaft der von Hand entworfenen Spielgeneratoren, nicht einer je Level ausgewerteten formalen Garantie [2, 5]. Crafter schließlich kennt gar kein einzelnes Ziel, sondern bewertet über erreichte *Achievements* in einer offenen Welt [29]. Der hier verfolgte Weg – eine *explizite* BFS-basierte Exit-Platzierung je erzeugter Welt (Abschnitt 4.4) – ist damit strenger: Er garantiert die Lösbarkeit nicht nur im Verteilungsmittel, sondern für jede einzelne evaluierte Welt und macht die Zielentfernung als kontrollierte Variable überhaupt erst nutzbar.

Die Schwierigkeit steckt in der Struktur der Hindernisse. Ein zweiter, in der Rückschau wichtiger Punkt betrifft nicht die Lösbarkeit, sondern die *Art* der Hindernisse. In der PCG-Literatur ist es gerade die Glättungsstufe über zelluläre Automaten, die aus verstreutem Rauschen zusammenhängende, konkave Strukturen formt: Höhlen, Sackgassen, U-förmige Wände [11, 8]. Erst solche Strukturen bringen eine rein reaktive, gedächtnislose Navigation zuverlässig zum Scheitern. Ohne diese Stufe verbleiben verstreute Einzelhindernisse (Tabelle 6), die ein Kompass-Folger auch ohne Gedächtnis weitgehend umgeht. Das erklärt die hohe Erfolgsquote der ungelerten Referenzpolitik in Abschnitt 7.4 unmittelbar. Eine frühere Auseinandersetzung mit genau dieser Stelle der Literatur hätte den Zusammenhang nahegelegt; sichtbar wurde er erst durch die nachträgliche Baseline-Kalibrierung.

Der Umkehrschluss trägt allerdings nicht, und das ist der lehrreichere Teil. Die Glättungsstufe in diesem Generator einzuschalten härtet die Welt nicht, es plant sie: Wanddichte 0,037 statt 0,238, Umwegfaktor 1,001 statt 1,10 (Tabelle 7). Was in der Literatur Höhlen

formt, radiert hier die Wände weg, weil die Ausgangsdichte unter der Geburtsschwelle liegt. Publierte PCG-Bausteine sind also nicht parameterfrei übertragbar. Welche Struktur ein zellulärer Automat erzeugt, entscheidet der Betriebspunkt der Stufe davor, nicht der Automat. Die methodische Lehre bleibt davon unberührt und ist in den Ausblick übernommen: die Schwierigkeit einer prozeduralen Umgebung mit einer ungelerten Heuristik *kalibrieren*, *bevor* das eigentliche Training beginnt (Abschnitt 8).

Gedächtnis in teilbeobachtbaren Umgebungen. Die Standardantwort auf partielle Beobachtbarkeit ist eine rekurrente Politik: DRQN erweiterte DQN um ein LSTM [14]; R2D2 zeigte, dass die korrekte Behandlung des rekurrenten Zustands (gespeicherte States im Replay, Burn-in) die Leistung bei ansonsten unveränderter Architektur erheblich beeinflusst [21]. Dieser Befund spiegelt sich in dieser Arbeit auf der Eval-Seite wider (korrekt geführter LSTM-State, Abschnitt 6). Für on-policy-Verfahren untersuchen Pleines et al. Implementierungsdetails rekurrenter PPO-Politiken und stellen Umgebungen vor, deren Gedächtnisanforderungen über reine Kapazität hinausgehen [30]. Der POPGym-Benchmark vergleicht 13 Gedächtnis-Architekturen systematisch; dort schneidet das einfache GRU insgesamt am besten ab; aufwändigere Gedächtnismodelle sind nicht automatisch besser [31]. Der hier verwendete RecurrentPPO-Ansatz entspricht dieser Standard-Baseline, und der Befund der eigenen E3-Ablation (LSTM 512 schlägt 256 nicht, Abschnitt 7.6) fügt sich in dieses Bild. Strukturierte Alternativen wie das In-Context-Verfahren AMAGO [32] werden im Ausblick diskutiert.

Curriculum und Exploration. Curriculum Learning geht auf Bengio et al. zurück [24]; das hier verwendete leistungsbasierte Gating ist eine einfache Instanz, Prioritized Level Replay (PLR) [25] die naheliegende Weiterentwicklung auf Level-Ebene. PLR wiederholt gezielt jene Level, an denen der Agent noch am meisten lernen kann, statt sie gleichverteilt zu ziehen. Das Training dieser Arbeit enthält eine stark vereinfachte Variante dieses Gedankens: einen *Seed-Pool*, der einen Teil der Episodenstarts durch bereits gelöste Welten ersetzt (Abschnitt 4.1). Wie in Abschnitt 10.5 diskutiert, kehrt diese Variante die Auswahlrichtung von PLR gerade um und ist deshalb kritisch zu sehen. Der Explorations-Bonus für neu betretene Tiles ist eine zählbasierte Form intrinsischer Motivation; curiositätsgetriebene Verfahren wie ICM [33] und RND [34] verallgemeinern dieses Prinzip auf Beobachtungsräume, in denen Neuheit nicht direkt abzählbar ist. Dies ist hier nicht erforderlich, da die Tile-Position Neuheit unmittelbar definiert.

Zur Vergleichbarkeit der absoluten Zahlen. Ein direkter Vergleich der hier erreichten Erfolgsquote mit publizierten Werten dieser Benchmarks ist *nicht* seriös möglich, weil sie unterschiedliche Metriken berichten: Procgen aggregiert über den *normalisierten Return* $(R - R_{\min}) / (R_{\max} - R_{\min})$ und nicht über eine Erfolgsquote [2]; Crafter bewertet erreichte *Achievements* [29]; für MiniGrid-Aufgaben berichten Folgearbeiten aufgabenspezifische Erfolgsquoten, die je nach Aufgabe bis 100 % reichen. Hinzu kommen Unterschiede in Sichtradius, Episodenlänge und Zielabstand. Eine Aussage der Form „unser Wert liegt über/unter dem Stand der Technik“ wäre daher eine Scheinpräzision und wird hier bewusst nicht getroffen.

Belastbar ist der Vergleich nur *innerhalb* dieser Arbeit: gegen die eigene Gegenüberstellung (Tabelle 13), gegen die Gap-Experimente (Abschnitt 7.6) und gegen die eigenen, vorab fixierten Kriterien (Abschnitt 6.5). Alle Vergleichsbedingungen laufen dort auf demselben Generator, mit derselben Metrik und demselben Eval-Protokoll.

4 Implementierung des Spiels Stoneforge

Stoneforge ist ein selbst entwickeltes, rundenbasiertes 2D-Spiel: ein C++-Simulationskern, ein grafischer Client, in dem ein Mensch das Spiel tatsächlich spielen kann, und eine Python-Anbindung über pybind11, über die ein Lernverfahren *dieselbe* Simulation ansteuert. Was der Agent erlebt, ist damit bis auf die Wahrnehmung identisch mit dem, was ein Spieler erlebt.

Die Implementierung ist auf zwei Kapitel verteilt, in der Reihenfolge, in der sie aufeinander aufbauen. Dieses Kapitel behandelt das Spiel: den Simulationskern, den Weltgenerator und den grafischen Client. Es endet mit einer Welt, die reproduzierbar, lösbar und in ihrer Zielentfernung steuerbar ist, aber von Reinforcement Learning noch nichts weiß. Kapitel 5 setzt darauf auf und macht daraus eine Lernumgebung.

Beide Teile sind technische Ingenieursleistungen gleichen Gewichts und Bestandteil derselben Arbeit; die Trennung beschreibt die Zuständigkeit (Tabelle 1), nicht eine Rangfolge. Der Schwerpunkt dieses Kapitels liegt auf dem Weltgenerator (Abschnitt 4.3), weil er festlegt, wie die Welten aussehen, in denen später alles gemessen wird.

4.1 Technologie-Stack

Das System besteht aus drei Schichten: einem C++-Kern für Simulation und Weltgenerierung, einem Python-Binding und dem Python-Trainingsstack. Tabelle 2 listet die verwendeten Komponenten.

Komponente	Technologie	Zweck
Simulationskern	C++20 (ca. 4 300 LOC)	Weltgenerierung, Spiellogik, Reward, BFS
Binding	pybind11 2.13	C++-Kern als Python-Modul (<code>stoneforge_sim</code>)
Build	CMake	Kern, Binding, Clients
RL-Umgebung	Python 3.12, Gymnasium 1.2	<code>StoneforgeWorldEnv</code> (Gym-API)
RL-Algorithmen	Stable-Baselines3 / sb3-contrib 2.8	PPO, DQN, A2C, RecurrentPPO
Deep Learning	PyTorch 2.12 (CPU)	Netz- und LSTM-Training
Spiel-Client	raylib 5.5 (C++, ca. 4 500 LOC)	spielbarer Client, KI-Zuschaumodus
Monitoring	TensorBoard, WebSockets	Trainingsmetriken, Live-Visualisierung
Auswertung	NumPy, Matplotlib	Evaluation, Abbildungen

Tabelle 2: Technologie-Stack.

Die Simulation ist deterministisch pro Seed. Training und Evaluation laufen vollständig auf der CPU; ein Vergleich mit der GPU ergab für dieses kleine Netz keine Beschleunigung. Der Durchsatz liegt je nach Konfiguration bei ca. 88–190 Umgebungsschritten pro Sekunde; die für die Endergebnisse verwendete Konfiguration (`batch_size=8`) liegt am unteren Ende dieser Spanne. Die verwendete Hardware und der zugehörige Benchmark sind in Anhang 10.8 dokumentiert.

4.2 Die Bausteine: Tile, Chunk und Welt

Bevor von Generierung die Rede sein kann, braucht es die drei Dinge, die erzeugt werden. Sie bauen streng aufeinander auf.

Das Tile. Die kleinste Einheit der Welt ist ein Feld auf einem ganzzahligen Gitter. Ein Tile trägt genau einen Typ aus einer Aufzählung von 15 Werten (`TileType` in `types.hpp`): leer, Wand, Erz, Exit, Baum, Werkbank, Holzwand, Holzstamm und sieben biomspezifische Struktur-Tiles. Es gibt keine Höhe, keine Ebenen und keine Teilbelegung, ein Feld ist genau eine Sache.

Jedem Typ ist ein `WorldObject` zugeordnet, das seine Eigenschaften kennt, darunter die für alles Weitere entscheidende Frage, ob man darüber laufen kann. Die Voreinstellung lautet

nein: Nur leere Felder und der Exit sind begehbar, alles andere blockiert. Das klingt nach einem Detail, hat aber Folgen bis in die Ergebnisse, denn Bäume und Erzadern sind damit genauso Hindernis wie Wände (Tabelle 5).

Der Chunk. Tiles werden nicht einzeln verwaltet, sondern in Blöcken von 8×8 Feldern. Ein Chunk ist ein flaches Array aus 64 Tile-Typen plus einem Flag, ob er bereits erzeugt wurde. Mehr nicht.

Die Welt. World hält eine Hashtabelle von Chunk-Koordinate auf Chunk. Diese Tabelle ist zu Beginn leer und füllt sich erst, während gespielt wird: Jeder Zugriff auf ein Tile rechnet die Weltkoordinate in Chunk- und lokale Koordinate um, und findet sich der Chunk nicht in der Tabelle, wird er in diesem Moment erzeugt und abgelegt. Materialisiert wird also nur, was jemand tatsächlich ansieht.

Daraus folgt die zentrale Eigenschaft des Generators, und sie ist bewusst so gebaut: Die Erzeugung eines Chunks hängt *ausschließlich* vom Seed und seinen eigenen Koordinaten ab, von keinem anderen Chunk und von keinem Zustand. Der Generator ist eine reine Funktion

$$(seed, c_x, c_y) \mapsto 64 \text{ Tile-Typen.} \quad (5)$$

Die Welt ist damit praktisch unbegrenzt, ohne je gespeichert zu werden, und zwei Durchläufe liefern denselben Chunk unabhängig davon, in welcher Reihenfolge der Spieler die Welt betreten hat.

Warum das nicht nur elegant, sondern notwendig ist. Genau diese Reproduzierbarkeit trägt später das Evaluationsprotokoll (Abschnitt 6.3). Testset A besteht nicht aus gespeicherten Karten, sondern aus 50 Zahlen (Seeds 7000 bis 7049). Der Seed *ist* die Welt. Nur weil derselbe Seed bitgenau dieselben Tiles liefert, lassen sich sieben Trainingsläufe auf demselben Testset vergleichen, lässt sich ein Trainingsseed sauber von einem Testseed trennen, und lässt sich eine Episode im grafischen Client exakt nachspielen. Ein Generator mit einem fortlaufenden Zufallsgenerator statt einer reinen Funktion würde all das zerstören, weil dann die Reihenfolge der Aufrufe das Ergebnis mitbestimmt.

4.3 Der Weltgenerator

Damit ist die Frage gestellt, die dieser Abschnitt beantwortet: Wie werden aus einem Seed und zwei Chunk-Koordinaten 64 Tile-Typen? Die begrifflichen Grundlagen dazu (Hash- gegenüber Wertrauschen, Domain Warping, zelluläre Automaten, Breitensuche) stehen in Abschnitt 2.1.2; hier geht es darum, wie sie zusammengesaltet sind.

Der Einstieg ist `World::generateChunk()`, und die Funktion ist kurz genug, um sie ganz zu zeigen (Listing 1). Sie ist die Landkarte für alles Weitere: Erst wird jedes der 64 Felder einzeln ausgewürfelt, dann optional geglättet, dann eventuell eine Landmarke eingestempelt. Mehr passiert nicht. Die Biomwahl steckt dabei in `sampleBaseTile` und wird gleich als eigene Stufe behandelt.

```

1 void World::generateChunk(int cx, int cy, Chunk& chunk) const {
2     const auto& cfg = gameConfig().world;
3
4     // Stufe 0+1: Basisbelegung, Feld fuer Feld
5     for(int ly = 0; ly < kChunkSize; ++ly) {
6         for(int lx = 0; lx < kChunkSize; ++lx) {
7             const int wx = cx * kChunkSize + lx;    // lokal ->
8                 Weltkoordinate
9             const int wy = cy * kChunkSize + ly;
```

```

9         chunk.tiles[ly * kChunkSize + lx] = sampleBaseTile(wx, wy, cfg)
10     };
11 }
12 // Stufe 2: optionale Glaettung (in dieser Arbeit abgeschaltet)
13 runCellularSmoothingStage(cx, cy, chunk, cfg);
14 // Stufe 3: seltene Landmarke
15 placeBiomeStructure(cx, cy, chunk);
16 }

```

Listing 1: Die vollständige Erzeugung eines Chunks (`World::generateChunk()`, `src/core/world.cpp`). Jedes Feld wird unabhängig aus seinen Weltkoordinaten bestimmt; nichts in dieser Funktion greift auf einen Nachbar-Chunk oder auf gespeicherten Zustand zu.

Woher der Zufall kommt. Alle Stufen ziehen ihre Zufallszahlen aus einer einzigen Funktion, `noise01(x, y, salt)` (Listing 2). Sie tut nichts weiter, als ihre drei Argumente zu einer 64-Bit-Zahl zu verrühren und daraus einen Wert in $[0, 1)$ zu machen: Die Koordinaten und der Seed werden mit großen ungeraden Konstanten multipliziert und per XOR verknüpft, dann läuft das Ergebnis durch einen Bit-Mixer, dessen Multiplikatoren aus `SplitMix64` stammen [35]. Der Mixer sorgt dafür, dass sich benachbarte Eingaben auf völlig verschiedene Ausgaben verteilen. Die unteren 48 Bit ergeben, skaliert, die Zufallszahl.

```

1 double World::noise01(int worldX, int worldY, std::uint64_t salt) const {
2     std::uint64_t value = static_cast<std::uint64_t>(worldX) * 0
        x9e3779b185ebca87ULL;
3     value ^= static_cast<std::uint64_t>(worldY) * 0xc2b2ae3d27d4eb4fULL;
4     value ^= seed_ * 0x165667b19e3779f9ULL;
5     value ^= salt;
6     value = mix(value); // SplitMix64-Finalizer
7
8     constexpr double kScale = 1.0 / static_cast<double>(0xFFFFFFFFFFFFFFFFULL);
9     return static_cast<double>(value & 0xFFFFFFFFFFFFFFFFULL) * kScale;
10 }

```

Listing 2: Die zustandsfreie Rauschquelle des Generators (`src/core/world.cpp`). Der Rückgabewert hängt ausschließlich von den Argumenten ab, nicht davon, wann oder wie oft die Funktion aufgerufen wird.

Der Unterschied zu einem gewöhnlichen Zufallszahlengenerator ist der Kern der Sache. Ein solcher Generator besitzt einen Zustand und liefert seine Werte in einer festen Reihenfolge; wer den tausendsten Wert braucht, muss die 999 davor ziehen. Damit hinge die Welt davon ab, in welcher Reihenfolge man ihre Chunks erzeugt, und die in Abschnitt 4.2 begründete Reproduzierbarkeit wäre dahin. `noise01` hat keinen Zustand. Der Wert eines Feldes hängt nur von dessen Koordinaten ab, ist also jederzeit, beliebig oft und in beliebiger Reihenfolge neu berechenbar, immer mit demselben Ergebnis.

Das dritte Argument, der *Salt*, ist eine feste Konstante je Merkmal. Er fließt in den Hash ein und erzeugt so aus derselben Koordinate mehrere voneinander unabhängige Rauschfelder. Der Generator hält getrennte Salts für Wanddichte, Erz und Bäume bereit. Ohne sie lägen alle drei Merkmale an exakt denselben Stellen, weil sie dieselbe Zahl auswerten würden.

Damit ist auch schon gesagt, welche Art von Rauschen hier vorliegt: Es ist *Hash-Rauschen* im Sinne von Abschnitt 2.1.2, also unkorreliertes weißes Rauschen. Nachbarfelder wissen nichts voneinander. Abbildung 6 zeigt, was das für die Form der Welt bedeutet, und der Unterschied ist drastisch: Derselbe Schwellwert schneidet aus Hash-Rauschen verstreute Einzelfelder, aus interpoliertem Wertrauschen dagegen zusammenhängende Barrieren. Diese Eigenschaft wird uns am Ende des Abschnitts wieder einholen.

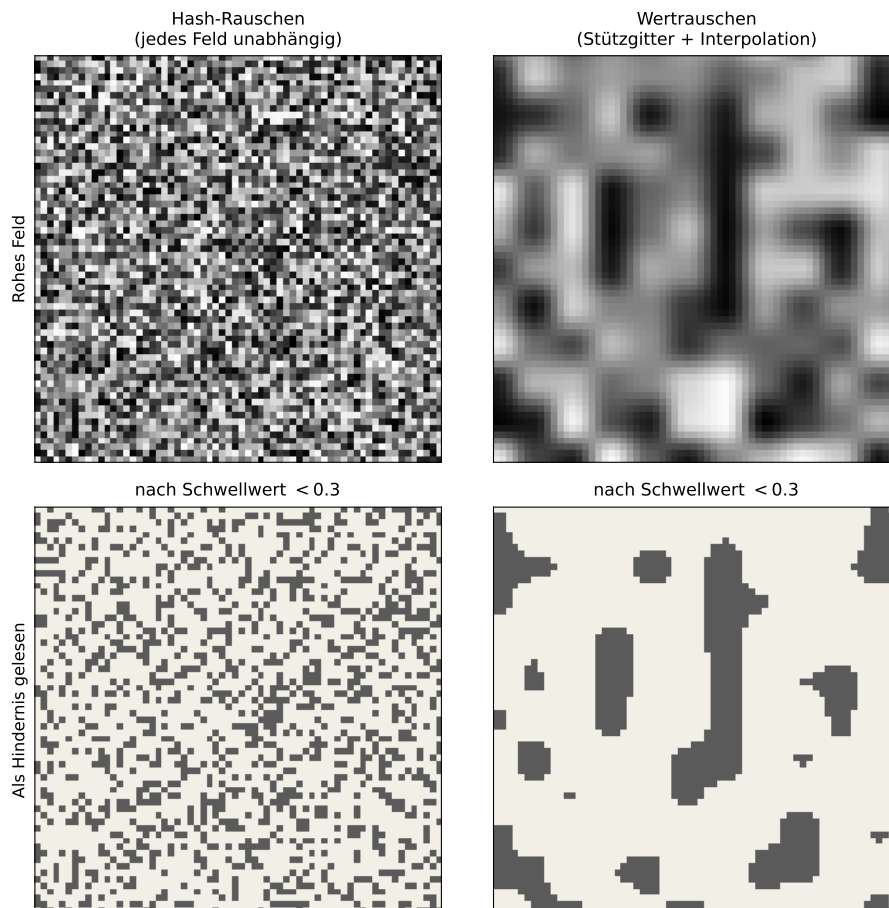


Abbildung 6: Warum die Wahl der Rauschquelle über die Form der Welt entscheidet. Links das Hash-Rauschen aus Listing 2, bei dem jedes Feld unabhängig aus seinen Koordinaten berechnet wird; rechts interpoliertes Wertrauschen über einem groben Stützgitter. Oben die rohen Felder, unten dasselbe Feld nach demselben Schwellwert als Hindernis gelesen. Aus dem Hash-Rauschen entstehen verstreute Einzelhindernisse, die man einfach umgeht; erst die Interpolation erzeugt zusammenhängende Barrieren, die einen Umweg erzwingen. Der Generator nutzt Hash-Rauschen für die Belegung der Felder (Stufe 1) und interpoliertes Wertrauschen nur für das Biomfeld (Stufe 0). Beide Felder mit `scripts/plot_worldgen.py` aus der Rauschfunktion des Kerns erzeugt.

Stufe 0: Welches Biom hat dieser Chunk?

Zuerst bekommt der *ganze Chunk* ein Biom, nicht jedes Feld einzeln. Das ist eine bewusste Vergrößerung: Sie hält die Landschaft zusammenhängend, statt Wald und Wüste feldweise abwechseln zu lassen.

Für das Hash-Rauschen aus Listing 2 allein wäre das nicht zu haben, denn es liefert für benachbarte Chunks völlig unabhängige Werte. Die Stufe baut deshalb daraus *Wertrauschen* (Abschnitt 2.1.2): Sie wertet `noise01` nur an den Ecken eines groben Stützgitters aus und interpoliert dazwischen bilinear. Damit die Übergänge nicht als Gitterkanten sichtbar werden, läuft der Interpolationsparameter zuvor durch Perlins Glättungsfunktion $3t^2 - 2t^3$ [10]. Benachbarte Chunks bekommen so ähnliche Werte, und es entstehen Regionen statt Flimmern.

Darüber liegen zwei weitere Kunstgriffe (Listing 3). Erstens werden drei solcher Felder mit steigender Frequenz gewichtet addiert (0,62 : 0,28 : 0,10); das grobe Feld gibt die Grundform vor, die feineren rauhen die Ränder auf. Zweitens *Domain Warping*: Bevor das Feld ausgewertet wird, verschieben zwei weitere Rauschfelder die Abtastkoordinate selbst. Ohne diesen Schritt lägen die Biomgrenzen erkennbar am Stützgitter und sähen rechteckig aus.

```
1 double x = static_cast<double>(cx) * 0.22;    // Chunk -> Feldkoordinate
2 double y = static_cast<double>(cy) * 0.22;
3
4 // Domain Warping: die Abtastkoordinate selbst verrauschen
5 const double warpX = sampleValue(x * 0.57 + 19.3, y * 0.57 - 7.1, ...);
6 const double warpY = sampleValue(x * 0.57 - 11.8, y * 0.57 + 13.4, ...);
7 x += (warpX - 0.5) * 2.8;
8 y += (warpY - 0.5) * 2.8;
9
10 // Drei Oktaven: grobe Form + zwei feinere Detailebenen
11 const double n1 = sampleValue(x, y, ...);
12 const double n2 = sampleValue(x * 2.1 - 3.7, y * 2.1 + 1.9, ...);
13 const double n3 = sampleValue(x * 4.2 + 8.4, y * 4.2 - 5.6, ...);
14
15 return std::clamp(n1 * 0.62 + n2 * 0.28 + n3 * 0.10, 0.0, 1.0);
```

Listing 3: Kern der Stufe 0 (`World::biomeFieldForChunk()`, `src/core/world.cpp`, gekürzt). `sampleValue` ist das interpolierte Wertrauschen; `warpX` und `warpY` verschieben die Abtastkoordinate, bevor die drei Oktaven überlagert werden.

Der Rückgabewert liegt in $[0, 1)$ und wird in sieben gleich breite Intervalle geschnitten. Diese ergeben die Biome Grasland, Wald, Wüste, Bergland, Steppe, Tundra und Hölle. Weil die Chunk-Koordinaten mit 0,22 skaliert eingehen, verändert sich das Feld nur langsam: Ein Biom erstreckt sich typischerweise über vier bis fünf Chunks, also rund 36 Felder, und ist damit deutlich größer als das 15×15 -Sichtfenster eines Agenten. Ein Agent sieht also fast nie eine Biomgrenze, sondern immer nur eine Landschaft.

Stufe 1: Was liegt auf diesem einzelnen Feld?

Jetzt erst wird Feld für Feld entschieden, und hier fällt die Entscheidung, die für die Navigation zählt: Hindernis oder frei. Der Generator zieht für jedes Feld drei unabhängige Rauschwerte, einen für Waddichte, einen für Erz, einen für Bäume, und vergleicht jeden mit einem Schwellwert, der vom Biom des Chunks abhängt. Die Schwellwerte stehen vollständig in Tabelle 4, die Entscheidungslogik in Listing 4.

```
1 const double density = noise01(worldX, worldY, cfg.densitySalt);
2 const double ore     = noise01(worldX, worldY, cfg.oreSalt);
3 const double trees   = noise01(worldX, worldY, cfg.treeSalt);
```

```

4 // wallThreshold / oreThreshold / treeThreshold: je nach Biom gesetzt
5
6 TileType tile = TileType::Empty;
7
8 // 1. Seen sind hindernisfrei und schlagen alles Weitere
9 if(lakeMaskAt(worldX, worldY)) { return TileType::Empty; }
10 // 2. Wand
11 if(density < wallThreshold) { tile = TileType::Wall; }
12 // 3. Erz, nur im Bergland; ueberschreibt auch eine Wand
13 if(biomeTag == 3 && ore < oreThreshold) { tile = TileType::Resource; }
14 // 4. Baum, nur auf noch leerem Feld
15 if(treeThreshold > 0.0 && tile == TileType::Empty && trees < treeThreshold)
16 {
17     tile = TileType::Tree;
18 }
19 return tile;

```

Listing 4: Die Entscheidung über ein einzelnes Feld (`World::sampleBaseTile()`, `src/core/world.cpp`, gekürzt). Die Reihenfolge der vier Prüfungen ist die Regel: See schlägt alles, Erz überschreibt bedingungslos, ein Baum wächst nur auf einem bis dahin leeren Feld.

Zwei Dinge daran sind leicht zu übersehen. Erstens ist die Reihenfolge der Prüfungen selbst eine Regel und kein Implementierungsdetail: Erz wird bedingungslos gesetzt und kann eine bereits bestimmte Wand ersetzen, ein Baum dagegen nur dort wachsen, wo bis dahin nichts liegt. Zweitens ist `oreThreshold` für sechs der sieben Biome wirkungslos, weil die Prüfung zusätzlich `biomeTag == 3` verlangt. Erz gibt es ausschließlich im Bergland; die übrigen Zeilen der Erzspalte in Tabelle 4 laufen ins Leere.

Die vorgelagerte Seenmaske ergibt sich aus einer gewichteten Linearkombination zweier Rauschfelder,

$$Score = 0,75 a + 0,25 b, \quad (6)$$

und weist eine Zelle oberhalb eines harten Cutoffs von 0,86 als Wasser aus. Beide Felder werden dabei auf ganzzahlig geteilten Koordinaten ausgewertet ($x/7$ bzw. $x/3$), sind also blockweise konstant; die Seen sind deshalb grobkörnig und nicht rund. Sie sind hindernisfrei und überschreiben die Basisbelegung vollständig.

Ein Chunk, Feld für Feld. Wie das konkret aussieht, zeigt Tabelle 3 an einer echten Zeile aus einer Eval-Welt. Alle acht Felder liegen im selben Chunk und damit im selben Biom, sehen also dieselben Schwellwerte; allein die drei Rauschwerte unterscheiden sie.

Stufe 2: Glättung, in dieser Arbeit abgeschaltet

Auf die Basisbelegung könnte ein zellulärer Automat folgen, der verstreute Einzelhindernisse zu zusammenhängenden Formationen verschmilzt (Moore-Nachbarschaft, Geburts- und Überlebensschwelle, Abschnitt 2.1.2). Er ist implementiert, in allen berichteten Läufen aber abgeschaltet; Abbildung 8 zeigt ihn deshalb gestrichelt. Warum das Einschalten die Welten nicht härter, sondern leerer macht, steht weiter unten.

Seine Konstruktion verdient trotzdem eine Bemerkung, weil sie ein echtes Problem löst. Ein Automat braucht die acht Nachbarn jeder Zelle. Am Chunk-Rand liegen die im Nachbar-Chunk, und den zu erzeugen würde eine Abhängigkeit zwischen Chunks einführen und damit genau die Reinheit zerstören, auf der die Reproduzierbarkeit beruht. Die Stufe berechnet deshalb einen *Halo* in Breite der Iterationszahl aus der Basisbelegung mit, lässt den Automaten darüber laufen und verwirft ihn danach. Das Ergebnis hängt so weiterhin nur von Seed und Chunk-Koordinaten ab.

Feld	Dichte	Erz	Baum	Ergebnis
	< 0,25?	< 0,10?	< 0,02?	
(−32, −31)	0,330	0,002	0,622	Erz
(−31, −31)	0,401	0,249	0,013	Baum
(−30, −31)	0,712	0,164	0,537	leer
(−29, −31)	0,078	0,436	0,378	Wand
(−28, −31)	0,357	0,994	0,217	leer
(−27, −31)	0,370	0,376	0,154	leer
(−26, −31)	0,186	0,149	0,064	Wand
(−25, −31)	0,559	0,481	0,579	leer

Tabelle 3: Eine vollständige Chunk-Zeile aus der Welt zu Seed 7000, Chunk (−4, −4), Biom Bergland (Schwellwerte 0,25 / 0,10 / 0,02). Fett gesetzt ist jeweils der Rauschwert, der seinen Schwellwert unterschreitet und damit die Belegung auslöst. Feld (−32, −31) zeigt die Vorrangregel: Die Dichte 0,330 hätte keine Wand erzeugt, der Erzwert 0,002 setzt trotzdem Erz. Werte reproduziert mit einem Nachbau des Generators in Python, der auf 3663 geprüften Feldern dieser Welt ausnahmslos dieselben Tile-Typen liefert wie der C++-Kern.

Stufe 3: Landmarken

Zuletzt setzt der Generator mit einer Wahrscheinlichkeit von $P = 0,10$ je Chunk eine biomspezifische Landmarke. Ihre Form stammt aus handgeschriebenen ASCII-Matrizen der Größe 5×5 , je eine pro Biom, die an einer aus dem Rauschen bestimmten Position im Chunk eingestempelt wird. Bei einer Chunk-Kantenlänge von 8 bleiben dafür drei Felder Spielraum in jeder Richtung. Chunks, die Spawn oder Exit enthalten, überspringt die Stufe, damit die Strukturen weder den Start noch das Ziel verbauen.

Die Pipeline an einer echten Welt. Abbildung 7 zeigt alle Stufen nebeneinander an einer der 50 Welten des Testsets. Links das Biomfeld, dessen Blöcke sichtbar an den Chunk-Grenzen ausgerichtet sind und dessen Regionen sich über mehrere Chunks erstrecken. In der Mitte die Basisbelegung: Der Biomwechsel ist als Dichtewechsel zu erkennen, im Bergland liegen sichtbar mehr Wände und Erz als in der Tundra. Rechts die fertige Welt mit Landmarken, Spawn, Exit und dem kürzesten Weg.

Dieser Weg ist die eigentliche Aussage der Abbildung. Er ist eine grobe Treppe zwischen zwei Punkten und weicht kaum einmal seitlich aus. Ein knappes Viertel der Felder ist blockiert, und trotzdem gibt es nichts zu umrunden, weil die Hindernisse einzeln stehen statt Wände zu bilden. Genau das quantifiziert der Umwegfaktor weiter unten.

Biom	Wand	Erz	Baum	Charakter
Wald	0,07	0,015	0,23	dicht bewachsen, kaum Stein
Grasland	0,10	0,03	0,05	offen
Steppe	0,11	0,03	0,09	offen, etwas mehr Bewuchs
Tundra	0,14	0,05	0,07	ausgeglichen
Wüste	0,19	0,08	0,00	steinig, baumlos
Hölle	0,23	0,09	0,06	steinig
Bergland	0,25	0,10	0,02	dichtester Fels, <i>einziges Erzbiom</i>

Tabelle 4: Schwellwerte der Stufe 1 je Biom (`World::sampleBaseTile()`). Ein Feld wird zur Wand, wenn sein Dichte-Rauschwert unter der Wandschwelle liegt, entsprechend für Erz und Bäume mit ihren eigenen Rauschfeldern. Erz entsteht ausschließlich im Bergland; die Erzspalte der übrigen Biome bleibt wirkungslos. Die Werte sind in `world.cpp` hartkodiert und über `game_config.json` nicht erreichbar.

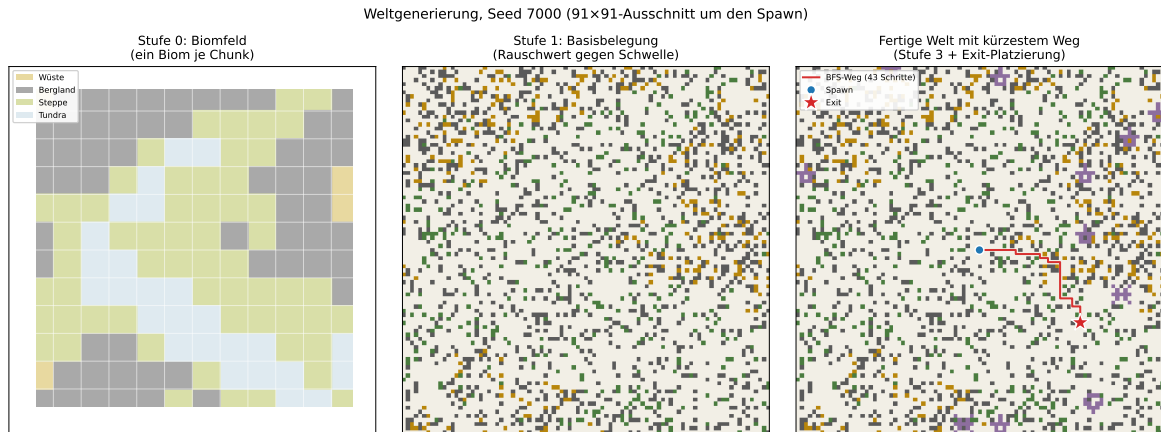


Abbildung 7: Die Stufen der Weltgenerierung an der Welt zu Seed 7000, einem der 50 Seeds des Testsets A, in einem 91×91 -Ausschnitt um den Spawn. **Links:** das Biomfeld aus Stufe 0, konstant je Chunk, mit eingezeichnetem 8×8 -Chunk-Raster. **Mitte:** die Basisbelegung aus Stufe 1, wobei jedes Feld allein aus seinen Koordinaten entsteht. **Rechts:** die fertige Welt inklusive Landmarken (violett), mit Spawn, Exit und dem kürzesten Weg von 43 Schritten. Grau sind Wände, grün Bäume, ocker Erz; alle drei blockieren (Tabelle 5). Erzeugt mit `scripts/plot_worldgen.py`. Die Tile-Darstellungen stammen direkt aus dem C++-Kern; das Biomfeld ist im Python-Binding nicht zugänglich und wurde aus einem Nachbau der Generatorfunktionen berechnet, der die Tile-Typen des Kerns auf allen 8115 geprüften Feldern dieses Ausschnitts exakt reproduziert.

Was am Ende tatsächlich im Weg steht. Die Schwellwerte in Tabelle 4 sind Parameter, keine Messwerte, und sie beschreiben allein die Wände. Wie viel eine erzeugte Welt dem Spieler wirklich in den Weg legt, ist eine andere Zahl: Weil Bäume, Erz und Landmarken ebenfalls blockieren, liegt der Anteil unpassierbarer Felder deutlich über jeder einzelnen Wandschwelle. Tabelle 5 schlüsselt ihn auf. Die Zusammensetzung erklärt zugleich, warum die Wandschwellen im Bereich 0,07 bis 0,25 liegen, die weiter unten diskutierte Hindernisdichte aber bei rund 0,24: Es sind zwei verschiedene Größen, und nur die zweite entscheidet über die Geometrie der Wege.

Tile	Anteil	begehrbar
Leer	0,7511	ja
Wand	0,1562	nein
Baum	0,0421	nein
Erz	0,0322	nein
Landmarken	0,0183	nein
Exit	0,0002	ja
unpassierbar gesamt	0,2487	

Tabelle 5: Zusammensetzung der erzeugten Welten, gezählt über die 100 Seeds beider Eval-Sets in einem 81×81 -Fenster um den Spawn (656100 Tiles). Nur ein knappes Sechstel der Welt besteht aus Wänden; Bäume und Erz steuern zusammen weitere sieben Prozentpunkte zur Undurchdringlichkeit bei. Die Landmarkenzeile fasst die sieben biomspezifischen Struktur-Tiles zusammen.

Nachbearbeitung und eine Grenze der Konfigurierbarkeit. Nach den vier Stufen räumt der Generator noch feste Radien um Spawn und Exit frei (5×5 bzw. 3×3 Fel-

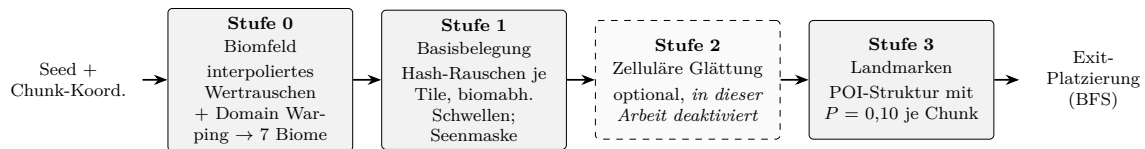


Abbildung 8: Die vierstufige Pipeline der Weltgenerierung, je Chunk (8×8 Tiles) deterministisch aus Seed und Koordinaten berechnet. Stufe 2 ist in der berichteten Konfiguration abgeschaltet (gestrichelt, Tabelle 6); die Welten entstehen also allein aus den Stufen 0, 1 und 3. Die Exit-Platzierung (Abschnitt 4.4) ist der Pipeline nachgelagert und arbeitet nicht mehr chunkweise, sondern auf der bereits erzeugten Welt.

der), damit weder Spieler noch Ziel in einem Hindernis stecken. Die Biomzuordnung dagegen ist starr: Die Sieben-Intervall-Teilung des Biomfeldes steht im Code und lässt sich über `game_config.json` nicht verändern. Ein Audit der Konfiguration entfernte hier mehrere Schlüssel (`coldBiomeMax`, `warmBiomeMax` und die `moss`-Schwellwerte), die zwar eingelesen, durch die feste Binning-Logik aber nie ausgewertet wurden. Sie versprachen ein Tuning, das faktisch wirkungslos war (Abschnitt 10.7). Dasselbe gilt für die Schwellwerte in Tabelle 4: Wer die Welten härter machen will, muss den Generator anfassen, nicht die Konfiguration.

Die Exit-Platzierung und die Lösbarkeitssicherung setzen auf der so erzeugten Welt auf und sind in Abschnitt 4.4 beschrieben. Der Generator implementiert darüber hinaus die in Abschnitt 2.1.2 genannten Sicherheitsnetze. Für die Bewertung der Ergebnisse ist entscheidend, welche davon in den berichteten Läufen tatsächlich wirksam waren (Tabelle 6).

Mechanismus	Aktiv	Begründung
BFS-Exit-Platzierung nach Laufweg	ja	Garantiert Erreichbarkeit per Konstruktion und steuert zugleich die Curriculum-Distanz (Abschnitt 10.7).
Zelluläre Glättung (Stufe 2)	nein	Abgeschaltet; die Welten entstehen allein aus Stufe 1.
Flood-Fill-Validierung des Start-Ziel-Fensters	nein	Durch die BFS-Platzierung redundant: Der Exit wird ausschließlich aus begehbaren erreichbaren Zellen gewählt.
Carving (<code>forceGuaranteedPath</code> und Fallback)	nein	Aus demselben Grund nicht erforderlich; damit entfällt zugleich der gerade Korridor als strukturelle Auffälligkeit.

Tabelle 6: Im Generator implementierte Mechanismen und ihr Status in der berichteten Konfiguration (Quelle: `assets/base/game_config.json`, Abschnitt `worldgen`).

Eine Konsequenz für die Ergebnisse. Aus Tabelle 6 folgt eine Eigenschaft der Welten, die für die Bewertung in Abschnitt 7.4 unmittelbar relevant ist: Da die Glättung abgeschaltet ist, stammt die Hindernisverteilung ausschließlich aus *unkorreliertem* Hash-Rauschen. Jedes Feld entscheidet für sich, ohne seine Nachbarn zu kennen, denn `noise01` ist genau dafür gebaut. Die Hindernisse sind damit statistisch unabhängig voneinander verteilt und bilden überwiegend verstreute Einzelfelder statt zusammenhängender Barrieren (Abbildung 3). Was den Generator reproduzierbar macht, kostet ihn also zugleich jede Struktur: Genau die Eigenschaft, die Abschnitt 4.2 als notwendig begründet, verhindert hier Sackgassen. Genau diese Struktur macht die Welten „offen“ im Sinne von Abschnitt 7.4: Sackgassen, die Gedächtnis erfordern würden, entstehen selten, und eine ungerichtete Zufallsbewegung verlässt sie zuverlässig.

Wie offen die Welten sind, misst der Umwegfaktor. Quantifizieren lässt sich diese Offenheit über das Verhältnis aus BFS-Laufweg und Manhattan-Distanz zwischen Spawn und Exit. Bei 1,0 ist der kürzeste Weg die Luftlinie, es gibt also nichts zu umrunden. Über beide Eval-Sets gemessen (100 Seeds, `scripts/probe_world_geometry.py`) liegt der Faktor bei **1,10**, bei einer Hindernisdichte von 0,238 (Tabelle 5): Der kürzeste Weg ist zehn Prozent länger als die Luftlinie. Ein knappes Viertel der Welt ist blockiert, und trotzdem gibt es fast nichts zu umrunden. Das ist die quantitative Ursache dafür, dass ein vierzeiliger Kompass-Folger 92 % erreicht (Abschnitt 7.4). Nicht die Politik ist stark, die Geometrie ist schwach.

Die naheliegende Reparatur funktioniert nicht. Es liegt nahe, dagegen die vorhandene Glättungsstufe einzuschalten; in der PCG-Literatur ist gerade sie es, die aus verstreutem Rauschen konkave Strukturen formt [11, 8]. Für diesen Generator haben wir das nachgemessen, und es trifft nicht zu. Mit den hinterlegten Regeln (Geburt ab 5, Überleben ab 4, zwei Iterationen) fällt die Wanddichte von 0,238 auf 0,037 und der Umwegfaktor auf 1,001. Die Welt wird zur leeren Ebene statt zum Labyrinth. Schuld ist die niedrige Ausgangsdichte: Bei 24 % Wandanteil hat kaum ein Wandtile die geforderten fünf soliden Nachbarn, also stirbt fast jede Wand und keine wird geboren. Wer die Stufe aktiviert, treibt die Erfolgsquote der ungelerten Heuristik nach oben statt sie einbrechen zu lassen.

Der Regelsweep in Tabelle 7 zeigt darüber hinaus einen harten Zielkonflikt. Jede Kombination, die den Umwegfaktor spürbar hebt, zerstört die Lösbarkeit; alles, was bei 50 von 50 lösbaren Welten bleibt, liegt beim Umweg im Rauschen der Referenz. Die Ursache liegt außerhalb des Automaten. `enableFloodFillValidation` und `enableMacroGraphPrecheck` stehen beide auf `false` (`assets/base/game_config.json`), und die Basisdichte ist in `World::sampleBaseTile()` hartkodiert. Der Automat zählt Nachbarn, von Erreichbarkeit weiß er nichts. Eine Härtung der Umgebung ist deshalb kein Konfigurationsschalter, sondern ein Eingriff in den Generator (Abschnitt 8).

Regel der Glättungsstufe	lösbar	Wanddichte	BFS	Umweg	p90
aus (berichtete Konfiguration)	50/50	0,240	40,1	1,123	1,312
an, 2 Iter., $b=5$ / $s=4$ (Default)	50/50	0,036	40,1	1,001	1,000
an, 2 Iter., $b=4$ / $s=3$	50/50	0,169	40,1	1,101	1,327
an, 1 Iter., $b=3$ / $s=3$	50/50	0,271	39,8	1,145	1,637
an, 2 Iter., $b=3$ / $s=2$	41/50	0,447	39,4	1,158	1,357
an, 4 Iter., $b=3$ / $s=2$	19/50	0,699	39,7	1,173	1,607
an, 2 Iter., $b=2$ / $s=1$	3/50	0,875	35,3	1,636	2,299

Tabelle 7: Regelsweep der zellulären Glättungsstufe auf Testset A (Seeds 7000–7049), gemessen mit `scripts/probe_world_geometry.py --sweep`. b ist die Geburts-, s die Überlebensschwelle. Der Umwegfaktor ist das Mittel aus BFS-Laufweg geteilt durch Manhattan-Distanz über die lösbaren Seeds. Die einzige Zeile mit deutlich erhöhtem Umweg ist zugleich die, in der nur noch 3 von 50 Welten lösbar sind.

4.4 Exit-Platzierung und garantierte Lösbarkeit

Warum das überhaupt eine Rolle spielt. Die zentrale Messgröße dieser Arbeit ist die Erfolgsquote: der Anteil der Welten, in denen der Agent den Exit erreicht. Diese Zahl ist nur dann interpretierbar, wenn jede gescheiterte Episode auch tatsächlich am Agenten lag. Erzeugt der Generator Welten, in denen der Exit hinter einer geschlossenen Wandbarriere liegt, mischt sich in jede Erfolgsquote ein unbekannter Anteil unlösbarer Karten. Ein Modell mit 65 % wäre dann von einem Modell mit 80 % nicht mehr zu unterscheiden, wenn 15 % der Welten von vornherein niemand lösen kann. Der Generator muss also *jede* Welt lösbar

erzeugen, und zwar nicht „meistens“, sondern immer.

Der übliche Weg und warum wir ihn nicht gehen. Naheliegender wäre Rejection Sampling: Welt erzeugen, Exit zufällig setzen, Erreichbarkeit per Flood-Fill prüfen, bei Misserfolg neu würfeln. Das ist teuer, weil der Verwerfungsanteil mit der Waddichte steigt, und es hat keine obere Laufzeitschranke. Die verbreitete Alternative, das nachträgliche Freiräumen eines Korridors zwischen Spawn und Exit (`forceGuaranteedPath`), ist noch schlechter: Sie hinterlässt einen geraden, ein Tile breiten Gang durch die Welt. Ein Agent kann lernen, genau diese Struktur zu erkennen und ihr zu folgen, statt zu navigieren. Beide Verfahren sind im Generator implementiert, beide sind in den berichteten Läufen abgeschaltet (Tabelle 6).

Stattdessen: den Exit aus dem Erreichbarkeitsbaum ziehen. Statt einen Exit zu setzen und danach zu prüfen, kehrt der Generator die Reihenfolge um. Er lässt eine Breitensuche vom Spawn aus über *ausschließlich begehbbare* Tiles laufen und wählt den Exit erst danach, aus genau der Menge, die diese Suche erreicht hat. Damit ist Erreichbarkeit keine Eigenschaft, die man prüfen müsste, sondern eine Eigenschaft der Konstruktion: Ein Tile, das die Breitensuche nie besucht hat, kann gar nicht erst Kandidat werden. Der Ablauf in `World::chooseExitPoint()` umfasst vier Schritte.

1. **Spawn freiräumen.** Um den festen Spawn wird ein 5×5 -Feld (Radius 2) auf *begehbar* gesetzt, damit der Agent nicht in einem Hindernis startet.
2. **Breitensuche vom Spawn.** Eine Vier-Nachbarschafts-BFS expandiert über begehbbare Tiles, begrenzt auf ein Fenster um den Spawn. Ihre Tiefe ist der *echte Laufweg* in Schritten, nicht der geometrische Abstand. Die Suche bricht bei Tiefe `exitMaxDistance` ab, weil die BFS-Tiefe monoton wächst.
3. **Kandidatenmenge bilden.** Jedes besuchte Tile mit einer Tiefe zwischen `exitMinDistance` und `exitMaxDistance` kommt in die Kandidatenliste. Diese Liste steuert zugleich das Curriculum: Ein Distanzfenster von 5 bis 12 erzeugt kurze Welten, eines von 35 bis 45 die Welten des Evaluationsprotokolls.
4. **Kandidat wählen.** Ein aus dem Seed gemischter Index bestimmt den Startpunkt in der Liste. Von dort prüft der Generator bis zu 24 Kandidaten gegen die Distanzbedingung des nächsten Absatzes und nimmt den ersten, der besteht.

Die Falle: die Freiräumung erzeugt eine Abkürzung. Nach der Wahl räumt `World::reset()` auch um den Exit ein Feld frei, hier 3×3 (Radius 1), damit das Ziel nicht in einer Wand verschwindet. Genau das kann die eben gemessene Distanz wieder kaputt machen. Öffnet die Freiräumung eine Wand, die vorher einen Umweg erzwungen hat, entsteht eine Abkürzung, und der reale Laufweg fällt unter `exitMinDistance`. Bei einem Fenster von 35 bis 45 wäre der Agent dann in einer Welt, die nominell zur Phase 3 gehört, tatsächlich aber deutlich leichter ist. Für ein distanzgesteuertes Curriculum ist das kein Schönheitsfehler, sondern ein verfälschter Schwierigkeitsgrad.

Die Lösung ist eine zweite Breitensuche, die die Freiräumung vorwegnimmt. Listing 5 zeigt den Kern: Tiles im Freiräumungsradius des Kandidaten gelten in dieser Suche bereits als begehbar, obwohl sie es noch nicht sind. Gemessen wird also der Laufweg der Welt, die *nach* der Freiräumung existiert. Nur wenn dieser Weg noch mindestens `exitMinDistance` beträgt, wird der Kandidat angenommen.

```
1 // BFS vom Spawn zum Kandidaten. Tiles im Freiräumungsradius zählen
2 // als begehbar, weil World::reset() sie ohnehin öffnen wird:
3 const bool virtuallyCleared =
```

```

4      std::abs(nx - target.x) <= cfg.exitClearRadius &&
5      std::abs(ny - target.y) <= cfg.exitClearRadius;
6  if(!virtuallyCleared && !isPassable(nx, ny)) { continue; }
7
8  // ... Kandidat nur annehmen, wenn der Weg auch danach lang genug bleibt:
9  for(std::size_t k = 0; k < maxTries; ++k) {
10     const Vec2i cand = candidates[(start + k) % n];
11     if(clearedDistance(cand) >= minDist) { return cand; }
12 }
13 return candidates[start];    // Fallback: erreichbar, Distanz ggf. kuerzer

```

Listing 5: Kandidatenprüfung bei der Exit-Platzierung (`src/core/world.cpp`, gekürzt). Die zweite Breitensuche behandelt Tiles im Freiräumungsradius des Kandidaten als begehbar und misst damit den Laufweg der Welt *nach* der Freiräumung. Ohne diese Vorwegnahme kann die Freiräumung eine Abkürzung öffnen und den Laufweg nachträglich unter `exitMinDistance` drücken.

Was garantiert ist und was nicht. Die beiden Zusagen sind unterschiedlich stark, und die Trennung ist wichtig:

Erreichbarkeit: hart garantiert. Jeder Kandidat stammt aus dem Erreichbarkeitsbaum der ersten Breitensuche. Ein unlösbarer Exit ist damit nicht unwahrscheinlich, sondern konstruktiv ausgeschlossen. Das gilt auch für den Fallback in der letzten Zeile von Listing 5.

Distanz: garantiert bis auf einen Fallback. Besteht keiner der 24 geprüften Kandidaten die Distanzbedingung, nimmt der Generator den zuerst gezogenen. Die Welt bleibt lösbar, ihr Laufweg kann aber unter `exitMinDistance` liegen. Dass dieser Zweig in der berichteten Konfiguration nicht greift, ist gemessen und nicht angenommen: Über 150 geprüfte Seeds lag der reale Laufweg ausnahmslos im Fenster [35, 45] (Minimum 35, Mittel 39,7 bis 40,1, Maximum 45), und in Phase 1 ebenso im Fenster [5, 12]. Bei deutlich höherer Waddichte oder einem engeren Distanzfenster wäre die Annahme neu zu prüfen.

Belege. Die Konstruktion wurde nicht nur behauptet, sondern nachgemessen. Über 3 150 geprüfte Seeds (alle Eval-Sets und die Trainingsbereiche) sowie eine erneute Stichprobe am 06.07.2026 (150 von 150) ergaben durchgängig **100 % Lösbarkeit**. Ein zweiter, schärferer Test prüft nicht nur die Weltgenerierung, sondern die gesamte Kette: Ein BFS-Orakel-Agent, der den kürzesten Weg direkt aus dem Distanzfeld des Kerns abliest, erreicht den Exit auf allen Test-Seeds in *exakt* der BFS-Distanz. Dass die tatsächlich gelaufene Schrittzahl mit der berechneten Distanz übereinstimmt, schließt neben unlösbaren Welten auch stille Fehler in Bewegungsmechanik, Kollisionsprüfung und Distanzfeld aus. Der gemessene Umwegfaktor von 1,10 (Abschnitt 4.3) stammt aus derselben Rechnung.

Damit ist eine methodische Frage entschieden, die im Evaluationsteil sonst offen bliebe: Wenn ein Agent in einer Welt scheitert, liegt es nie an der Welt.

4.5 Grafischer Client und Zuschaumodus

Der raylib-Client (ca. 4500 LOC) macht Stoneforge zu einem tatsächlich spielbaren 2D-Spiel: Ein Mensch steuert dieselbe Simulation, die auch das Training ansteuert, in Echtzeit über Tastatur und Maus. Jede Episode lässt sich damit nachspielen. Abbildung 9 zeigt die Oberfläche aus Sicht eines menschlichen Spielers.

Neben dem Spieler-Modus besitzt der Client einen KI-Zuschaumodus: `demo_agent.py` lädt einen trainierten Checkpoint und reicht dessen Aktionen über die Standardeingabe an



Abbildung 9: Benutzeroberfläche des spielbaren C++-Clients (Stoneforge 2D, echter Screenshot): Gezeigt ist die Spielumgebung aus Sichtweise eines menschlichen Spielers mit vollem Sichtbereich, dem Status-Header (Lebenspunkte, Energie, Schritte) und der Spielfigur im Zentrum.

den Client weiter, der dabei denselben Seed wie die Trainings-Umgebung verwendet und so dieselbe Welt erzeugt. Ein Mensch kann dem Agenten auf diese Weise beim Navigieren zusehen. Eine Launcher-GUI (Tkinter) bündelt Training, Evaluation und Abspielen zu einer gemeinsamen Oberfläche.

5 Implementierung der Lernumgebung

Das vorige Kapitel endet mit einem lauffähigen Spiel. Dieses Kapitel macht daraus eine Lernumgebung, und zwar ohne die Simulation dafür zu verändern: Der C++-Kern aus Abschnitt 4.1 bleibt derselbe, ein Mensch spielt weiterhin dieselbe Welt wie der Agent. Hinzu kommen eine Übersetzungsschicht, die das Spiel als Markov-Entscheidungsproblem darstellt, ein Reward-Design und die Trainings- und Monitoring-Infrastruktur.

5.1 Was der Agent davon sieht: POMDP-Charakter

Die Welt ist damit vollständig beschrieben: Sie ist reproduzierbar, lösbar und in ihrer Zielentfernung gesteuert. Für den Agenten ist von alledem nichts unmittelbar zugänglich. Er sieht nur einen lokalen 15×15 -Ausschnitt der Welt sowie einen *Luftlinien*-Kompass zum Exit, nicht den Weg. Wände, Sackgassen und Umwege außerhalb des Sichtfensters sind unbekannt: Die Aufgabe ist ein *Partially Observable Markov Decision Process* (POMDP, Abschnitt 2.2.2). Beide Perspektiven stellt Abbildung 3 auf Seite 8 gegenüber: links die Welt, wie sie tatsächlich aussieht, rechts das, was der Agent davon sieht. Für eine gedächtnislose Politik sind dabei verschiedene Weltpositionen ununterscheidbar (Abschnitt 7.2).

5.2 Umgebung, Beobachtung und Aktionen

Damit ein Lernverfahren die Simulation ansteuern kann, muss sie sich als *Markov-Entscheidungsproblem* präsentieren: ein Zustand, den der Agent beobachtet, eine Menge von Aktionen, aus denen er wählt, und ein Reward pro Schritt. Stoneforge ist von Haus aus kein solches Problem, sondern ein Spiel mit Kamera, Inventar und Echtzeitschleife. `StoneforgeWorldEnv` (Python) übersetzt zwischen beiden Welten und kapselt den C++-Kern hinter der Gymnasium-API. Was der Agent dabei zu sehen bekommt, listet Tabelle 8.

Bereich	Inhalt	Dim.
Lokales Grid	15×15 Tile-Typen um den Agenten (Sichtradius 7)	225
HP	Lebenspunkte (normiert)	1
Kompass	exitDx, exitDy (Luftlinie, normiert)	2
Episodenfortschritt	Schritt / 4000	1
Gesamt (Env v11)		229

Tabelle 8: Observation der Umgebungsversion v11. Die früheren Features *Energie* und *Inventar* waren im RL-Setup konstant (tote Features) und wurden am 06.07.2026 entfernt (vorher 231 Dimensionen).

Der Aktionsraum ist `Discrete(4)` (hoch, runter, links, rechts). Die im Spiel vorhandenen Aktionen Mining, Bauen und Kampf sind im RL-Binding deaktiviert und werfen bei Aufruf einen Fehler; sie existieren nur im spielbaren Client (Abschnitt 4.5). Eine Episode endet bei Erreichen des Exits (Sieg), nach 4000 Schritten (Timeout) oder als Early-Stop-Truncation nach 256 Schritten ohne positiven Reward.

Tabelle 9 führt daneben „Tod“ als terminales Ereignis und einen Schadens-Penalty auf. Beide können im RL-Pfad nicht eintreten: Die einzigen Schadensquellen des Simulationskerns sind Mob-Nähe und Verhungern, und die Umgebung schaltet beides ab (`disable_mobs`, `disable_energy`). Die Lebenspunkte bleiben damit über die gesamte Episode konstant. **Das HP-Merkmal in Tabelle 8 ist folglich ein totes Feature**, so wie es Energie und Inventar vor der v11-Revision waren (Abschnitt 10.7); es wurde dort übersehen und ist eine der 229 Dimensionen ohne Informationsgehalt. Der Schadens-Penalty ist entsprechend inert. Beides

ist folgenlos für die berichteten Ergebnisse, aber bei einer Neufassung der Beobachtung zu streichen.

5.3 Die Schnittstelle: was bei `reset()` und `step()` tatsächlich passiert

Die Aufgabenteilung zwischen den beiden Sprachen ist eng geschnitten. Der C++-Kern besitzt den Weltzustand und rechnet alles, was pro Schritt anfällt: Weltgenerierung, Bewegung samt Kollision, das BFS-Distanzfeld und die Reward-Terme aus Tabelle 9. Die Python-Seite besitzt keinen Weltzustand. Sie führt Buch über die Episode, formt die Beobachtung und implementiert genau jene Regeln, die sich ohne Neuübersetzung ändern lassen sollten. Das pybind11-Binding dazwischen ist bewusst dünn: Es exportiert im Wesentlichen `reset(seed)`, `step(action)` und eine Handvoll lesender Abfragen wie `current_bfs_distance_to_exit()` oder `player_pos()`. Es enthält keine Spiellogik. Wer den Reward ändern will, ändert C++ und übersetzt neu; wer das Curriculum ändert, ändert Python.

Ein `reset()` in vier Schritten. Der Aufruf legt eine neue Episode an, und die Reihenfolge ist nicht beliebig:

1. **Seed bestimmen.** Übergibt der Aufrufer einen Seed, wird dieser verwendet. Das ist der Fall im Evaluationsprotokoll, wo die Testsets aus festen Seeds bestehen. Im Training zieht die Umgebung stattdessen selbst einen Zufallsseed, sodass jede Episode eine neue Welt sieht.
2. **Episodenbuchführung zurücksetzen.** Schrittzähler, Besuchszähler je Tile, Zähler für Schritte ohne positiven Reward und der Aktionspuffer werden geleert. Diese Zähler tragen den Revisit-Penalty und den Early Stop, existieren also nur auf der Python-Seite.
3. **Weltgenerator-Konfiguration neu setzen.** Erst danach folgt der eigentliche Reset des Kerns. Der Grund steht im nächsten Absatz.
4. **Kern zurücksetzen und Beobachtung bauen.** Der Kern erzeugt die Welt zum Seed, platziert den Exit (Abschnitt 4.4) und liefert einen rohen Zustandsvektor. Die Python-Seite normiert ihn und schneidet die toten Legacy-Felder heraus.

Ein Detail, das lange Zeit die Messungen verfälscht hat. Die Weltgenerator-Konfiguration liegt im C++-Kern als *prozessglobale* Struktur. Das ist für ein Spiel unproblematisch, in dem genau eine Welt existiert. Im Training existieren aber 16 parallele Umgebungen, und der Evaluations-Callback erzeugt zusätzlich eine eigene. Setzt diese Evaluations-Umgebung ihr Distanzfenster auf 35 bis 45, überschreibt sie damit das Fenster *aller* Trainingsumgebungen, obwohl sie fachlich nichts miteinander zu tun haben. In Phase 3 verschob der Evaluations-Callback auf diese Weise die Trainingsverteilung nach der ersten Evaluation still von 25 bis 45 auf 35 bis 45. Das Curriculum trainierte also nicht mehr auf dem Bereich, der in der Konfiguration stand.

Die Lösung ist die dritte Zeile der Aufzählung oben: Jede Umgebung stempelt ihre Konfiguration vor jedem einzelnen `reset()` neu, statt sie einmalig im Konstruktor zu setzen. Nachweisen lässt sich der Fehler mit einem BFS-Orakel: Vor dem Fix brauchte es auf den Test-Seeds durchschnittlich 8,3 Schritte, nachdem eine zweite Umgebung mit dem Fenster 5 bis 12 erzeugt worden war; danach wieder die korrekten 40,2 (Abschnitt 10.7). Der Fehler war im Python-Code nicht zu sehen, denn dort stand die richtige Konfiguration. Sichtbar wurde er erst an Zahlen, die sich nicht erklären ließen.

Ein `step()` in zwei Hälften. Der Aufruf reicht die Aktion an den Kern durch, der Bewegung, Kollision und Reward rechnet und den neuen Zustand zurückgibt. Danach ergänzt die Python-Seite drei Regeln, die absichtlich nicht in C++ stehen, weil sie zur Trainingsmethodik gehören und nicht zum Spiel:

- **Revisit-Penalty.** Ein Besuchszähler je Tile bestraft ab dem 26. Besuch desselben Feldes, gedeckelt ab dem 50. Er greift damit nur bei ausgeprägtem Kreiseln (Tabelle 9).
- **Early Stop.** Bleiben 256 Schritte ohne positiven Reward, endet die Episode als *Truncation*. Ohne diese Regel verbraucht ein feststeckender Agent bis zu 4000 Schritte pro Episode und der Durchsatz bricht ein.
- **Seed-Pool („Swarm“).** Erfolgreiche Seeds werden gesammelt und in 30 % der Fälle wiederholt.

Zwei Dinge sind an dieser Aufteilung erwähnenswert. Erstens ist der Early Stop eine *Truncation* und kein Terminal: Der Wert des Endzustands wird weiter gebootstrapped, die Episode gilt nicht als Misserfolg im Sinne des Zielkriteriums. Zweitens ist die Trennung nicht kosmetisch. Jede Änderung an den drei Python-Regeln wirkt sofort, während jede Änderung am Reward einen Rebuild des Kerns erfordert. Über die Laufzeit des Projekts hat das die Iterationsgeschwindigkeit spürbar bestimmt.

5.4 Reward-Design

Das Reward-Design folgt dem Prinzip: *sparse Primärsignal + theoretisch sauberes dichtes Shaping*. Kernstück ist Potential-Based Reward Shaping (PBRS) auf der BFS-Pfaddistanz zum Exit:

$$F(s, s') = \gamma \Phi(s') - \Phi(s), \quad \Phi(s) = -\frac{d_{\text{BFS}}(s)}{128}, \quad (7)$$

mit $\gamma = 0.999$ (identisch zum RL-Discount) und Gewicht $\beta = 2.5$. Ein Tile echter Fortschritt liefert damit rund +0,02; abzüglich des Schritt-Malus von −0,01 (Tabelle 9) verbleiben netto etwa +0,01. Listing 6 zeigt die Umsetzung im Kern.

```

1 constexpr float PBRS_MAX_DIST = 128.0F; // Normierung der BFS-Distanz
2 constexpr float PBRS_GAMMA    = 0.999F; // identisch zum RL-Discount
3 constexpr float PBRS_BETA     = 2.5F;   // +0,01 netto pro Tile bei d=40
4
5 const float phi_prev = -static_cast<float>(previousDistance) /
    PBRS_MAX_DIST;
6 const float phi_cur  = -static_cast<float>(currentDistance) /
    PBRS_MAX_DIST;
7 const float shaping  = PBRS_GAMMA * phi_cur - phi_prev;
8 reward += PBRS_BETA * shaping;

```

Listing 6: Potential-Based Reward Shaping auf der BFS-Distanz (`Simulation::computeReward()`, `src/core/simulation.cpp`). Der Shaping-Discount ist bewusst identisch zum Discount des Lernverfahrens; nur unter dieser Bedingung ist F policy-invariant [23].

Beide Konstanten sind aus dieser Netto-Vorgabe rückwärts bestimmt. Der Nenner 128 normiert die BFS-Distanz auf die Größenordnung der übrigen Reward-Terme und entspricht zugleich der Skalierung des BFS-Puffers im C++-Kern, sodass Φ auch im ungünstigsten Fall betragsmäßig klein gegenüber dem Terminal-Reward bleibt. Das Gewicht $\beta = 2,5$ ist so gewählt, dass sich bei der mittleren Eval-Distanz $d \approx 40$ genau die genannten +0,01 netto pro Fortschritts-Tile ergeben. Ein zuvor verwendetes $\beta = 5,0$ wurde halbiert, nachdem ein zu klein

dimensionierter BFS-Puffer in Kombination mit dem Manhattan-Fallback zeitweise Beträge um +2,5 pro Schritt erzeugt und damit ein falsches Lernziel gesetzt hatte (Abschnitt 10). PBRs ist beweisbar *policy-invariant*; es beschleunigt die Konvergenz, ohne die optimale Politik zu verändern [23]. Entscheidend ist, dass das Potential die *echte* Pfaddistanz (BFS durch Wände gerechnet) verwendet, nicht die Luftlinie: Ein Luftlinien-Potential würde den Agenten systematisch in Sackgassen führen. Abbildung 10 zeigt den Unterschied an einem minimalen Beispiel.

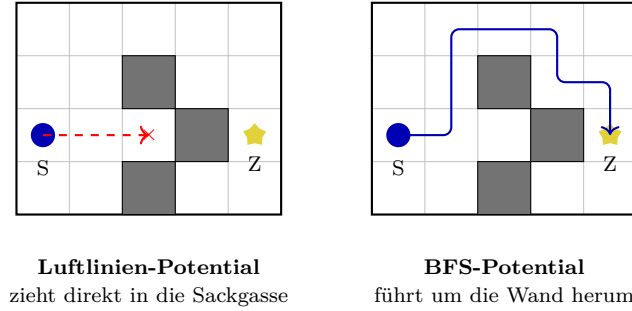


Abbildung 10: Minimalbeispiel einer dreiseitig umschlossenen Sackgasse (grau) zwischen Start S und Ziel Z. Links: Ein Potential auf Luftlinien-Distanz nimmt an jeder Position den kürzesten geometrischen Abstand zum Ziel und zieht den Agenten geradewegs in die Sackgasse, wo er endet (×), ohne die Wand einzurechnen. Rechts: Das in dieser Arbeit verwendete Potential auf BFS-Pfaddistanz kennt nur begehbare Wege und führt korrekt um die Wand herum. Schematische Darstellung, kein realer Weltausschnitt.

Diese Garantie gilt allerdings nur für den PBRs-Term selbst. Die übrigen Komponenten in Tabelle 9 (Explorations-Bonus, Schritt-, Loop-, Idle-, Schadens- und Revisit-Penalty) sind gewöhnliche Reward-Terme und verändern die optimale Politik prinzipiell mit. Der frühere eskalierende Wand-Penalty ($-0.05/-0.25$) wurde aus genau diesem Grund entfernt: Die Summe vieler kleiner Strafen machte Erkunden in engen Korridoren unwirtschaftlicher als Stillstand. Der Revisit-Penalty ist der einzige verbliebene Term dieser Art; er greift erst ab dem 26. Besuch *desselben* Tiles und damit ausschließlich bei ausgeprägtem Kreisel, wirkt in der Sache aber in dieselbe Richtung und ist bei einer Neufassung des Reward-Designs (Abschnitt 8) mit zu prüfen.

Komponente	Wert
Exit erreicht (terminal)	+100
Timeout / Tod (terminal)	-10
PBRs (BFS-Distanz, pro Schritt)	$\approx \pm 0.02$
Explorations-Bonus (neue Tile)	+0.02
Schritt-Penalty	-0.01
2-Schritt-Loop-Penalty	-0.05
Idle-Penalty (Aktion ohne Positionswechsel)	-0.04
Schadens-Penalty (pro verlorenem HP)	-0.5
Revisit-Penalty (ab 26. Besuch eines Tiles)	-0.03 bis -0.06

Tabelle 9: Vollständige Reward-Komponenten der Umgebungsversion v11. Die oberen Terme stammen aus `Simulation::computeReward()` (C++-Kern); der Revisit-Penalty wird im Python-Wrapper `StoneforgeWorldEnv.step()` addiert und wächst linear von -0.03 (26. Besuch) bis zur Deckelung bei -0.06 (ab dem 50. Besuch desselben Tiles).

5.5 Trainings-Pipeline

`train_curriculum.py` orchestriert die vier Phasen aus Tabelle 10. Ein Lauf ist damit kein einzelnes Training, sondern eine Kette aus vier Trainings, die einander das Modell weiterreichen. Vier Bausteine tragen diese Kette.

Parallele Umgebungen. Je Lauf arbeiten 16 Umgebungsinstanzen in einem `DummyVecEnv`. Sie laufen im selben Prozess, nacheinander pro Schritt, und bündeln ihre Beobachtungen zu einem Batch. Der Gewinn liegt nicht in echter Parallelität, sondern darin, dass ein Rollout Erfahrung aus 16 verschiedenen Welten gleichzeitig enthält, statt aus einer.

Evaluation und Gating. Alle 25 000 Schritte misst ein Callback das aktuelle Modell auf 50 Validierungs-Seeds, in beiden Betriebsarten. Diese Messung erfüllt zwei Aufgaben zugleich. Sie entscheidet erstens, wann eine Phase endet: Erreicht die maßgebliche Metrik die Ziel-Erfolgsquote in zwei aufeinanderfolgenden Evaluationen, schaltet das Curriculum weiter. Die Doppelbedingung filtert Ausreißer, denn ein einzelner guter Messpunkt ist bei dieser Lerndynamik kein Beleg (Abschnitt 6.4). Zweitens wählt sie das beste Modell der Phase aus, und genau dieses wird beim Phasenwechsel geladen. Welche Metrik dabei zählt, wechselt: Die Phasen 1 und 2 gaten auf der stochastischen Erfolgsquote, die Phasen 3 und 4 auf der deterministischen.

Absenkung der Exploration. Der Entropie-Koeffizient hält die Politik früh breit gestreut und muss später aus dem Weg. In Phase 3 senkt ein Callback ihn linear von 0,05 auf 0,001 über 500 000 Schritte, Phase 4 trainiert mit 0,0001. Diese letzte Phase existiert nur, um die deterministische Politik nachzuziehen (Abschnitt 7.5).

Seed-Pool. Ein optionaler Pool („Swarm“) sammelt bereits gelöste Seeds und startet 30 % der Episoden auf einem davon. Bei jedem Phasenwechsel wird er geleert, damit gelöste Welten der leichten Phase nicht die schwere Phase verwässern.

Jeder Lauf legt pro Phase Modell-Checkpoints sowie `eval_history.json` und `config.json` ab; alle Experimente sind zusätzlich im `CHANGELOG.md` des Repositories dokumentiert.

5.6 Monitoring und Werkzeuge

Trainingsmetriken werden nach TensorBoard geschrieben. Neben der bereits eingeführten `explained_variance` (Güte der Wertfunktion, Abschnitt 2) und der Entropie der Politik ist dabei `approx_kl` von Bedeutung: ein Schätzwert dafür, wie stark sich die Politik durch ein Update verändert hat. Werte nahe null zeigen an, dass das Training faktisch stillsteht; sehr große Werte deuten auf zu grobe Schritte hin. Die Größe dient in dieser Arbeit als schneller Test, ob ein Lauf überhaupt lernt (Anhang 10). Zusätzlich streamt ein WebSocket-Server den Trainingszustand in eine Browser-Ansicht: Erfolgsquoten-Kurven (deterministisch und stochastisch), Minimaps der 16 parallelen Agenten und eine Explorations-Heatmap.

Abbildung 11 fasst die Implementierung dieses Kapitels zusammen: Ein einziger C++-Simulationskern wird auf zwei unabhängigen Wegen angesteuert, einmal über das Python-Binding zum Training, einmal direkt vom grafischen Client. Beide Pfade erzeugen bei gleichem Seed dieselbe Welt, was den KI-Zuschaumodus (Abschnitt 4.5) überhaupt erst sinnvoll macht.

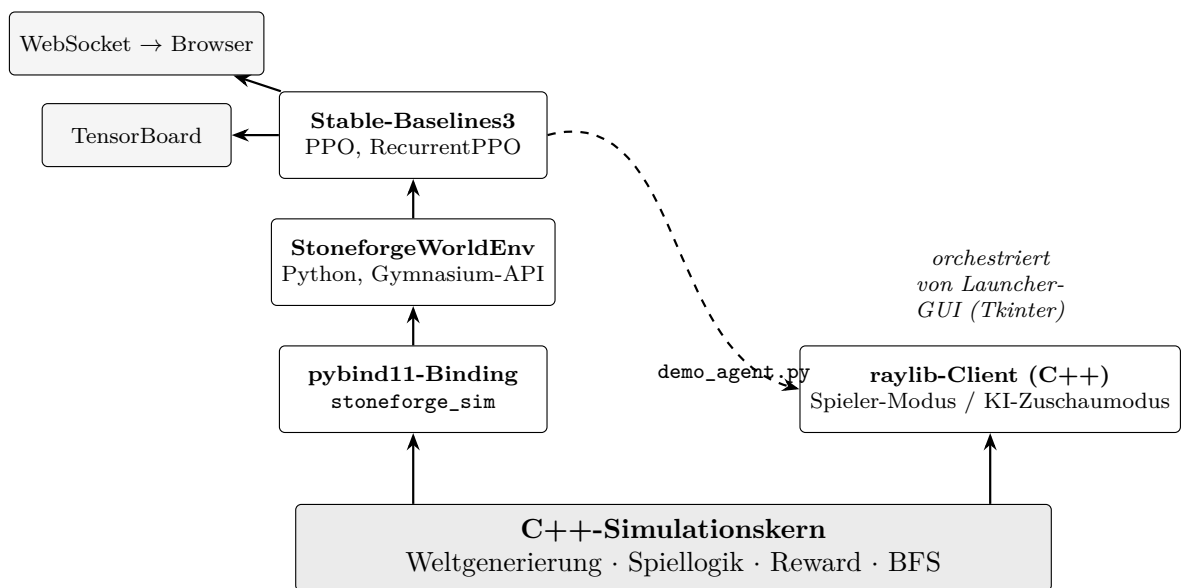


Abbildung 11: Ein C++-Simulationskern, zwei Ansteuerungswege: Training über pybind11/Gymnasium/Stable-Baselines3 (links) und der interaktive raylib-Client (rechts), der denselben Kern direkt einbindet. `demo_agent.py` verbindet beide Pfade für den KI-Zuschaumodus, indem es die Aktionen eines geladenen Checkpoints über Standardeingabe an den Client weiterreicht; da beide Pfade denselben Seed verwenden, entsteht dieselbe Welt.

6 Methodik

6.1 Algorithmus

Trainiert wird RecurrentPPO (PPO [18] mit LSTM-Politik [20]; beide Verfahren in Abschnitt 2) aus *Stable-Baselines3* / *sb3-contrib* [19]. Zentrale Hyperparameter: $n_{\text{steps}} = 256$, $n_{\text{epochs}} = 10$, Lernrate $3 \cdot 10^{-4}$, $\gamma = 0.999$, $\text{ent_coef} = 0.05$, LSTM mit 256 Hidden Units (separater Critic-LSTM), 16 parallele Umgebungen.

Begründung der Verfahrenswahl. Drei Eigenschaften der Aufgabe führen auf RecurrentPPO. Erstens ist der Aktionsraum diskret (**Discrete(4)**); das lässt wertbasierte und policy-gradient-Verfahren gleichermaßen zu und entscheidet für sich genommen noch nichts. Zweitens ist die Aufgabe partiell beobachtbar (Abschnitt 2.2.2), wo eine gedächtnislose Politik im Allgemeinen nicht optimal ist; das begründet die rekurrente statt der reaktiven Politik. Drittens war die Umgebung während der Entwicklung selbst noch Gegenstand von Korrekturen, und hier unterscheiden sich die Verfahrensklassen praktisch: Ein off-policy-Verfahren konserviert die Folgen eines fehlerhaften Reward-Signals im Replay-Buffer über dessen Korrektur hinaus, ein on-policy-Verfahren nicht. Diese Robustheit gegenüber Fehlern in der eigenen Umgebung gab den Ausschlag für PPO.

Die Trainings-Infrastruktur unterstützt daneben DQN und A2C. Auf der finalen Umgebung v11 wurde keines von beiden vermessen; eine quantifizierte nicht-rekurrente Vergleichsbasis steht damit aus und wird als Grenze der Arbeit ausgewiesen (Abschnitt 8).

Zustandekommen der Konfiguration. Die in diesem Kapitel beschriebene Konfiguration ist nicht das Ergebnis einer einmaligen Wahl, sondern von zwölf Umgebungs- und Trainingsiterationen. Der Verlauf einschließlich der gescheiterten Varianten und ihrer Ursachenanalysen ist in Anhang 10 dokumentiert; dort begründete Festlegungen werden im Folgenden ohne Wiederholung der Herleitung übernommen. Die methodischen Regeln, die sich daraus für Training und Messung ergeben, stehen in Abschnitt 6.3.

6.2 Curriculum

Das Training verläuft in vier Phasen steigender Exit-Distanz. Der Phasenwechsel ist leistungsbasiert: Eine Phase endet, sobald die Ziel-Erfolgsquote in zwei aufeinanderfolgenden Evaluationen erreicht wird. Zusätzlich begrenzt ein festes Schrittbudget jede Phase nach oben; wird das Ziel nicht erreicht, endet die Phase mit Erschöpfung dieses Budgets. Beide Abbruchbedingungen sind in Tabelle 10 aufgeführt, weil die zweite in den Auswertungsläufen die tatsächlich wirksame war (Abschnitt 7.3).

Das Distanz-Curriculum staffelt dabei implizit auch die *Beobachtbarkeit* des Ziels, und damit die Fähigkeit, die je Phase gelernt werden muss. Eine Messung über je 200 frisch erzeugte Welten pro Phasenkonfiguration ergibt (eine Eigenschaft des Generators, unabhängig von den Evaluations-Seedmengen): In Phase 1 liegt der Exit beim Episodenstart in **72 %** der Welten bereits im 15×15 -Sichtfeld (zu lernen: „geh zum sichtbaren Ziel, weiche lokalen Hindernissen aus“); in Phase 2 nur noch in **6 %** (zu lernen: dem Kompass folgen, Umwege um Hindernisse); in Phase 3 in **0 %** (reine Kompass-Navigation über lange Distanz, inklusive Sackgassen, die nur mit Gedächtnis überwindbar sind). Ein zusätzliches Curriculum über die Sichtfeldgröße (in der Literatur als Übergang von voller zu partieller Beobachtbarkeit beschrieben) wäre daher redundant; zudem wäre es hier riskant, weil ein großes Sichtfeld anfangs eine *reaktive* Strategie ohne Gedächtnisnutzung belohnt, die beim Verkleinern des Fensters wertlos wird (dieselbe Nicht-Übertragbarkeit, die die Gegenüberstellung in Abschnitt 7.2 für das BFS-Orakel zeigt).

Phase	Exit-Distanz		Budget	Ziel-SR	Gating	ent_coef
	Training	Eval				
1	5–12	5–12	500k	85 %	stoch.	0.05
2	12–25	12–25	500k	70 %	stoch.	0.05
3	25–45	35–45	1,0M	70 %	det.	0.05 $\rightarrow$ 0.001
4	25–45	35–45	200k	60 %	det.	0.0001

Tabelle 10: Curriculum (Stand v11); *Budget* bezeichnet die maximale Zahl an Trainings-schritten der Phase. Angegeben ist die Exit-Distanz im Training und in der phaseninternen Evaluation; in den Phasen 1 und 2 sind beide identisch, ab Phase 3 wird auf dem schwersten Teilbereich evaluiert, um die Vergleichbarkeit mit dem finalen Protokoll herzustellen. Das Episodenlimit der Evaluation beträgt in allen Phasen 4000 Schritte. In Phase 3 wird `ent_coef` linear über 500k Schritte abgesenkt (Annealing), Phase 4 ist der Greedy Fine-Tune. Phase 1/2 gaten auf der stochastischen SR, da die Politik dort mit `ent_coef`=0.05 absichtlich nahe-uniform ist und die deterministische SR konstruktionsbedingt kein Signal trägt. Die Gating-Metrik steuert zugleich die Auswahl des besten Modells der Phase. Quelle: PHASES in `scripts/train_curriculum.py`.

6.3 Evaluationsprotokoll

Alle in dieser Arbeit berichteten Messungen folgen dem hier festgelegten Protokoll. Es ist bewusst als Teil des Versuchsaufbaus dokumentiert und nicht als technisches Detail: Mehrere seiner Festlegungen, insbesondere das Episodenlimit und die Seed-Trennung, haben sich als ergebnisbestimmend erwiesen (Anhang 10).

Metrik. Zentrale Metrik ist die *Erfolgsquote* (*Success Rate*, SR): der Anteil von 50 festen, trainingsfremden Welten, in denen der Agent den Exit innerhalb des Episodenlimits erreicht. Ergänzend wird die *Pfadeffizienz* η berichtet, das Verhältnis aus kürzestem Weg und tatsächlich benötigten Schritten; sie wird dort eingeführt, wo sie erstmals gebraucht wird (Abschnitt 7.4). Als Diagnosemetrik des Trainings dienen die `explained_variance` des Critics und `approx_kl`.

Episodenlimit. Gemessen wird stets mit dem vollen Limit von 4000 Schritten, identisch zum Maximum der Umgebung. Ein kürzeres Limit unterschätzt kompetente Politiken erheblich, da erfolgreiche Episoden ohne Orakel lange Suchwege enthalten; dasselbe Modell misst sich bei einem Limit von 600 als 48%-Agent, bei 4000 als 86%-Agent (Abschnitt 10.6).

Neben dem Limit bleibt in der Evaluation eine zweite Abbruchbedingung aktiv: die Early-Stop-Truncation nach 256 Schritten ohne positiven Reward (Abschnitt 5). Sie ist Teil der Umgebung und wird in der Evaluation nicht abgeschaltet; sie gilt damit für den Agenten und für die Referenzpolitiken (Abschnitt 7.4) gleichermaßen. Zwei Konsequenzen sind zu beachten. Erstens ist das effektive Budget einer Episode nicht in jedem Fall 4000 Schritte, sondern kann früher enden; die berichteten Erfolgsquoten sind insofern eine untere Schranke gegenüber einem Protokoll ohne Truncation. Zweitens hängt die Bedingung von der Explorationsrate der jeweiligen Politik ab, da bereits ein neu betretenes Tile oder ein BFS-Fortschritt den Zähler zurücksetzt: Eine flächig suchende Politik löst die Truncation seltener aus als eine, die sich festfährt. Das Protokoll ist für alle verglichenen Politiken identisch, die Bedingung wirkt aber nicht auf alle gleich stark. Ein Vergleich mit abgeschalteter Truncation steht aus und ist im Ausblick (Abschnitt 8) als Prüfschritt vermerkt.

Seed-Trennung. Die Seed-Mengen sind disjunkt und haben feste Rollen. Der Validierungsbereich umfasst die Seeds 6000–6199; davon steuern die ersten 50 (6000–6049) Phasenwechsel

und Modellauswahl. Der Bereich darüber hinaus ist für Diagnosemessungen vorgesehen, die mehr als 50 Welten benötigen. Solche Messungen nutzen den Validierungsbereich als Ganzes und schließen die Selektions-Seeds mit ein: Die nach Startdistanz gruppierte Auswertung in Abschnitt 7.5 etwa läuft auf 6000–6119 (120 Welten). Das ist unkritisch, weil es sich um eine Diagnose und nicht um eine Leistungsaussage handelt; die berichteten Erfolgsquoten stammen ausschließlich aus A und B. Testset A (7000–7049) und Holdout B (8000–8049) werden ausschließlich für die finale Messung verwendet und gehen in keine Auswahl- oder Diagnoseentscheidung ein.

Betriebsarten. Jede Evaluation misst *beide* Betriebsarten: stochastisch (Sampling aus $\pi(a \mid s)$) und deterministisch (Argmax). Bei der rekurrenten Politik werden LSTM-Zustand und Episoden-Start-Signal an jeden `predict()`-Aufruf durchgereicht und zu Episodenbeginn zurückgesetzt, damit kein Zustand über Episodengrenzen übertragen wird; dies ist ein häufiger, leicht übersehener Fehler beim Evaluieren rekurrenter Politiken [21]. Listing 7 zeigt die Umsetzung. Der Fehler ist deshalb tückisch, weil er nicht abstürzt: Wer den Zustand weglässt, misst ein Gedächtnismodell ohne Gedächtnis und erhält systematisch zu niedrige Werte, ohne dass die Messung als fehlerhaft auffällt.

```

1 def reset(self):
2     return (None, np.ones((1,), dtype=bool))    # Episodenstart-Signal
3
4 def __call__(self, obs, ctx):
5     state, episode_start = ctx
6     action, state = self.model.predict(
7         obs, state=state, episode_start=episode_start,
8         deterministic=self.deterministic,
9     )
10    return int(action), (state, np.zeros((1,), dtype=bool))

```

Listing 7: Führung von LSTM-Zustand und Episoden-Start-Signal im Evaluationsprotokoll (ModelPolicy in `scripts/eval_baselines.py`). Das Start-Signal steht nur im ersten Schritt einer Episode auf `True` und setzt damit den rekurrenten Zustand zurück.

Aggregation über Läufe. Für das Endergebnis werden sieben unabhängige Trainingsläufe gemittelt, mit Stichproben-Standardabweichung (`ddof = 1`) und 95 %-Konfidenzintervall über die t -Verteilung. Berichtet wird stets der Mittelwert, nie der beste Lauf [6].

Entscheidungen auf Kurven, nicht auf Endpunkten. Die Lerndynamik dieser Umgebung ist hochvariabel; einzelne Messpunkte einer schwankenden Eval-Kurve können jede beliebige Hypothese scheinbar stützen. Vergleiche zwischen Konfigurationen werden deshalb nur auf ganzen Eval-Kurven gezogen, nicht auf Endständen. Der Fall, der zu dieser Regel führte, ist in Abschnitt 10.8.2 dokumentiert.

6.4 Methodische Fallstricke der Evaluation

Drei Beobachtungen dieses Projekts betreffen nicht den Agenten, sondern die Frage, was eine RL-Messung überhaupt aussagt. Sie werden hier gebündelt, weil sie über den Einzelfall hinaus gelten und mehrere der oben getroffenen Protokoll-Festlegungen begründen; ihre vollständige, chronologische Herleitung steht im Entwicklungsverlauf (Anhang 10).

Das Episodenlimit ist Teil der Metrik, nicht ihres Umfelds. Dasselbe Modell misst sich bei einem Eval-Limit von 600 Schritten als 48-%-, bei 4 000 Schritten als 86-%-Agent (Abschnitt 10.6). Ohne Orakel enthält jede erfolgreiche Episode lange Suchwege; ein zu knappes

Limit schneidet gerade die kompetenten, aber langsamen Läufe ab und lässt sie als Modell-schwäche erscheinen. Ein Episodenlimit ist deshalb kein Ausführungsdetail, sondern eine die Erfolgsquote mitdefinierende Größe und gehört zu jedem berichteten SR-Wert angegeben.

Ein einzelner Eval-Snapshot ist keine Validierung. Die Lerndynamik dieser Umgebung ist so variabel, dass ein einzelner Messpunkt einer schwankenden Kurve fast jede Hypothese scheinbar stützt. Eine Änderung der `batch_size` ($8 \rightarrow 64$) galt nach einem günstigen Snapshot zunächst als „validiert“, erwies sich bei Betrachtung der *gesamten* Eval-Kurve aber als destabilisierend (*explained variance* des Critics nahe 0, chaotisch oszillierende SR; Abschnitt 10.8.2). Hyperparameter-Entscheidungen werden deshalb nur auf ganzen Kurven getroffen, nie auf Endständen.

Eine hohe Erfolgsquote ist nicht automatisch aussagekräftig. Bei einem Episodenlimit von 4000 Schritten und einer mittleren Optimallänge von rund 40 Feldern (Faktor 100) diskriminiert die Erfolgsquote kaum: Schon eine ungelernete Kompass-Heuristik erreicht 92 % (Abschnitt 7.4). Die Erfolgsquote ist daher stets zusammen mit der Pfadeffizienz und einer ungelerten Referenzpolitik zu berichten – erst der Bezugspunkt macht sie interpretierbar. Diese drei Punkte sind zugleich der Grund, warum das Evaluationsprotokoll oben so ausführlich als Teil des Versuchsaufbaus und nicht als technische Randnotiz dokumentiert ist.

6.5 Herleitung der Erfolgsschwellen

Die folgende Herleitung setzt Kenntnis der Umgebung voraus und steht deshalb hier und nicht bei der Nennung der Kriterien in Abschnitt 1.4.

Die dort genannten Schwellen (70 % auf Testset A, 60 % auf Holdout B) ersetzen die im Exposé angesetzte Metrik von $\geq 90\%$ über 100 unbekannte Seeds. Drei Eigenschaften der Aufgabe, die zu Projektbeginn noch nicht bekannt waren, machen einen Wert in dieser Höhe unerreichbar:

1. **Der POMDP-Charakter der Umgebung.** Bei einem Sichtradius von 7 und nicht sichtbarem Ziel ist die Aufgabe partiell beobachtbar (Abschnitt 2.2.2). Eine deterministische Politik über der *Beobachtung* ist damit nicht mehr garantiert optimal [15], hinzu kommt die epistemische Unsicherheit über den Seed [22]. Eine Schwelle von 90 % unterstellt dagegen eine nahezu vollständig lösbare Aufgabe.
2. **Die tatsächliche Schwierigkeit der Distanzvorgabe.** Bis zur Umgebungsrevision v11 wurde die Exit-Distanz als Luftlinie gemessen; eine Vorgabe von „35–45“ entsprach real einem Laufweg von 42–75 Feldern (Abschnitt 10.7).
3. **Die Streuung über Trainingsläufe.** Sie beträgt rund ± 12 Prozentpunkte (Tabelle 14). Eine Schwelle von 90 % liegt weit außerhalb dessen, was das Verfahren hier stabil liefert.

Zur Vorab-Festlegung. Eine nachträglich abgesenkte Schwelle wäre wertlos, weshalb der Zeitpunkt der Festlegung hier als Beleg und nicht als Chronik angeführt wird: Die Schwellen 70 %/60 % stehen seit dem 16.05.2026 im Experiment-Changelog, rund zwei Monate vor den finalen Läufen (08.07.) und der $n = 7$ -Auswertung (17.07.). Sie wurden also nicht in Kenntnis der hier berichteten Ergebnisse gewählt und sind damit kein Fall von *HARKing* (*Hypothesizing After the Results are Known* [26]). Dass das Testset-Kriterium am Ende dennoch verfehlt wird, zeigt, dass auch die revidierte Schwelle keine bequeme war.

Gegen die ursprüngliche Exposé-Metrik gemessen ergeben Testset A und Holdout B zusammen die geforderten 100 unbekannten Seeds und einen Mittelwert von $66,3\% \pm 12,1$ (stochastisch); die 90 %-Schwelle wird deutlich verfehlt. Das Exposé nennt als zweite Zielgröße

die Minimierung der *Steps per Episode*. Auch sie wird verfehlt, und zwar deutlicher als die Erfolgsquote: Die in Abschnitt 7.4 berichtete Pfadeffizienz von $\eta \approx 0,05$ entspricht rund dem Zwanzigfachen des kürzesten Weges.

6.6 Versuchsaufbau

Die Angaben dieses Abschnitts folgen den Reproduzierbarkeits-Checklisten von Pineau et al. [27] und Henderson et al. [6]; Anhang 9 führt sie zusammen. Alle Läufe der Arbeit teilen die in Tabelle 11 gelistete Basiskonfiguration. Je Lauf existiert eine `config.json` mit genau diesen Werten; die vollständige Historie inklusive Zwischenständen liegt im Experiment-Changelog (`CHANGELOG.md`).

Auf dieser Basis wurden drei Varianten mit je einem veränderten inhaltlichen Faktor gerechnet (E1 Critic-Gewicht, E2 Curriculum-Zuschnitt, E3 Gedächtnisgröße). Da sie einer Fragestellung dienen, die sich erst aus den Messergebnissen stellt, sind Setup und Befund gemeinsam in Abschnitt 7.6 dargestellt.

Komponente	Wert
Algorithmus	RecurrentPPO (SB3-contrib), MlpLstmPolicy
n_{steps} / <code>batch_size</code> / n_{epochs}	256 / 8 / 10
Lernrate / γ / λ_{GAE} / clip	$3 \cdot 10^{-4}$ / 0.999 / 0.95 / 0.2
<code>ent_coef</code>	0.05 (Phase 3: Annealing auf 0.001; Phase 4: 0.0001)
<code>vf_coef</code>	0.5
LSTM	1 Schicht, 256 Hidden Units, separater Critic-LSTM
Beobachtung	229 Features (Grid 15×15 + HP + Kompass + Step-Frac), Env v11
Parallele Umgebungen	16 (DummyVecEnv), Device CPU
Seed-Pool („Swarm“)	aktiv, Sampling-Wahrscheinlichkeit 0.3
Curriculum	4 Phasen (Tab. 10), leistungsbasiert mit Schrittbudget
Eval	Testset A 7000–7049, Holdout B 8000–8049, Cap 4000, det + stoch

Tabelle 11: Gemeinsame Basiskonfiguration aller Läufe (Quelle: `config.json` der v12-Läufe).

7 Evaluation

Das Evaluationsprotokoll (Seed-Trennung, Metrik, Betriebsarten) ist in Abschnitt 6 beschrieben. Bewertet werden beide Teile des Projekts, und zwar in der Reihenfolge, in der sie aufeinander aufbauen: zuerst die Plattform gegen die Anforderungen Z1 (Abschnitt 7.1), dann der Agent gegen die Schwellen Z2. Innerhalb des zweiten Teils sind die Ergebnisse nach Beweislast geordnet: Der Vergleich gegen ungelernte Referenzpolitiken (Abschnitt 7.4) schränkt die Aussagekraft aller vorher berichteten Zahlen ein und steht deshalb nach ihnen.

7.1 Bewertung der Plattform (Z1)

Die Anforderungen aus Abschnitt 1.4 sind funktionaler Art; sie werden hier zusammengeführt und gegen die jeweils erhobene Evidenz gestellt.

Anforderung	Nachweis	Status
Determinismus	Weltgenerierung und Simulation hängen ausschließlich von Seed und Koordinaten ab. Die chunkweise Erzeugung ist durch die Halo-Berechnung der Glättungsstufe reihenfolgeunabhängig; ein prozessglobales Konfigurationsleck, das dies zeitweise verletzte, wurde gefunden und behoben (Abschnitt 10.7). Zwei Trainingspfade stimmen über 8 192 Schritte <i>bit-identisch</i> überein (Abschnitt 10.6).	erfüllt
Garantierte Lösbarkeit	Exit-Platzierung ausschließlich über BFS-erreichbare Zellen. 3 150 geprüfte Seeds sowie eine Nachmessung über 150 Seeds ergaben 100 % Lösbarkeit; ein BFS-Oracle-Agent erreicht das Ziel auf allen Test-Seeds in exakt der BFS-Distanz (Abschnitt 4.4).	erfüllt
Steuerbare Schwierigkeit	Nach der Umstellung von Luftlinie auf BFS-Laufweg liegen alle 150 Eval-Seeds im vorgegebenen Distanzband; zuvor streute eine Vorgabe von „35–45“ real auf 42–75 Tiles (Abschnitt 10.7).	erfüllt
RL-Tauglichkeit	Gymnasium-Schnittstelle, 16 parallele Umgebungen, 88–190 Schritte/s auf einem Notebook; ein vollständiger Curriculum-Lauf über 2,2 Mio. Schritte dauert rund 8 h (Abschnitt 4.1).	erfüllt
Spielbarkeit	Grafischer raylib-Client mit KI-Zuschaumodus; Client und Trainingsumgebung erzeugen aus demselben Seed dieselbe Welt (Abschnitt 4.1).	erfüllt

Tabelle 12: Bewertung der Plattform gegen die Anforderungen aus Abschnitt 1.4.

Was damit belegt ist: und was nicht. Alle fünf Anforderungen sind erfüllt; die Plattform leistet, wofür sie gebaut wurde. Drei der fünf Nachweise entstanden allerdings erst als Korrektur eines zuvor unbemerkten Fehlers: Der Determinismus war durch das prozessglobale Konfigurationsleck verletzt, die Schwierigkeitssteuerung durch das Luftlinienmaß, und die Lösbarkeitsgarantie stand vor der BFS-basierten Exit-Platzierung auf dem schwächeren Fundament eines gecarvten Korridors. Keiner dieser Fehler war am Verhalten des Agenten unmittelbar ablesbar, sie wurden sichtbar, weil das RL-Experiment Zahlen lieferte, die sich

nicht erklären ließen. Das ist der erste von zwei Punkten, an denen die beiden Projektteile ineinandergreifen.

Der zweite folgt in Abschnitt 7.4 und wiegt schwerer: Z1 verlangt lösbare und in ihrer Entfernung steuerbare Welten, nicht aber *hinreichend schwierige*. Genau diese fehlende Anforderung führt dazu, dass die erzeugten Welten die Fähigkeit, um die es in F2 geht, gar nicht einfordern. Die Plattform erfüllt damit ihre Spezifikation vollständig, und die Spezifikation erweist sich als unvollständig.

7.2 Zentrale Gegenüberstellung: Gedächtnis statt lokalem Orakel

Bedingung	Arch.	BFS in Obs	Curric.	Obs-Dim.	SR stoch.	SR det.
A (ppo_phase4)	MLP	ja	ja	236	32 %	0 %
B (ppo_no_bfs)	MLP	nein	nein	230	–	≈ 0 %
C (ppo_lstm_curriculum)	LSTM	nein	ja	231	86 %	36 %

Tabelle 13: Gegenüberstellung dreier Modelle auf Testset A (Exit 35–45), Stand Juni 2026; je ein Einzellauf. Zur eingeschränkten Aussagekraft siehe den folgenden Absatz.

Aussagekraft. Diese Gegenüberstellung ist *keine* kontrollierte Ablation. Vier Einschränkungen begrenzen, was aus ihr folgt. Erstens unterscheiden sich A und C in *zwei* Faktoren gleichzeitig (Architektur *und* BFS-Features), der Effekt ist also nicht separierbar. Zweitens laufen die drei Bedingungen auf unterschiedlichen Observationsräumen und Umgebungsständen, alle vor der v11-Revision; A wurde zudem mit `forceGuaranteedPath` und 236 Merkmalen trainiert. Die Protokollabhängigkeit ist dabei erheblich und betrifft *beide* Betriebsarten: Unter dem damaligen kurzdistanzigen Eval-Protokoll misst sich A als 98-%-Agent (stochastisch) bzw. 2 % deterministisch, unter dem hier verwendeten Protokoll (Exit 35–45) dagegen als 32 % bzw. 0 % (Abbildung 16). Die in dieser Tabelle genannten Werte sind durchgehend die des Protokolls Exit 35–45; die höheren Zahlen im Entwicklungsverlauf (Anhang 10) sind mit ihnen nicht vergleichbar. Drittens ist B kein vollwertiger Vergleichsarm: Der Lauf wurde bei 720 k von 2 Mio. Schritten abgebrochen und enthielt einen Stagnations-Penalty, der später als schädlich erkannt und entfernt wurde. Viertens ist jede Bedingung ein Einzellauf, bei einer Streuung über Läufe von rund ± 12 Punkten (Tabelle 14).

Belastbar ist daher nur die schwache Aussage, dass eine rekurrente Politik *ohne* lokale BFS-Features eine Leistung erreicht, die eine gedächtnislose Politik *mit* diesen Features unter demselben Protokoll nicht erreicht hat.

Das finale Modell (v12, Mittel über sieben Läufe) erreicht auf Holdout B 66,9 % (stochastisch) und erfüllt damit das Holdout-Kriterium; auf Testset A wird die 70 %-Schwelle mit 65,7 % verfehlt (Abschnitt 7.3). Diese Zahl ist *nicht* als Rückschritt gegenüber den 86 % der Bedingung C zu lesen: Vor der v11-Revision wurde die Exit-Entfernung als Luftlinie vorgegeben, eine Vorgabe von „35–45“ entsprach real 42–75 Feldern (Abschnitt 10.7). Bedingung C erzielte ihren Wert also auf deutlich längeren Wegen als v12. Abbildung 12 zeigt den Trainingsverlauf dieses historischen Referenzlaufs; die Verläufe der sieben finalen v12-Läufe zeigt Abbildung 14.

7.3 Endergebnis (v12, $n = 7$)

Die sieben finalen Curriculum-Läufe (alle vier Phasen durchlaufen) ergeben auf dem je Lauf nach Validierungs-SR besten Modell:

Abbildung 13 zeigt dieselben Zahlen grafisch und macht die Streuung sichtbar: Die sieben Einzelläufe liegen auf Testset A zwischen 50 % und 84 %, die Schwelle aus Abschnitt 1.4 liegt mitten in dieser Wolke. Ein einzelner Lauf hätte hier jedes gewünschte Ergebnis geliefert.

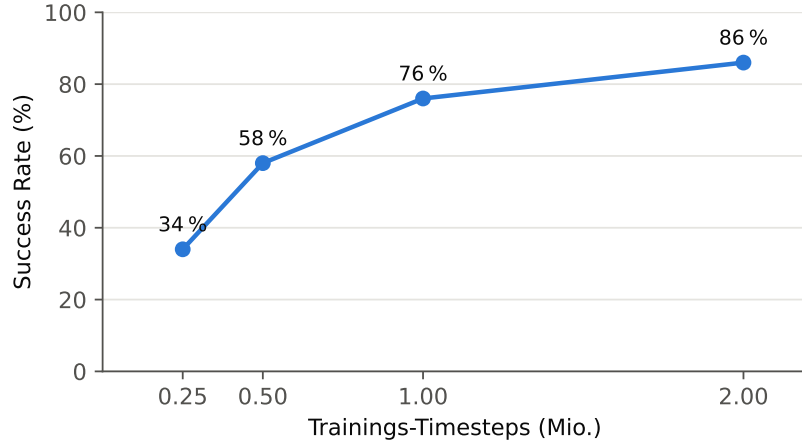


Abbildung 12: Lernkurve des historischen Referenzlaufs (ppo_lstm_curriculum, Bedingung C, Umgebung vor der v11-Revision; Curriculum-Phase 3, stochastische Evaluation auf 50 Seeds).

Lauf	A stoch.	A det.	B stoch.	B det.
Seed 1	62 %	38 %	76 %	46 %
Seed 2	84 %	26 %	76 %	44 %
Seed 3	76 %	32 %	80 %	36 %
Seed 4	74 %	40 %	74 %	38 %
Seed 5	58 %	20 %	62 %	34 %
Seed 6	50 %	20 %	50 %	14 %
Seed 7	56 %	28 %	50 %	16 %
Mittel $\pm$ Std	65,7 $\pm$ 12,4	29,1 $\pm$ 8,0	66,9 $\pm$ 12,8	32,6 $\pm$ 12,7
95 %-CI	[54,2; 77,2]	[21,8; 36,5]	[55,0; 78,7]	[20,8; 44,4]

Tabelle 14: Endergebnis der sieben v12-Läufe auf Testset A und Holdout B (je 50 Seeds, Limit 4000, beide Betriebsarten). Standardabweichung als Stichproben-Std (ddof = 1), Konfidenzintervall über die t -Verteilung (df = 6).

Ausgewertet wird je Lauf nicht der letzte Checkpoint, sondern das nach Validierungserfolgsquote beste Modell, gemessen in der Gating-Metrik der jeweiligen Phase (Tabelle 10), für das Ausgabemodell also deterministisch. Die Auswahl erfolgt ausschließlich auf den disjunkten Seeds 6000–6049. Da über einen Lauf hinweg mehrere Dutzend Checkpoints bewertet werden, enthält der so gewählte Wert einen Selektionsoptimismus gegenüber der Validierung, nicht jedoch gegenüber A und B.

Das Curriculum war faktisch budget- und nicht leistungsgesteuert. Der Phasenwechsel ist leistungsbasiert angelegt (Ziel-SR in *zwei aufeinanderfolgenden* Evaluationen), war es in den Auswertungsläufen aber nur in Ausnahmefällen. Dabei ist zwischen dem einmaligen *Berühren* des Zielwerts und dem *Bestehen* der Phase zu unterscheiden, denn die Eval-Kurven schwanken innerhalb einer Phase um mehrere Dutzend Punkte (Abbildung 14): Nur ein einziger Lauf verließ Phase 1 vorzeitig (Lauf 1, nach 50k statt 500k Schritten); in Phase 2 gelang das drei Läufen. Der Zielwert von 85 % war dabei nicht zu hoch angesetzt, sondern aus einem *nackten* Phase-1-Lauf ohne Curriculum-Stack abgeleitet, der nach der Batch-Korrektur stabil 84–88 % erreicht hatte (Abschnitt 10.6). Dass dieselbe Konfiguration innerhalb des vollen Curriculum-Laufs deutlich stärker schwankt, ist der eigentliche Befund dieser Bilanz: Nicht das Niveau, sondern die Stabilität unterscheidet beide Setups.

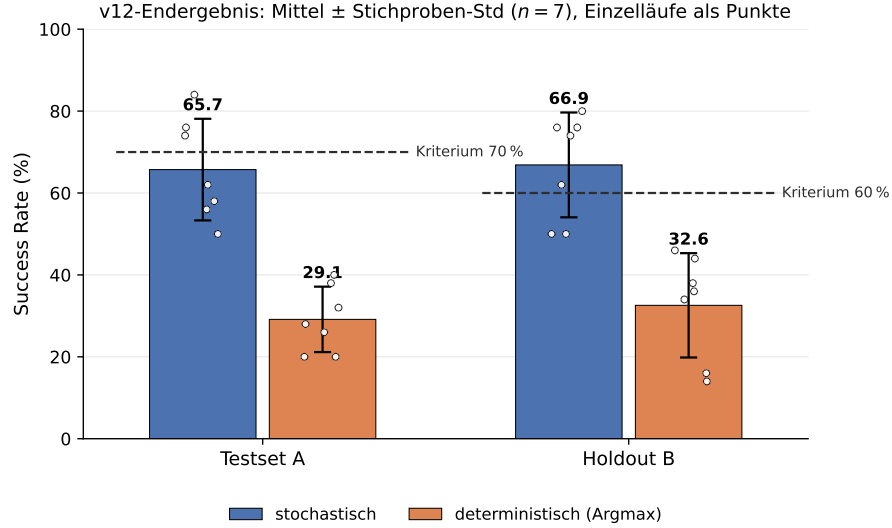


Abbildung 13: Dieselben Daten wie Tabelle 14 als Balken mit Stichproben-Std-Fehlerbalken; die sieben Einzelläufe sind als Punkte überlagert, die gestrichelten Linien markieren die vorab festgelegten Erfolgsschwellen aus Abschnitt 1.4. Erzeugt mit `scripts/plot_v12_endergebnis.py`.

Phase	Ziel-SR	Ziel berührt	Phase bestanden
1	85 % (stoch.)	5 / 7	1 / 7
2	70 % (stoch.)	7 / 7	3 / 7
3	70 % (det.)	0 / 7	0 / 7
4	60 % (det.)	0 / 7	0 / 7

Tabelle 15: Gating-Bilanz der sieben v12-Läufe. *Berührt* = Zielwert in mindestens einer Evaluation erreicht; *bestanden* = in zwei aufeinanderfolgenden Evaluationen erreicht, d. h. die Phase endete vorzeitig statt am Schrittbudget. Quelle: `eval_history.json` je Lauf.

Die Phasen 3 und 4 erreichten ihre Ziele in *keinem* Lauf (Phase 3: 26–54 % gegenüber 70 %; Phase 4: 26–42 % gegenüber 60 %, jeweils deterministisch auf der Validierung) und endeten ausnahmslos mit Erschöpfung ihres Schrittbudgets. Für genau die beiden Phasen, aus denen das berichtete Modell stammt, war das Curriculum damit vollständig budgetgesteuert. Das ist keine Fehlfunktion, sondern die erwartete Folge einer Zielvorgabe oberhalb dessen, was das Verfahren hier liefert; es begrenzt aber die Aussagekraft der Phasenziele als Qualitätsschwelle. Die Einzelwerte je Lauf liegen in `results.json`.

Das Holdout-Kriterium ist erfüllt, das Testset-Kriterium verfehlt: B erreicht $66,9\% \geq 60\%$, A bleibt mit $65,7\%$ um 4,3 Prozentpunkte unter der 70 %-Schwelle. Deterministisch werden beide Kriterien verfehlt ($29,1\% / 32,6\%$); zur Einordnung dieses Gaps siehe Abschnitt 7.5. Abbildung 14 zeigt die zugrunde liegenden Trainingsverläufe aller sieben Läufe: Die hohe Volatilität der Lerndynamik innerhalb der Phasen ist dieselbe Streuungsquelle, die sich in den ± 12 Punkten der Endergebnisse niederschlägt.

Der Weg von $n = 3$ zu $n = 7$ ist selbst ein Ergebnis. Eine Zwischenauswertung über die ersten drei Läufe hatte $73,3 \pm 8,3$ (A) und $80,0 \pm 8,0$ (B) ergeben und beide Kriterien scheinbar erfüllt. (Skriptkonsistent gerechnet, d. h. mit demselben Eval-Skript wie die $n=7$ -Auswertung, lauten dieselben drei Läufe $74,0\%$ bzw. $77,3\%$; der Effekt der Aufstockung ist also $74,0 \rightarrow 65,7$ und nicht ein Artefakt des Skriptwechsels.) Da der Abstand zur Schwelle

v12: Success Rate auf den Validierungs-Seeds, je Curriculum-Phase

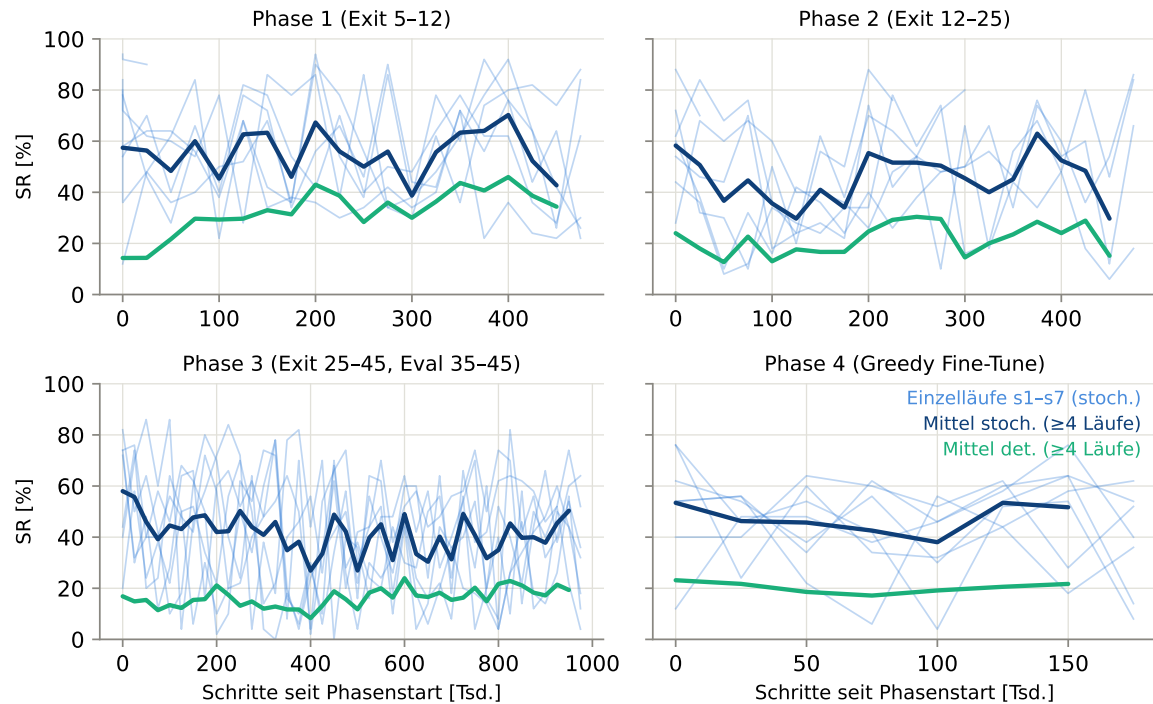


Abbildung 14: Trainingsverläufe der sieben v12-Läufe auf den Validierungs-Seeds 6000–6049, getrennt nach Curriculum-Phase. Dünne Linien: stochastische Erfolgsquote der Einzelläufe; dicke Linien: Ensemble-Mittel stochastisch (dunkelblau) und deterministisch (grün). Da die Schwierigkeit je Phase verschieden ist, sind die Werte nur innerhalb eines Panels vergleichbar. Die x-Achse zählt die Evaluationspunkte im 25k-Raster ab null; das letzte Panel endet deshalb bei 175k, obwohl das Phasenbudget 200k beträgt und ausgeschöpft wurde (Tabelle 15).

(3,3 Punkte) dort kleiner war als die Streuung und das damalige Bootstrap-Intervall auf A ([66,7; 80,0]) die 70 %-Schwelle einschloss, war die Aussage „robust über 70 %“ nicht belegbar. Vier zusätzliche Läufe wurden gezielt zur Klärung dieser Frage gestartet und senkten beide Mittelwerte deutlich. Der Dreier-Mittelwert war zu optimistisch.

Das ist exakt der Effekt, den Henderson et al. [6] für Deep RL beschreiben: Die Varianz über Trainingsläufe ist so groß, dass wenige Seeds systematisch zu Fehlschlüssen führen, weshalb sie Mittelwert $\pm$ Standardabweichung über mehrere Läufe fordern und ausdrücklich vor *best-of-runs*-Berichten warnen. Die vorliegende Arbeit reproduziert diesen Befund am eigenen Gegenstand.

Drei Alternativerklärungen wurden geprüft und ausgeschlossen. Da die vier Zusatzläufe im Mittel schwächer ausfielen (59,5 % gegenüber 74,0 % auf A), wurde geprüft, ob eine systematische Ursache statt Seed-Varianz vorliegt:

1. **Messfehler:** ausgeschlossen. Das für $n = 7$ neu implementierte Evaluationsskript reproduziert alle sechs deterministischen Werte der Läufe 1–3 exakt (A: 38/26/32; B: 46/44/36); die stochastischen Werte weichen um 2–8 Punkte ab, was dem erwarteten Sampling-Rauschen entspricht.
2. **Code-Stand:** ausgeschlossen. Die Läufe 1–3 und 4–7 entstanden auf unterschiedlichen Commits. Ein Diff über den gesamten Simulationspfad (`src/core`, `src/python`,

`src/include, python/, game_config.json, train_curriculum.py`) ergibt genau eine geänderte Datei: `doc_logger.py`, und zwar jene 22 Zeilen, die den Commit-Hash in die `config.json` schreiben. Simulation, Environment und Trainingsskript sind identisch; die übrigen Änderungen betreffen ausschließlich Client-Dateien, die nicht in `stoneforge_sim.so` einfließen.

3. **Laufzeitumgebung:** ausgeschlossen. Die installierten Versionen von `torch` und `sb3-contrib` sind älter als beide Lauf-Chargen; die Versions-Pinnung erfolgte deklarativ ohne Neuinstallation.
4. **Energie-Drosselung:** ausgeschlossen. Dieser Punkt verdient besondere Aufmerksamkeit, weil die Ursache exakt mit der Chargengrenze zusammenfällt: Genau die Läufe 4–7 liefen bei zugeklapptem Laptop und gedrosselter CPU und brauchten 29–36 h Wall-Clock statt der sonst üblichen rund 8 h (Anhang 9). Ergebnisrelevanz hat das dennoch nicht, weil weder das Training noch das Curriculum eine Zeitgröße kennt: Optimiert wird pro Umgebungsschritt, die Evaluationen liegen auf einem festen Schrittraster (alle 25 k), und die Phasen enden an einem Schrittbudget (Tabelle 15). Eine langsamere CPU streckt damit die Wall-Clock-Zeit, verändert aber weder die Zahl noch die Reihenfolge der Gradientenschritte. Der bit-identische Vergleich zweier Trainingspfade in Abschnitt 10.6 bestätigt diese Zeitunabhängigkeit unabhängig davon empirisch.

Ein Welch-Test zwischen beiden Chargen (74,0 vs. 59,5) ist mit $p = 0,149$ nicht signifikant; bei $n = 3$ bzw. $n = 4$ ist die Stichprobe für eine solche Aussage zu klein. Es handelt sich um Seed-Varianz; die $n = 7$ -Werte sind belastbar.

Weitere Beobachtungen. Der bei $n = 3$ auffällige Abstand $B > A$ verschwindet bei $n = 7$ nahezu (66,9 vs. 65,7); beide Sets stammen aus demselben Generator und derselben Distanzverteilung, die frühere Differenz war eine Eigenschaft der kleinen Stichprobe. Ein Gegentest mit den Phase-2-Zwischenmodellen ergab keine Verbesserung; das finale Modell bleibt das Ausgabemodell.

7.4 Ungelernte Referenzpolitiken und die Grenzen der Erfolgsquote

Eine Erfolgsquote ist ohne Untergrenze nicht interpretierbar: Ohne Bezugspunkt bleibt offen, ob 65 % eine gelernte Fähigkeit ausdrücken oder bereits durch Zufall erreichbar sind.

Verglichen werden zwei ungelernte Politiken, die *dieselbe* Beobachtung erhalten wie der Agent:

- **Random:** gleichverteilte Bewegung; absolute Untergrenze.
- **Kompass-Zufallslauf:** mit Wahrscheinlichkeit ε gleichverteilt zufällig, sonst ein Schritt in die dominante Richtung des Luftlinien-Kompasses ($exitDx/exitDy$). Diese Politik nutzt *zwei* der 229 Merkmale; das lokale 15×15 -Grid wird vollständig ignoriert. Sie hat kein Gedächtnis, keine Parameter und wurde nicht trainiert. Der einzige freie Wert ε wurde ausschließlich auf den Validierungs-Seeds 6000–6049 gewählt, also nach demselben Protokoll wie die Modellselektion; Testset A und Holdout B wurden dafür nicht verwendet.

Als zweite Metrik dient die **Pfadeffizienz** $\eta = d_{\text{BFS}}(s_0) / T_{\text{Erfolg}}$, gemittelt über die erfolgreichen Episoden: das Verhältnis aus kürzestem Weg und tatsächlich benötigten Schritten. $\eta = 1$ bedeutet optimale Navigation, $\eta = 0,05$ einen zwanzigfachen Umweg. Gemittelt wird über die *Quotienten* der Einzelepisoden, nicht der Quotient der Mittelwerte gebildet. Beide Größen fallen hier deutlich auseinander, weil die Schrittzahl stark rechtsschief verteilt ist:

Dieselben Läufe ergeben $\eta \approx 0,05$ als Mittel der Quotienten, aber nur 0,03 als Quotient der Mittel ($\varnothing 40$ Schritte Optimum zu $\varnothing 1\,329$ tatsächlichen, Tabelle 18). Das ist kein Widerspruch, sondern die Jensen-Ungleichung; berichtet wird durchgehend die erste Variante. Da nur erfolgreiche Episoden eingehen, wird jede Politik auf *ihrer eigenen* Teilmenge gelöster Welten bewertet: Der Agent löst 65,7 %, der Kompass-Zufallslauf mit $\varepsilon = 0,9$ dagegen 92 %. Der Agent wird also auf der leichteren, die Referenzpolitik auf der vollständigeren Teilmenge gemessen. Die Verzerrung wirkt damit zugunsten des Agenten und schwächt den nachfolgenden Befund nicht ab, sondern verstärkt ihn. Die Metrik ist erforderlich, weil die Erfolgsquote bei einem Episodenlimit von 4 000 Schritten und einer mittleren Optimallänge von 40 Schritten (Faktor 100) kaum diskriminiert. Sie greift zugleich die zweite, im Exposé neben der Erfolgsquote genannte Zielgröße auf (Minimierung der *Steps per Episode*), die dort nicht quantifiziert worden war. Tabelle 16 stellt beide Metriken für alle Politiken gegenüber.

Politik	Testset A		Holdout B	
	SR	η	SR	η
Random (gleichverteilt)	5,2 $\pm$ 2,7	0,021	10,4 $\pm$ 1,7	0,025
Kompass-Zufallslauf, $\varepsilon = 0,4$	60,4 $\pm$ 4,8	0,153	61,6 $\pm$ 5,0	0,161
Kompass-Zufallslauf, $\varepsilon = 0,9$	92,0 $\pm$ 5,1	0,047	91,6 $\pm$ 4,3	0,048
RecurrentPPO v12 ($n = 7$, stoch.)	65,7 $\pm$ 12,4	0,049	66,9 $\pm$ 12,8	0,057

Tabelle 16: Ungelernte Referenzpolitiken gegenüber dem trainierten Agenten; Protokoll identisch (je 50 Seeds, Exit 35–45, Limit 4 000). η bezeichnet die Pfadeffizienz, die beiden ε -Werte die auf der Validierung erfolgs- bzw. effizienzoptimale Einstellung. Alle Baseline-Werte sind Mittel über fünf Wiederholungen mit unterschiedlichem Politik-Zufallsstrom (RNG-Seeds 100–104) bei festen Weltseeds. Erzeugt mit `scripts/eval_baselines.py` mit `--repeats 5 --rng-offset 100`. Wie stark eine Zeile auf die Wahl des Politik-Zufallsstroms reagiert, zeigt der Wechsel auf die RNG-Seeds 0–4: Die Random-Zeile springt auf A von 5,2 auf 10,4 $\pm$ 4,6, verdoppelt sich also; die Kompass-Zeile bewegt sich bei $\varepsilon = 0,9$ nur von 92,0 auf 89,6. Der Kernbefund hängt damit nicht an der Wahl des Zufallsstroms, die absolute Höhe der Zufallsuntergrenze schon. **Zur Lesart der Streuungen:** Sie sind nicht unmittelbar vergleichbar. Bei den Referenzpolitiken bezeichnet $\pm$ die Streuung über fünf Politik-Zufallsströme bei festen Weltseeds und enthält damit keine Trainingsvarianz; beim Agenten die Streuung über sieben unabhängige Trainingsläufe. Der Abstand zwischen beiden Zeilen ist so groß, dass er von dieser Unterscheidung nicht berührt wird, die Fehlerbalken messen aber verschiedene Größen.

Befund. Der Kompass-Zufallslauf übertrifft den trainierten Agenten auf *beiden* Achsen. Bei vergleichbarer Pfadeffizienz ($\eta \approx 0,05$) erreicht er 92 % statt 65,7 %; bei vergleichbarer Erfolgsquote (≈ 60 %) ist er mit $\eta = 0,15$ rund dreimal wegeffizienter. Der Agent liegt zwar deutlich über der Zufallsuntergrenze von 5–10 %, erreicht aber an keinem Betriebspunkt die Leistung einer Heuristik aus vier Zeilen Code, die 227 seiner 229 Eingangsmerkmale ignoriert.

Kein Episodenbudget kehrt das Bild um. Naheliegend wäre der Einwand, das Problem liege allein am großzügigen Limit von 4 000 Schritten. Das lässt sich exakt prüfen: Wer eine Welt bei Schritt T löst, löst sie unter jedem Budget $\geq T$; aus einem einzigen Lauf mit protokollierten Erfolgsschritten folgt daher die vollständige Kurve *Erfolgsquote über Budget* (Tabelle 17). Das Ergebnis ist eindeutig: Bei *jedem* Budget erreicht oder übertrifft mindestens eine der beiden ungelernten Politiken alle drei trainierten Läufe. Ein engeres Episodenlimit ist als Korrektur damit nicht ausreichend.

Budget	Kompass-Zufallslauf		RecurrentPPO v12		
	$\varepsilon = 0,4$	$\varepsilon = 0,9$	Lauf 1	Lauf 2	Lauf 3
120 ($3 \times$ Optimum)	4 %	0 %	0 %	2 %	0 %
200	22 %	0 %	0 %	4 %	0 %
300	28 %	0 %	2 %	10 %	6 %
500	36 %	10 %	12 %	22 %	12 %
800	54 %	18 %	20 %	32 %	30 %
1 500	62 %	54 %	40 %	48 %	54 %
4 000	64 %	88 %	64 %	78 %	74 %

Tabelle 17: Erfolgsquote auf Testset A in Abhängigkeit des Episodenbudgets (BFS-Optimum $\varnothing$ 40 Schritte). Hervorgehoben ist je Zeile der höchste Wert. Die Kurve stammt aus je einem Lauf mit protokollierten Erfolgsschritten; da die Early-Stop-Truncation dabei aktiv war (Abschnitt 6.3), misst die Tabelle den Budget-Effekt *unter* dieser Bedingung und nicht in Reinform.

Was daraus folgt und was nicht. Der Befund entwertet weder die Plattform noch die methodische Arbeit dieses Projekts; er entwertet die *Aufgabenstellung* in ihrer aktuellen Parametrisierung. Drei Eigenschaften wirken zusammen: (1) Das Episodenlimit übersteigt die Optimallänge um den Faktor 100, sodass ungerichtete Suche genug Zeit hat; (2) der Luftlinien-Kompass liefert dauerhaft eine grob richtige Richtung, sodass ein verrauschter Gradientenaufstieg genügt; (3) die erzeugten Welten sind hinreichend offen, dass Sackgassen durch Zufallsbewegung wieder verlassen werden. Unter diesen Bedingungen ist Gedächtnis für die Erfolgsquote schlicht nicht erforderlich, und die Metrik kann folglich nicht messen, ob es gelernt wurde.

Das Reward-Design verstärkt diesen Effekt. Zu den drei Eigenschaften der Aufgabe kommt eine vierte des eigenen Belohnungsentwurfs. Tabelle 18 bilanziert die dichten Reward-Terme über eine typische erfolgreiche Episode, gemessen an den Läufen 1–3 auf Testset A.

Term	Summe je Episode	Anteil (Betrag)
Explorations-Bonus ($+0,02 \times 203$ neue Tiles)	+4,1	16 %
Schritt-Malus ($-0,01 \times 1\,329$ Schritte)	−13,3	53 %
Revisit-Penalty	−6,2	25 %
PBRs gesamt (gemessen)	+1,3	5 %
davon teleskopierter Anteil $\beta d_0/128$	+0,8	
davon Restterm $\beta(1 - \gamma) \sum_t d_t/128$	+0,5	
Summe der dichten Terme	−14,1	
Terminal-Reward (sparse)	+100	

Tabelle 18: Bilanz der dichten Reward-Terme über eine mittlere erfolgreiche Episode (Mittel über die Läufe 1–3, Testset A, stochastisch, $\varnothing$ 1 329 Schritte bei $d_0 \approx 40$). Die PBRs-Summe ist Schritt für Schritt aus der tatsächlichen BFS-Distanz aufsummiert, nicht aus der Teleskopierungsformel abgeleitet; zur Aufspaltung siehe den folgenden Absatz. Erzeugt mit `scripts/reward_bilanz.py` (fester Politik-RNG).

Entscheidend ist die Asymmetrie in der letzten Spalte. Das einzige dichte Signal, das *Richtungen unterscheidet*, ist der PBRs-Term, und er trägt rund 5 % der dichten Reward-Masse. Die übrigen 95 % hängen ausschließlich davon ab, *wie lange* der Agent unterwegs ist und *wie viel Fläche* er dabei abdeckt. Über die Episode hinweg sieht er damit vor allem ein Signal über Weglänge und Flächendeckung, kaum eines über die Richtung zum Ziel; allein der

Explorations-Bonus wiegt rund dreimal so schwer wie die gesamte Zielinformation.

Warum die PBRs-Summe nicht wegunabhängig ist. Die Teleskopierung legt die Summe nur für $\gamma = 1$ fest. Mit $F(s, s') = \beta(\gamma\Phi(s') - \Phi(s))$ und $\Phi = -d/128$ ergibt die Summe über eine Episode, die im Ziel endet ($\Phi(s_T) = 0$):

$$\sum_t F_t = \frac{\beta}{128} \left[\underbrace{d_0}_{\text{teleskopiert}} + \underbrace{(1-\gamma)\sum_t d_t}_{\text{Restterm}} \right]. \quad (8)$$

Der Restterm verschwindet nur bei $\gamma = 1$. Bei $\gamma = 0,999$, einer mittleren Episodenlänge von 1329 Schritten und einer mittleren Distanz von $\bar{d} \approx 20$ während der Episode beträgt er gemessen +0,5 und macht damit rund 40 % der gesamten PBRs-Summe aus. Ein einfaches Einsetzen der Teleskopierungsformel unterschätzt den Beitrag also um etwa den Faktor 1,7; Tabelle 18 weist deshalb den gemessenen Wert aus. Der Restterm wächst mit Episodenlänge und Abstand zum Ziel, das Shaping zahlt also rechnerisch einen kleinen Bonus fürs Fernbleiben ($\approx +0,0004$ pro Schritt bei $\bar{d} = 20$, gegenüber $-0,01$ Schritt-Malus).

Das ist allerdings *kein* Widerspruch zur Politikinvarianz. Die Garantie von Ng et al. [23] bezieht sich auf das *diskontierte* Optimierungsziel bei identischem γ , und genau diese Bedingung ist hier erfüllt ($\gamma_{\text{Shaping}} = \gamma_{\text{RL}} = 0,999$). Der Restterm tritt nur in der *undiskontierten* Episodensumme auf, die kein Optimierungsziel ist; die optimale Politik bleibt unverändert. Für die hier geführte Bilanzrechnung ist er dennoch zu berücksichtigen, weil sie Größenordnungen und nicht Optima vergleicht.

Zwei Einschränkungen. Erstens bleibt der Terminal-Reward das dominierende Zielsignal. Der Vergleich ist dabei auf beiden Seiten diskontiert zu führen, weil die Werte in Tabelle 18 undiskontiert angegeben sind (dort werden sie nur untereinander verglichen). Beide Größen sind je Episode gemessen und anschließend gemittelt, nicht durch einen auf die Summen angewandten Skalar gebildet:

- **Terminal-Reward:** $100 \cdot \mathbb{E}[\gamma^T] = +37,7$. Maßgeblich ist der Erwartungswert von γ^T über die Episoden, nicht γ^T an der mittleren Länge: Weil γ^T konvex in T ist, liegt letzteres mit +26,6 systematisch zu niedrig (dieselbe Jensen-Ungleichung wie bei η oben).
- **Dichte Terme:** $-5,3$, aufsummiert als $\sum_t \gamma^t r_t$ über den tatsächlichen Episodenverlauf. Der daraus resultierende mittlere Diskontfaktor von 0,37 ist ein Messergebnis; er liegt unter dem Wert, den eine gleichmäßige Verteilung der Terme über die Episode ergäbe (0,55), weil der Revisit-Penalty erst spät in der Episode anfällt und damit stärker diskontiert wird.

Der Terminal-Reward überwiegt die dichte Bilanz damit um rund das Siebenfache. Die Aufgabe ist also nicht falsch gestellt, sondern nur schwach dicht geführt. Zweitens ist Herumlaufen entgegen einer naheliegenden Vermutung *nicht* kostenlos: Schritt- und Revisit-Penalty übersteigen den Explorations-Bonus deutlich, die dichte Bilanz ist mit $-14,1$ klar negativ. Die Schwäche liegt also nicht darin, dass Umherirren belohnt würde, sondern darin, dass der Agent aus dem dichten Signal kaum lernen kann, *wohin* er gehen soll. Ein Ansatzpunkt für Folgearbeiten ist entsprechend nicht das Streichen des Explorations-Bonus, sondern die Verstärkung oder dichtere Führung der Zielinformation (Abschnitt 8).

Damit ist die zentrale Leistungsaussage dieser Arbeit einzuschränken: Belegt ist, dass der Agent die Aufgabe löst und dabei deutlich über der Zufallsuntergrenze liegt; *nicht* belegt ist, dass er eine Strategie gelernt hat, die über gerichtete Zufallssuche hinausgeht. Diese Einschränkung betrifft auch die Gegenüberstellung in Abschnitt 7.2: Wenn eine kompassgetriebene Zufallspolitik 92 % erreicht, ist der Vergleich „LSTM ohne BFS“ gegen „MLP mit

BFS“ kein Test der Gedächtnisfähigkeit, sondern misst überwiegend, wie gut die jeweilige Politik den Kompass ausnutzt und wie stark sie in Schleifen gerät.

Konsequenz für Folgearbeiten. Die naheliegende Reaktion ist nicht, den Agenten weiter zu tunen, sondern die Aufgabe selbst zu korrigieren. Da Tabelle 17 zeigt, dass eine Änderung des Episodenbudgets allein nicht genügt, führt der Weg über die Weltgenerierung: Der Wandanteil muss so weit erhöht werden, dass ungerichtete Suche scheitert, ohne die Lösbarkeit zu gefährden. Die Baseline-Politik dient dabei als Kalibrierinstrument, mit dem sich ein geeigneter Schwierigkeitsgrad *vor* jedem weiteren Training bestimmen lässt. Das Vorgehen ist in Abschnitt 8 ausgeführt. Erst unter einem solchen Regime lässt sich die Forschungsfrage dieser Arbeit beantworten.

Eine unabhängige Wiederholung der Modellmessung mit anderem Zufallsstrom ergab 68,6 % auf Testset A statt der in Tabelle 14 berichteten 65,7 %. Die Abweichung von knapp drei Punkten entspricht dem erwarteten Sampling-Rauschen und ändert die Aussage nicht.

7.5 Der Det/Stoch-Gap: Weglängen-Analyse und POMDP-Einordnung

Der Abstand zwischen stochastischer und deterministischer SR ist kein Schwelleneffekt, sondern wächst kontinuierlich mit der Weglänge (Abbildung 15; 120 Welten aus dem Validierungsbereich, Seeds 6000–6119, nach Startdistanz gruppiert): Bei 5–15 Feldern erreicht die deterministische Betriebsart 79 %, bei 35–45 Feldern noch 31 %, während die stochastische zwischen 100 % und 82 % bleibt. Die Interpretation ist kumulativ: Je länger der Weg ohne direkte Zielsicht (Sichtradius 7), desto mehr Kreuzungsentscheidungen unter unsicherem Belief-State; an jeder liegen die Aktionswahrscheinlichkeiten nahe beieinander, und jede einzelne kann die Argmax-Politik in eine Oszillationsschleife führen, aus der Sampling entkommt. Der Gap ist damit teils eine Gedächtnis-Limitation, teils dem POMDP-Charakter geschuldet (unter Unsicherheit ist eine stochastische Politik nicht per se suboptimal, Abschnitt 2).

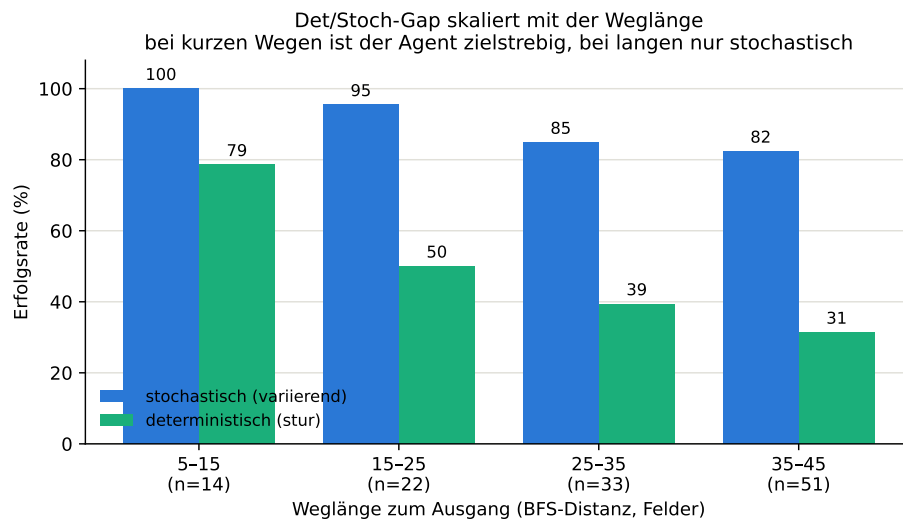


Abbildung 15: Erfolgsquote nach Startdistanz zum Exit (120 Welten des Validierungsbereichs, Seeds 6000–6119, finales v12-Modell, Limit 4000): Die deterministische SR fällt kontinuierlich mit der Weglänge, die stochastische bleibt hoch. Erzeugt mit `scripts/plot_det_gap_distanz.py`.

Zur theoretischen Einordnung ist eine Unterscheidung nötig, die in der Zwischendokumentation dieses Projekts zunächst unscharf blieb. Singh et al. zeigen, dass in POMDPs

stochastische gedächtnislose Politiken jede deterministische gedächtnislose erheblich überreffen können [15]. Dieses Resultat erklärt den hier beobachteten Gap jedoch *nicht*, denn die verwendete Politik ist nicht gedächtnislos: Über einem hinreichend genauen Belief-State existiert wieder eine optimale deterministische Politik. Dass die Argmax-Auswertung des LSTM-Agenten dennoch deutlich abfällt, ist daher primär ein Hinweis auf einen *unvollständigen* Belief-State, also eine Gedächtnis-Limitation, und nur mittelbar eine Eigenschaft der Aufgabe. Singh et al. liefern die Motivation dafür, warum stochastisches Verhalten unter partieller Beobachtbarkeit überhaupt nützlich sein kann, nicht aber eine Rechtfertigung des konkreten Gaps.

Der Teil der Begründung, der auch *mit* Gedächtnis trägt, stammt aus der Generalisierungs-Anforderung: Ghosh et al. zeigen, dass bereits das Ziel, auf unbekannte Level (hier: neue Seeds) zu generalisieren, selbst eine Form partieller Beobachtbarkeit induziert (*epistemic POMDP*), deren optimale Politik stochastisch ist und die durch mehr Gedächtnis über die *aktuelle* Episode nicht aufgelöst wird [22]. Die Unsicherheit liegt hier über der Identität der Welt, nicht über dem Zustand innerhalb einer bekannten Welt. Auf dieses Argument, nicht auf Singh et al., stützt sich die Wahl der stochastischen Evaluation als Primärmetrik. Der Gap ist damit teils erwartbar (epistemische Unsicherheit über Seeds), teils ein offenes Defizit des gelernten Belief-States; die Trennung beider Anteile ist mit den vorliegenden Daten nicht möglich und wird als Diagnoseschritt 1 im Ausblick (Abschnitt 8) vorgeschlagen.

Wichtig ist, die beiden Ebenen nicht gegeneinander auszuspielen. Die *unmittelbare*, mechanistische Ursache des Gaps ist banal: In nahezu gleichwahrscheinlichen Kreuzungsentscheidungen fährt sich die deterministische Argmax-Politik in eine Oszillation zwischen wenigen Feldern fest, aus der das Sampling durch Zufall wieder entkommt. Der Verweis auf den epistemic POMDP ersetzt diese Erklärung nicht, sondern beantwortet eine andere Frage: Er begrenzt, wie weit sich der Gap *überhaupt* schließen ließe, denn er zeigt, dass stochastisches Verhalten hier selbst bei *perfektem* Gedächtnis über die laufende Episode optimal bliebe [22]. Die Theorie liefert also die obere Grenze des Erreichbaren, die Oszillationsmechanik die konkrete Ursache im vorliegenden Lauf.

7.6 Versuche zur Schließung des Det/Stoch-Gaps

Um zu prüfen, ob der Gap durch gezielte Eingriffe verkleinerbar ist, wurden drei Varianten gerechnet, die je einen inhaltlichen Faktor gegenüber der v12-Basiskonfiguration (Tabelle 11) verändern: **E1** verstärkt den Critic (`vf_coef` 0,5 $\rightarrow$ 1,0), **E2** glättet den Curriculum-Übergang (Phase 3 nur bis Distanz 35 statt 45), **E3** vergrößert das Gedächtnis (LSTM 256 $\rightarrow$ 512). Tabelle 19 stellt Setup und Ergebnis je Arm gegenüber.

Variante	Änderung ggü. Baseline	Start	Test A (stoch/det)	Holdout B (stoch/det)
v12 (Baseline, $n=3$)	– (LSTM 256, <code>vf_coef</code> 0.5)	Phase 1	73/32 %	80/42 %
E1 critic	<code>vf_coef</code> 0.5 $\rightarrow$ 1.0	Phase 3	56/42 %	72/36 %
E2 curric	Phase 3 Exit 25–45 $\rightarrow$ 25–35	Phase 3	26/16 %	42/24 %
E3 lstm512	LSTM 256 $\rightarrow$ 512	Phase 1	44/14 %	58/20 %

Tabelle 19: Varianten zur Schließung des Det/Stoch-Gaps; jeder Arm ein Einzellauf, die Baseline ein Mittel aus drei Läufen. Werte standardisiert (50 Seeds, Limit 4 000). Die Baseline-Zeile stammt aus derselben Messreihe wie die drei Arme und damit aus dem seinerzeit verwendeten Eval-Skript; skriptkonsistent zur $n=7$ -Auswertung gerechnet lauten dieselben drei Läufe 74,0 (A) und 77,3 (B) statt 73 und 80 (Abschnitt 7.3). Der Vergleich *innerhalb* der Tabelle ist davon nicht betroffen, ein Vergleich mit Tabelle 14 dagegen nur bis auf diese wenigen Punkte belastbar. Zur Aussagekraft siehe den folgenden Absatz.

Aussagekraft. Diese drei Läufe sind Einzelmessungen ($n = 1$) gegen eine Baseline aus drei Läufen. Die in Abschnitt 7.3 gemessene Streuung über Trainingsläufe beträgt $\sigma \approx 12$ Prozentpunkte; ein Einzellauf kann also allein durch Seed-Varianz um mehr als 20 Punkte von der Baseline abweichen. Hinzu kommt, dass die Arme nicht vollständig kontrolliert sind: E1 und E2 setzen aus einem gemeinsamen Phase-2-Checkpoint auf und erhalten nur 400k statt 1,0 Mio. Phase-3-Schritte, variieren gegenüber der Baseline also zwei Größen und nicht eine; E3 ist dagegen ein voller Lauf ab Phase 1 mit identischem Budget. Die Arme sind daher *kein* Wirksamkeitsnachweis, sondern eine Vorauswahl: Sie können eine Hypothese schwächen, aber nicht widerlegen. Konsequenterweise wird hier nur die schwache Form berichtet. Damit unterschreiten diese drei Läufe bewusst das in Abschnitt 1.4 vorab festgelegte Kriterium von mindestens drei Trainingsläufen je Konfiguration. Das ist eine Ressourcenentscheidung, kein Versehen: Die verfügbare Rechenzeit floss in die Aufstockung des Hauptergebnisses von $n = 3$ auf $n = 7$, weil dort das Erfolgskriterium der Arbeit entschieden wird. Die Konsequenz daraus ist, dass aus E1–E3 keine Wirksamkeitsaussage abgeleitet wird; ihre Replikation steht entsprechend im Ausblick (Abschnitt 8).

Kein Arm zeigt einen Hinweis auf Verbesserung. E1 hebt die deterministische SR (42 % statt 32 % auf A), liegt stochastisch aber niedriger (56 % statt 73 %). Beide Differenzen bleiben unter $1,5\sigma$: Der deterministische Abstand von 10 Punkten entspricht bei $\sigma_{\text{det}} = 8,0$ (Tabelle 14) rund $1,25\sigma$, der stochastische von 17 Punkten bei $\sigma_{\text{stoch}} = 12,4$ rund $1,37\sigma$. Da hier ein Einzellauf gegen ein Dreier-Mittel steht, ist genau genommen $\sigma_{\text{diff}} = \sigma\sqrt{1 + 1/3} \approx 1,15\sigma$ das richtige Maß, womit beide Abstände auf $1,08$ bzw. $1,19\sigma$ schrumpfen. Sie sind bei $n = 1$ folglich nicht belastbar; die Richtung (mehr Value-Gewicht $\rightarrow$ determinierteres Verhalten) ist plausibel, aber unbestätigt. E2 liegt mit A 26 % rund 4σ unter der Baseline; das ist auch bei $n = 1$ deutlich und mechanistisch erklärbar, weil das Modell auf den längsten, eval-relevanten Distanzen untertrainiert bleibt; der Rückstand von E1 und E2 ist wegen des reduzierten Phase-3-Budgets allerdings nur teilweise ein Faktoreffekt. E3 liegt mit A 44 % rund $2,5\sigma$ unter der Baseline, ist ein voller Lauf mit identischem Budget und zeigt in *allen* Curriculum-Phasen Werte auf oder unter der 256er-Baseline bei rund dreifachem Rechenaufwand. Das ist der belastbarste der drei Befunde: Mehr Gedächtnis-*Kapazität* bedeutet in dieser Umgebung jedenfalls nicht mehr Gedächtnis-*Nutzen*. Er passt zu den bekannten Grenzen rekurrenter Politiken in prozeduraler Navigation, deren Gedächtnisanforderungen über reine Kapazität hinausgehen [30]. Eine Replikation von E1 und E3 mit je zwei weiteren Seeds steht aus und ist der nächste offene Punkt (Abschnitt 8).

Ergänzend wurde geprüft, ob der Gap lediglich ein Kalibrierungsartefakt der Argmax-Auswertung ist. Ein Temperatur-Sweep zwischen reinem Argmax ($T=0$) und vollem Sampling ($T=1$) auf den drei v12-Modellen (Tabelle 20) zeigt einen *monotonen* Verlauf: Mehr Stochastik ist durchweg besser, keine Zwischentemperatur übertrifft das volle Sampling. Der Gap ist damit *kein* Argmax-Artefakt, sondern hat eine der in Abschnitt 7.5 diskutierten Ursachen: epistemische Unsicherheit über den Seed [22] und einen unvollständigen Belief-State. Bereits $T=0,5$ holt den Großteil des Argmax-Defizits zurück (A: +31, B: +29 Prozentpunkte); dies ist ein geeigneter Betriebspunkt, falls ein weniger zufälliges Verhalten gewünscht ist.

Ein Hinweis zur Konsistenz: Die Zeile $T=1$ (70,7 % auf A) misst dieselbe Größe wie die oben genannten 73,3 %, die in Abschnitt 7.3 nachgerechneten 74,0 % und die 72,0 %, die sich aus der Zeile „Budget 4000“ von Tabelle 17 ergeben (64/78/74). Diese vier Werte stammen aus unabhängigen Messungen mit vier Skripten; auch auf Ebene der Einzelläufe weichen sie voneinander ab (62/84/76 in Tabelle 14 gegenüber 64/78/74 in Tabelle 17). Die Streuung von rund drei Punkten im Mittel entspricht dem Sampling-Rauschen über 50 Bernoulli-Versuche. Das ist kein Widerspruch, zeigt aber, dass Unterschiede dieser Größenordnung nicht interpretierbar sind. Vergleiche *innerhalb* einer Tabelle bleiben zulässig, da sie aus demselben Lauf und Skript stammen.

Zusammengenommen widersteht der Det/Stoch-Gap dem Tuning einzelner Stellschrau-

Temperatur	Testset A	Holdout B
Argmax ($T=0$)	$32,0 \pm 6,0$	$42,0 \pm 5,3$
$T = 0,25$	$52,0 \pm 0,0$	$52,7 \pm 2,3$
$T = 0,5$	$63,3 \pm 5,0$	$71,3 \pm 8,1$
$T = 0,75$	$63,3 \pm 9,2$	$72,0 \pm 8,0$
Sampling ($T=1$)	$70,7 \pm 10,1$	$79,3 \pm 8,1$

Tabelle 20: Erfolgsquote in Abhängigkeit der Sampling-Temperatur (Mittel $\pm$ Std über die drei v12-Läufe, je 50 Seeds, Limit 4000). Streuung als Stichproben-Standardabweichung (ddof = 1), einheitlich zu Tabelle 14; die Zeile $T=0$ entspricht den Spalten „det“ der Läufe 1–3 dort. Bei $n = 3$ ist die Streuung entsprechend grob geschätzt und trägt für sich genommen keine Aussage; belastbar ist allein die Monotonie der Mittelwerte.

ben; er ist teils fundamental (POMDP). Nicht-architektonische Hebel wie Hilfs- bzw. Repräsentationsverluste oder gezielt mehr Langdistanz-Training bleiben ergänzend denkbar; das *harte*, fundamentale Langhorizont-Belief-Tracking ist jedoch am ehesten über strukturiertes statt größeres Gedächtnis anzugehen (Abschnitt 8).

8 Fazit und Ausblick

8.1 Ergebnis

Die Arbeit hatte zwei Fragen gestellt. Sie fallen unterschiedlich aus, und gerade ihr Verhältnis zueinander ist das eigentliche Ergebnis.

F1 (Weltgenerator): beantwortet. Der entwickelte Generator erzeugt aus einem Seed reproduzierbare, chunkweise und damit unbegrenzt erweiterbare Welten. Alle fünf Anforderungen aus Z1 sind nachgewiesen erfüllt (Tabelle 12): Determinismus bis hin zu bit-identischen Trainingspfaden, 100 % Lösbarkeit über 3 150 geprüfte Seeds, eine Zielentfernung, die nach der Umstellung auf das BFS-Laufwegmaß exakt eingehalten wird, ausreichender Durchsatz für Millionen Schritte und ein spielbarer Client auf derselben Simulation. Die Kombination aus Spielbarkeit und Kontrollierbarkeit ist dabei der Punkt, an dem sich der Aufwand eines eigenen Generators gegenüber einem fertigen Benchmark auszahlt (Kapitel 3).

F2 (Navigation ohne Orakel): nicht entscheidbar. Über sieben unabhängige Läufe erreicht der Agent $65,7\% \pm 12,4$ (Testset A) und $66,9\% \pm 12,8$ (Holdout B) stochastische Erfolgsquote. Das Holdout-Kriterium ($\geq 60\%$) ist damit erfüllt, das Testset-Kriterium ($\geq 70\%$) verfehlt. Diese Zahlen sind jedoch nur gegen die Zufallsuntergrenze von 5–10 % aussagekräftig, nicht gegen eine kompetente ungelernete Referenz: Ein Kompass-Zufallslauf erreicht 92 % und ist bei vergleichbarer Erfolgsquote zugleich dreimal wegeffizienter (Abschnitt 7.4). Bei einem Episodenbudget vom Hundertfachen der Optimallänge misst die gewählte Metrik nicht die Fähigkeit, um die es geht, und ein engeres Budget behebt das nicht (Tabelle 17). **F2 ist mit dem vorliegenden Aufbau daher nicht entscheidbar: Die Umgebung erfordert das Gedächtnis nicht, dessen Nutzen sie messen soll.** Dieser Befund betrifft die Interpretation sämtlicher Leistungszahlen der Arbeit, einschließlich der Gegenüberstellung in Abschnitt 7.2.

Das Verhältnis beider Antworten. Auf den ersten Blick steht ein gelungener Generator neben einem unentschiedenen Lernexperiment. Tatsächlich haben beide dieselbe Ursache. Der Generator erfüllt seine Spezifikation vollständig, und die Spezifikation war unvollständig: Z1 verlangt lösbare und in ihrer Entfernung steuerbare Welten, aber keine, in denen die untersuchte Fähigkeit auch erforderlich ist. Die konkrete Stelle, an der das entsteht, ist im Generator benennbar und inzwischen vermessen: Ohne die abgeschaltete Glättungsstufe stammt die Hindernisverteilung aus unkorreliertem Rauschen und bildet verstreute Einzelhindernisse statt zusammenhängender Barrieren. Der Umwegfaktor liegt bei 1,10, der kürzeste Weg ist also nur zehn Prozent länger als die Luftlinie (Abschnitt 4.3). Sackgassen, die Gedächtnis erzwingen würden, entstehen so kaum. Die naheliegende Reparatur, einfach die vorhandene Glättungsstufe einzuschalten, scheitert an derselben niedrigen Basisdichte und macht die Welt noch leerer (Tabelle 7).

Damit ist das Ergebnis dieser Arbeit weder eine Aussage über den Agenten noch eine über den Generator, sondern eine über deren Zusammenspiel: **Die Schwierigkeit einer Lernaufgabe ist eine Eigenschaft der Weltgenerierung und muss dort spezifiziert und geprüft werden, bevor ein Lernverfahren darauf angesetzt wird.** Die Referenzpolitik aus Abschnitt 7.4 ist genau das Instrument dafür, und sie kostet Sekunden statt Trainingsstunden. Dass dieser Zusammenhang überhaupt sichtbar wurde, setzte beide Projektteile voraus: Ohne den eigenen, in seinen Parametern zugänglichen Generator wäre die Ursache nicht lokalisierbar gewesen, ohne das RL-Experiment wäre sie nie aufgefallen.

8.2 Beiträge

Technisch. Entstanden ist ein vollständiges Spiel, das zugleich als Experimentierplattform dient: ein C++-Kern mit deterministischer prozeduraler Weltgenerierung (Biomfeld aus interpoliertem Wertrauschen, biomabhängige Basisbelegung, zelluläre Glättungsstufe, BFS-basierte Exit-Platzierung mit Lösbarkeitsgarantie), ein spielbarer raylib-Client mit KI-Zuschaumodus und eine pybind11-Kapselung als Gymnasium-Umgebung, dazu Curriculum-Trainingspipeline, standardisierte Evaluationsskripte und Live-Monitoring (Kapitel 4 und 5). Die deterministische Reproduzierbarkeit pro Seed und die durchgängige Experiment-Protokollierung waren die Voraussetzung für die Root-Cause-Analysen im Anhang 10 und sind für Folgearbeiten wiederverwendbar.

Der praktische Wert dieser Plattform liegt weniger in den einzelnen Verfahren, die sämtlich Stand der Technik sind, als in ihrer Zugänglichkeit: Weil Wandanteil, Glättung, Distanzmaß und Sicherheitsnetze einzeln schaltbar sind, ließ sich die Geometrie der Welten in Stunden vermessen statt sie zu erraten. Genau das hat die zunächst geplante Härtung über die Glättungsstufe widerlegt, bevor eine einzige Trainingsstunde darauf verfiel (Tabelle 7). Ein fremder Benchmark hätte diese Sonde nicht zugelassen.

Inhaltlich. Ein rekurrenter Agent ohne BFS-Gradientenfeatures erreicht mindestens dieselbe stochastische Leistung wie ein MLP *mit* diesen Features; der LSTM-Zustand ersetzt die vorberechnete lokale Pfadinformation. Diese Aussage ist doppelt begrenzt: durch die Einschränkungen der Gegenüberstellung (Tabelle 13) und dadurch, dass der globale Luftlinien-Kompass in beiden Bedingungen erhalten bleibt. Ersetzt wird also die *lokale* Wegweiserinformation, nicht jede globale Zielinformation.

Methodisch. Die Zwischenauswertung über drei Läufe hatte 73,3 % / 80,0 % ergeben und beide Kriterien scheinbar erfüllt. Die Aufstockung auf sieben Läufe senkte beide Werte deutlich. Ein auf drei Läufen berichtetes „beide Ziele erfüllt“ wäre nicht falsch gemessen, aber nicht haltbar gewesen. Zusammen mit dem Baseline-Befund reproduziert die Arbeit damit zweimal am eigenen Gegenstand, wovon Henderson et al. [6] warnen: zu wenige Läufe und ein fehlender Referenzpunkt führen unabhängig voneinander zu Fehlschlüssen.

8.3 Einschränkungen

1. **Die Erfolgsmetrik ist nicht trennscharf** und wird von einer ungelerten Heuristik übertroffen (Abschnitt 7.4). Dies ist die schwerwiegendste Einschränkung, weil sie alle übrigen Leistungsaussagen relativiert.
2. **Das Testset-Kriterium wird verfehlt** (65,7 % statt $\geq 70\%$); nur das Holdout-Kriterium ist erfüllt.
3. **Der Det/Stoch-Gap bleibt bestehen** (29,1 % bzw. 32,6 % deterministisch). Er wächst mit der Weglänge und ist monoton in der Sampling-Temperatur, also kein Argmax-Artefakt. Theoretisch erwartbar ist nur der Anteil aus der epistemischen Unsicherheit über den Seed [22]; der Rest ist ein Defizit des gelernten Belief-States. Die drei Versuche zu seiner Schließung sind Einzelläufe und belegen nicht, dass er tuning-resistent *ist*.
4. **Begrenzte statistische Präzision:** Die Streuung über Läufe beträgt rund ± 12 Punkte. Das wirkt in beide Richtungen: Das 95 %-Konfidenzintervall auf Holdout B reicht bis 55,0 % hinab und damit unter die im Mittel erfüllte Schwelle; das Intervall auf Testset A reicht bis 77,2 % hinauf und schließt die 70 %-Schwelle ein. Das Testset-Kriterium ist im Punktschätzer verfehlt, nicht aber signifikant verfehlt. Für den Hauptbefund der

Arbeit ist das ohne Belang, da er nicht an dieser Schwelle hängt (Abschnitt 7.4). Die Aufstockung von $n = 3$ auf $n = 7$ war eine Ressourcenentscheidung; der naheliegende und rechnerisch billige nächste Schritt – ein voller Lauf kostet auf CPU nur wenige Stunden – ist eine Erhöhung auf $n \geq 15$, die das Konfidenzintervall spürbar verengen und die Schwellenfrage entscheiden würde. Sie steht aus und ist im Ausblick vermerkt (Abschnitt 8).

5. **Ein einziger Generator und keine quantifizierte Baseline nicht-rekurrenter Verfahren** (DQN, A2C) auf der finalen Umgebung: Der frühe DQN-Anlauf scheiterte vor der v11-Revision und wurde nicht wiederholt.
6. **Selektion und Berichterstattung verwenden verschiedene Betriebsarten.** Ausgewählt wird das nach *deterministischer* Validierungs-SR beste Modell (Gating-Metrik der Phasen 3 und 4), berichtet und als Primärmetrik begründet wird die *stochastische* SR. Bei einem Det/Stoch-Gap von über 30 Punkten, also dem zentralen Untersuchungsgegenstand, wird damit auf die eine Betriebsart selektiert und die andere gemessen. Das ist eine bewusste Entscheidung, weil die stochastische Auswertung theoretisch begründet ist (Abschnitt 7.5) und die deterministische in den frühen Phasen kein Signal trägt (Tabelle 10); der so ausgewählte Checkpoint muss aber nicht der stochastisch beste sein. Eine Selektion auf der stochastischen Validierungs-SR steht als Gegenprobe aus (Abschnitt 8).

8.4 Ausblick

Die Reihenfolge der folgenden Punkte ergibt sich aus dem Befund von Abschnitt 7.4: Solange die Aufgabe die untersuchte Fähigkeit nicht erfordert, ist jede weitere Optimierung des Agenten nicht interpretierbar. Die Reparatur des Messregimes steht deshalb vor allem anderen.

1. **Härtung der Umgebung.** Der Wandanteil wird derzeit pro Biom fest vorgegeben (7 % im Wald bis 25 % im Bergland) und stammt aus unkorreliertem Hash-Rauschen; die Hindernisse sind dadurch überwiegend verstreute Einzeltiles statt zusammenhängender Barrieren (Abschnitt 4.3). Nicht der Wandanteil allein ist deshalb der wirksame Hebel, sondern die *Korreliertheit* der Verteilung. Der naheliegende Weg dorthin ist versperrt, und zwar gemessen: Die implementierte Glättungsstufe einzuschalten senkt die Waddichte auf 0,037 und den Umwegfaktor auf 1,001, und jede Regelvariante, die den Umweg spürbar hebt, zerstört die Lösbarkeit, im Extremfall auf 3 von 50 Welten (Tabelle 7). Eine Härtung verlangt deshalb drei Eingriffe im Code statt eines Schalters: eine höhere und korrelierte Basisdichte in `World::sampleBaseTile()`, eine aktive Konnektivitätsvalidierung (`enableFloodFillValidation`, `enableMacroGraphPrecheck`) und eine Neuvalidierung nach jeder Automateniteration. Zielgröße bleibt nicht ein bestimmter Wandanteil, sondern ein Betriebspunkt, an dem der Kompass-Zufallslauf einbricht, die Lösbarkeit aber bei 100 % bleibt. Das Vorgehen ist zweistufig: erst die Baseline-Politik als Kalibrierinstrument einsetzen, dann trainieren. Ein Trainingslauf ohne diese Vorabprüfung wäre erneut nicht interpretierbar.
2. **Statistische Absicherung des Endergebnisses.** Bei einer Streuung von rund ± 12 Punkten über Läufe überlappt das Konfidenzintervall die Erfolgsschwelle (Abschnitt 7.3). Da ein voller Trainingslauf auf CPU nur wenige Stunden kostet, ist eine Erhöhung von $n = 7$ auf $n \geq 15$ der billigste Weg, die Schwellenfrage auf Testset A statistisch zu entscheiden; sie ist rein rechenzeitgebunden und erfordert keine Codeänderung.

3. **Nachschärfung des dichten Reward-Signals.** Die Bilanz in Tabelle 18 zeigt, dass nur rund 5 % der dichten Reward-Masse überhaupt Richtungsinformation tragen. Naheliegend sind eine Anhebung von β bzw. eine kleinere Normierungskonstante, eine Deckelung des Explorations-Bonus über die Episode und die Überprüfung des Revisit-Penalty, der als letzter verbliebener Term des früher verworfenen „Straf-Stackings“ mit $-6,2$ je Episode überraschend schwer wiegt, deutlich schwerer als der Explorations-Bonus, den er damit faktisch neutralisiert. Da PBRs policy-invariant ist, die übrigen Terme aber nicht, verschiebt die aktuelle Gewichtung das Optimum weg von der eigentlichen Aufgabe.
4. **Billige Diagnosen vor jedem Architekturwechsel.** (a) Eine lineare Probe auf dem LSTM-Hidden-State, die Exit-Richtung bzw. BFS-Distanz vorhersagt: Kollabiert sie auf langen Wegen, liegt ein Belief-Tracking-Limit vor, sonst liegt das Problem in der Politik. (b) Entropie-Annealing in Phase 4 mit Modellselektion auf der deterministischen Erfolgsquote. (c) One-Hot- oder Embedding-Kodierung des lokalen Grids statt der bisherigen ordinalen Tile-IDs; dass das Grid kaum genutzt wird, legt die Baseline-Analyse nahe. (d) Replikation von E1 und E3 mit je zwei weiteren Seeds. (e) Wiederholung des Baseline-Vergleichs mit abgeschalteter Early-Stop-Truncation, um auszuschließen, dass diese explorationsabhängige Abbruchbedingung (Abschnitt 6.3) den Vergleich beeinflusst; bei den trainierten Läufen greift sie in unter 3 % der Episoden, für die Referenzpolitiken ist sie bislang nicht ausgezählt.
5. **Restpotenzial in RecurrentPPO.** Ein Auxiliary-Loss-Kopf, der aus dem Hidden-State die BFS-Distanz vorhersagt, formt den Belief-State explizit und nutzt das Orakel als Trainings-, nicht als Testzeitsignal. Ergänzend Burn-in für Sequenzstarts nach R2D2 [21], temperaturkalibrierte Evaluation ($T = 0,5$ ohne Neutraining) sowie vollwertiges Prioritized Level Replay [25] und Self-Imitation [36].
6. **Architekturvergleich als Folgearbeit.** Der Wechsel auf Transformer-basierte In-Context-Verfahren (z. B. AMAGO [32]) ist die meistgenannte Richtung, hier aber bewusst nur eine Hypothese. Denn erstens ist nicht belegt, dass der Gap ein Kapazitätsproblem ist: Eine unter hoher Entropie trainierte Politik kann ihr Restrauschen zum Verlassen von Schleifen nutzen, ohne je einen Anreiz für eine deterministisch tragfähige Aktionswahl gehabt zu haben. Zweitens bleiben rekurrente Modelle im breitesten verfügbaren POMDP-Vergleich gegenüber den dort geprüften Attention-Modellen konkurrenzfähig; zu beachten ist, dass es sich dabei um *lineare* Transformer-Varianten handelt und nicht um volle Softmax-Attention, wie AMAGO sie verwendet [31]. Die eigene E3-Ablation zeigte, dass mehr Gedächtniskapazität hier schadet. Erst wenn Schritt 2 ein Kapazitätslimit nachweist, wird der Wechsel zur begründeten nächsten Frage; dann als kontrollierter Vergleich auf demselben Generator und Protokoll.

9 Anhang: Reproduzierbarkeit

Dieser Anhang folgt den Reproduzierbarkeits-Checklisten von Pineau et al. [27] und Henderson et al. [6]. Alle Trainingsläufe sind über Seed, gepinnte Umgebung und `config.json` reproduzierbar; die vollständige Änderungshistorie inklusive verworfener Varianten liegt im Experiment-Changelog. Tabelle 21 listet die verwendeten Software-Versionen.

Komponente	Version / Wert
Python	3.12.13
Betriebssystem	macOS (arm64)
Rechen-Device	CPU (bewusst gewählt, s. Performance-Analyse)
PyTorch	2.12.0
Stable-Baselines3 / sb3-contrib	2.8.0 / 2.8.0
Gymnasium	1.2.3
NumPy	2.4.4
Abhängigkeiten (gepinnt)	<code>requirements.txt (==) + requirements.lock.txt</code>

Tabelle 21: Systemumgebung. Nach jeder Änderung an `game_config.json` oder am C++-Kern ist ein Rebuild des Python-Bindings erforderlich.

Reproduzierbarkeits-Checkliste:

- **Code & Konfiguration:** Repository mit Trainings-/Eval-Skripten; je Lauf eine `config.json` mit allen Hyperparametern (Tab. 11, 19).
- **Durchsuchter Hyperparameter-Bereich:** vollständig im `CHANGELOG.md` dokumentiert, inklusive *negativer* Ergebnisse (v7–v10-Fehleranalyse, Tab. 22; Iterationen E1–E3).
- **Anzahl Läufe:** sieben unabhängige Seeds; berichtet werden Mittelwert $\pm$ Standardabweichung, *kein* Best-of-Runs (Kernkritik von Henderson et al. [6]).
- **Datentrennung:** disjunkte Seed-Mengen: Validierungsbereich 6000–6199 (davon 6000–6049 für Phasenwechsel und Modellauswahl, der Rest ausschließlich für Diagnosemessungen), Testset A (7000–7049) und Holdout B (8000–8049) ausschließlich für die finale Messung.
- **Evaluationsprotokoll:** 50 Seeds, Episodenlimit 4 000 (= Umgebungs-Maximum), stochastisch *und* deterministisch. Die Early-Stop-Truncation der Umgebung (256 Schritte ohne positiven Reward) bleibt dabei aktiv und gilt für alle verglichenen Politiken gleichermaßen (Abschnitt 6.3). Bei der rekurrenten Politik werden LSTM-Zustand und Episoden-Start-Signal korrekt an `predict()` durchgereicht und je Episode zurückgesetzt; ein häufiger Fehler beim Evaluieren rekurrenter Politiken wird damit vermieden.
- **Statistische Unsicherheit:** Bei 50 Seeds beträgt das 95 %-Konfidenzintervall einer Erfolgsquote bis zu ± 14 Prozentpunkte; Unterschiede unterhalb dieser Größe sind nicht signifikant zu interpretieren.
- **Rechenaufwand:** je voller Curriculum-Lauf ca. 8 h (CPU) bei voller Taktung; die Läufe 4–7 liefen durch Energie-Drosselung (zugeklappter Laptop) 29–36 h Wall-Clock. Ergebnisrelevanz hat das nicht, da das Training schrittbasiert ist, die Evaluationen auf einem festen Schrittraster liegen und die Phasen an einem Schrittbudget enden; die Diskussion dazu steht in Abschnitt 7.3. Trainingszeiten je Lauf in `results.json`.
- **Zufallsquellen:** Seed steuert Umgebungs-Generierung und -Reset (deterministisch); die stochastische Politik samplet Aktionen (torch-RNG). Restnicht-Determinismus (Thread-Scheduling) wird durch die Mittelung über sieben Läufe aufgefangen.

9.1 Eingesetzte Hilfsmittel

Diese Aufstellung ergänzt die Eidesstattliche Erklärung.

Software und Bibliotheken. Die verwendeten Fremdbibliotheken sind in Tabelle 2 (Technologie-Stack) und Tabelle 21 (Versionen) vollständig aufgeführt. Sie sind sämtlich frei verfügbar und werden an der jeweiligen Stelle zitiert; die Versionen sind über `requirements.txt` und `requirements.lock.txt` gepinnt.

Literatur und Recherche. Die zitierte Literatur ist im Literaturverzeichnis vollständig nachgewiesen. Alle inhaltlichen Aussagen wurden gegen die Primärquellen geprüft.

Generative KI-Werkzeuge. Sie sind vollständig in der „Erklärung zur Nutzung generativer KI“ im Vorspann dieser Arbeit ausgewiesen, einschließlich Einsatzform, betroffener Teile und Art der eigenen Prüfung. Zur Erzeugung von Messergebnissen wurden sie nicht verwendet.

10 Anhang: Entwicklungsverlauf von der Baseline zum LSTM-Agenten

Primärquelle der folgenden Chronologie ist das durchgängig geführte Experiment-Changelog (`CHANGELOG.md`), in dem jede Änderung mit Problem, Lösung und Messergebnis festgehalten wurde. Negative Ergebnisse sind bewusst enthalten; sie haben die Architekturentscheidungen maßgeblich geprägt.

10.1 Etappe 1 (Mai 2026): Baselines mit BFS-Orakel

Nach Fertigstellung der Spiel-Engine (Mitte Mai) begann das RL-Training mit Features aus dem BFS-Distanzfeld *in der Observation* (5 Werte: aktuelle Distanz + 4 Nachbarrichtungen), einem „Orakel“, das dem Agenten die lokal richtige Richtung lieferte.

Erster Anlauf mit DQN: Der erste Agent (DQN) lernte nicht. Diagnose: Die Q-Werte kollabierten auf ein nahezu aktionsunabhängiges Niveau; der Replay-Buffer konservierte dabei die Folgen eines später behobenen Reward-Fehlers über dessen Korrektur hinaus. Hinzu kamen 2-Zellen-Oszillationen und eine Visited-Mask in der Observation (455 Dimensionen), die sich jeden Schritt änderte und den Q-Wert-Argmax destabilisierte. Konsequenz: Wechsel auf das gegenüber solchen Fehlerfolgen strukturell robustere on-policy-Verfahren PPO (Begründung in Abschnitt 6), Visited-Mask entfernt.

BFS-Kraftfeld-Bug: Ein Oracle-Test (greedy dem BFS-Feld folgen) scheiterte in 3/3 Seeds; `bfsDistanceAt()` lieferte für Wand-Tiles einen Manhattan-Fallback, wodurch Wände als attraktive Richtung erschienen. Nach dem Fix (Wand $\rightarrow$ 9999): Oracle 3/3. Dieser Test wurde zum Standard-Werkzeug, um Umgebung und Reward von Trainingsproblemen zu trennen.

Curriculum-Lektion: Phase-1-Training (Exit 5–12) erreichte 56 %, aber der naive Transfer auf Phase 2 (12–25) kollabierte (22 % $\rightarrow$ 6 %); die zuvor gelernte Kurzdistanz-Politik ließ sich nicht überschreiben. Lösung: Mixed-Distribution-Training (Exit 5–45). Damit erreichte *Phase 3* 86 % stochastisch auf Testset A (drei unabhängige Läufe: $85,3 \pm 3,8$ % auf A, $79,3 \pm 8,1$ % auf B) und *Phase 4* nach Reward-Tuning 98 % (A) / 94 % (B).

Der deterministische Kollaps: Trotz 98 % stochastischer SR lag die Argmax-Auswertung bei 2 %. Die Analyse fand die Wurzel in der Feature-Kodierung: Die BFS-Absolutwerte (/64) unterschieden sich zwischen Nachbarzellen nur um $\approx 0,016$, zu wenig für eine robuste Richtungspräferenz, und Wände waren durch Clamping nicht von „weit weg“ unterscheidbar (Aliasing). Der Wechsel auf *Delta-Kodierung* (Richtungssignal $\pm 0,5$ statt 0,016, Wand = +1,0) hob die deterministische SR auf 42 % (offene Welt) bzw. 100 % (wanddichte Hard-World); Temperature-Sampling ($\tau = 0,2$) erreichte 100 % bei $\varnothing$ 90 Schritten. Ein Experiment, `exitDx/exitDy` zu entfernen („Phase 5“), scheiterte (max. 40 %); der Luftlinien-Kompass ist als Fernbereichssignal notwendig.

10.2 Etappe 2 (Juni 2026): Kernbeitrag, Navigation ohne Orakel

Mit funktionierender Baseline wurde die Forschungsfrage geschärft: *Kann der Agent auch ohne BFS-Orakel in der Observation navigieren?* Der Wechsel auf RecurrentPPO (LSTM) mit Curriculum, Explorations-Bonus und `step_frac`-Feature (231-dim Obs, kein BFS) wurde am 07.06.2026 gestartet und lieferte nach der Auswertung am Folgetag das zentrale Ergebnis: **86 % stochastisch / 36 % deterministisch** auf Testset A und 68 % / 18 % auf Holdout B; damals waren beide Zielkriterien stochastisch erfüllt (der final über sieben unabhängige Läufe reproduzierte Wert liegt bei 65,7 %/66,9 % für A/B, Abschnitt 7.3), erstmals ohne Wegweiser-Orakel (Abbildung 16, Ablation in Tabelle 13). Parallel wurde die POMDP-These formuliert (Abschnitt 7.2) und, inspiriert vom PokéRL-Projekt, Infrastruktur ergänzt: Seed-Pool (Swarm), WebSocket-Live-Map, Heatmap-Evaluation, Early-Stop-Truncation.

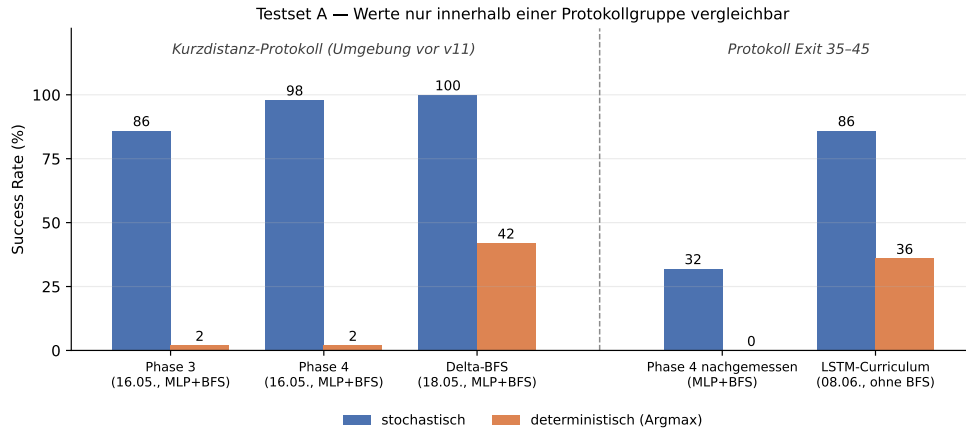


Abbildung 16: Entwicklung über die Modellgenerationen auf Testset A. Links die MLP-Baselines mit BFS-Orakel in der Observation, rechts der LSTM-Agent ohne Orakel. **Die beiden Gruppen sind nicht miteinander vergleichbar:** Die Mai-Messungen entstanden auf der Umgebung vor der v11-Revision unter einem Kurzdistanz-Protokoll, die Juni-Messungen unter dem Standardprotokoll (Exit 35–45). Wie groß der Unterschied ist, zeigt der vierte Balken: Dasselbe Phase-4-Modell fällt unter dem Standardprotokoll von 98 % auf 32 % stochastisch und von 2 % auf 0 % deterministisch. Nur die beiden rechten Balken tragen den Vergleich in Tabelle 13. Erzeugt mit `scripts/plot_entwicklung.py`.

Methodische Härting (11.06.): Ein Data-Leakage-Fehler wurde entdeckt und behoben. Der Trainings-Callback hatte Modellauswahl und Phasenwechsel auf den *Test-Seeds* (7000er) gesteuert; seitdem existiert das getrennte Validierungsset (6000er). Zusätzlich wurde die Lösbarkeit aller relevanten Welten bewiesen (3 150 Seeds, 100 %).

10.3 Etappe 3 (13.–25.06.): Fehlkonfigurierte Hyperparameter und Root-Cause-Analyse

Symptom: Eine Serie von Verbesserungsversuchen (v6–v9: Wand-Penalty-Entfernung, `visit_count`- und Action-Buffer-Features, diverse Batch-/Epochen-Kombinationen) scheiterte durchgängig (SR 0–12 %, `approx_k1` $\approx$ 0). **Root-Cause:** Die Aufklärung gelang durch *Reverse-Engineering* des letzten funktionierenden Modells (v2): Die zwischenzeitlichen „Optimierungen“ hatten unbemerkt die kritischen Hyperparameter verstellt (Tabelle 22); dominanter Faktor war `ent_coef` (0,05 $\rightarrow$ 0,01: der Agent explorierte nicht mehr genug, um den Exit je zu finden). **Ergebnis:** Die v2-Reproduktion (v10) stellte das Lernen wieder her; der abgeschlossene Lauf vom 25.06. erreichte 86 % (Validierung) nach 2,2 Mio. Steps und wurde zum Referenzmodell dieser Etappe. (Nicht zu verwechseln mit dem Lauf in Abbildung 12: Der stammt vom 08.06. und erreichte seine 86 % auf Testset A, nicht auf der Validierung.) Zur Laufzeit dieses Laufs liegt keine protokollierte Messung vor; die für dieselbe Konfiguration dokumentierte Erwartung lag bei rund 5,5 h, die später gemessenen v12-Läufe gleichen Umfangs brauchten 7,7–8,5 h (Anhang 9).

10.4 Etappe 4 (06.07.2026): Umgebungsrevision v11 und Performance

Ein systematisches Audit (Umgebung, Reward, Trainingspipeline, Usability) führte zu zehn dokumentierten Änderungen; Details in Abschnitt 10.7 und Abschnitt 10.8. Höhepunkte: präzise Schwierigkeitssteuerung über BFS-Laufweg, Behebung eines prozess-globalen Config-Lecks (das zuvor unbemerkt die Phase-3-Trainingsverteilung verschob), Entfernung toter Features und Straf-Terme, sowie die Wahl der CPU als Rechen-Device. Die an diesem Tag eben-

Parameter	v2 (86 %)	v7–v9 (0–12 %)	Wirkung des Fehlers
ent_coef	0.05	0.01	zu wenig Exploration; Exit wird nie gefunden
n_steps	256	512	BPTT zu lang → verschwindende Gradienten
batch_size	8	256	zu wenige Gradient-Updates pro Rollout
Obs-Dimension	231	249	Zusatzfeatures ohne funktionierende Basis

Tabelle 22: Root-Cause-Analyse: `ent_coef` erwies sich als dominanter Faktor. Die Reproduktion der v2-Konfiguration (v10) stellte das Lernen wieder her.

falls vorgenommene Verdopplung der Trainingsgeschwindigkeit über `batch_size` 8 → 64 galt zunächst als validiert, wurde am Folgetag jedoch widerlegt (Abschnitt 10.6) und zurückgenommen.

10.5 Etappe 5 (07.07.2026): Erster v11-Gesamtlauf, Monitoring-Redesign und Seed-Replay-Analyse

Erster v11-Gesamtlauf (Seed 0, laufend). Der erste vollständige Curriculum-Lauf auf der revidierten v11-Umgebung (`models/ppo_lstm_curriculum_v11`) zeigt folgendes Zwischenbild (Abbildung 17): Phase 1 lag über weite Strecken bei 12–18 % (stochastisch) und stieg erst gegen Ende auf 42 %, mit dem das Step-Limit erreicht wurde; die Batch-Validierung derselben Konfiguration hatte 52 % bereits bei 150 k erreicht. Die Diagnose im Trainingslog: Die Policy-Updates sind durchgehend gesund (`approx_kl` ≈ 0.03 , stabile Entropie), aber die `explained_variance` des Critics verharrt bei ≈ 0.1 (Validierungslauf: 0.72); der Lauf wurde zunächst als Kandidat für eine ungünstige Zufallsinitialisierung eingestuft. (*Nachtrag: Die systematische Fehlersuche am selben Abend, Abschnitt 10.6, fand die tatsächliche Ursache: die Batch-Größe.*) Bemerkenswert: Direkt nach dem Wechsel in Phase 2 stieg die SR auf 48 % bei größerer *Exit-Distanz*; die breitere Aufgabenverteilung wirkt als Regularisierer.

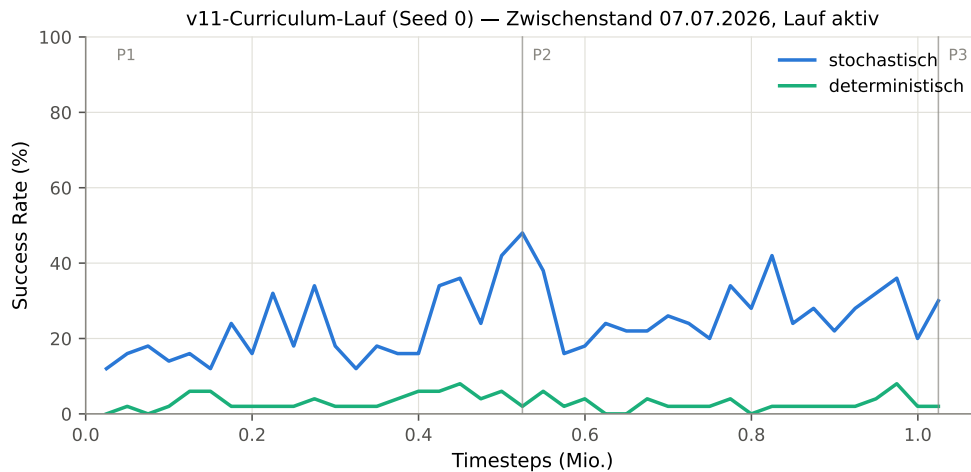


Abbildung 17: Lernkurve des ersten v11-Gesamtlaufs (Seed 0, Zwischenstand in Phase 3), beide Betriebsarten. Vertikale Linien: Phasenwechsel; die x-Achse ist über die Phasen kumuliert. Erzeugt mit `scripts/plot_run_curve.py`.

Live-Monitoring-Redesign. Das WebSocket-Dashboard aus Etappe 2 wurde nach Dashboard- und Farb-Literatur überarbeitet (Abbildung 18): (a) Charts auf uPlot mit EMA-Glättung nach TensorBoard-Vorbild; (b) die SR-Kurve zeigt det. und stoch. getrennt; der zentrale Untersuchungsgegenstand war zuvor im Monitoring unsichtbar; (c) die Explorations-Heatmap wechselte von der Jet- auf die perzeptuell uniforme Viridis-Skala (Jet erzeugt

künstliche Kontraste und ist nicht farbfeldsichten-tauglich); (d) die Agenten-Minimaps rekonstruieren aus dem gestreamten 15×15 -Sichtfeld eine wachsende Weltkarte (Wände, Bäume, Exit) statt nur der Agentenposition. Beim Umbau wurden zwei Fehler gefunden und behoben: Der Exit-Richtungspfeil war seit der Obs-Umstellung auf 229 Dimensionen stumm defekt, und der SB3-Stepzähler springt beim Phasenwechsel zurück (es wird das Bestmodell der Vorphase geladen), was unkorrigiert alle stepbasierten Zeitachsen verfälscht.

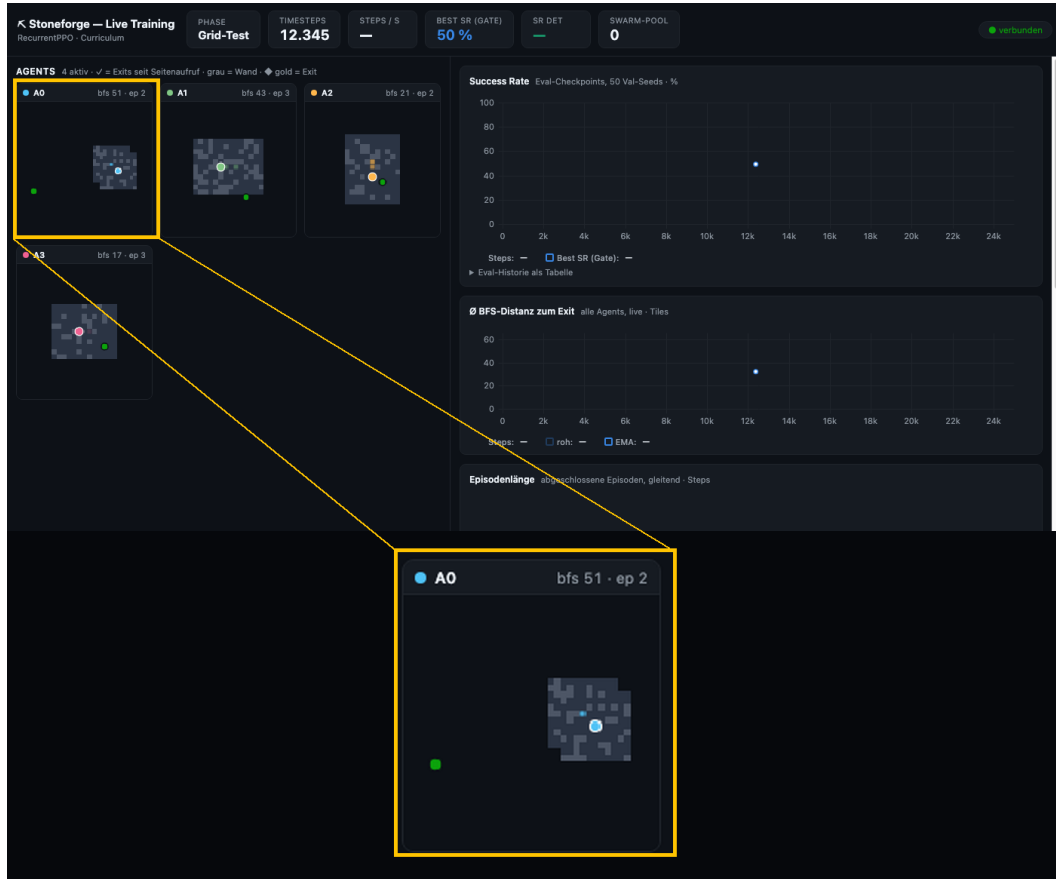


Abbildung 18: Live-Monitoring (Demo mit Zufallsagenten): KPI-Zeile, Weltkarten-Minimaps (grau = Wand, blaugrau = erkundeter Boden, grün = Startpunkt) und uPlot-Charts. Im Training zusätzlich: goldene Raute = gesichteter Exit, grüner Blitz bei Exit-Erfolg, Viridis-Heatmap.

Seed-Replay-Analyse (Swarm vs. PLR). Das Training wiederholt seit Etappe 2 bei 30 % der Episoden-Resets einen bereits *gelösten* Seed („Erfolgs-Swarm“). Ein Literaturabgleich stellt diese Semantik in Frage: Prioritized Level Replay [25] zeigt, dass das Replay von Levels mit *hohem Lernpotenzial*, also gerade *nicht* gemeisterten, Sample-Effizienz und Test-Generalisierung verbessert; das Wiederholen gemeisteter Levels ist der Fehlermodus, den PLR behebt, und riskiert im Kontext des Projektziels (Generalisierung auf unbekannte Seeds) Layout-Memorierung. Da der 86 %-Referenzlauf jedoch *mit* Erfolgs-Swarm entstand, wird die Frage nicht per Meinung, sondern als A/B-Test entschieden (Erfolgs-Swarm vs. `--plr` vs. `--no-swarm`; Maßnahme B8 in docs/BEWERTUNG_UND_PLAN.md). Ein dabei entdeckter Designfehler wurde sofort behoben: Der Seed-Pool überlebte Phasenwechsel, obwohl ein „gelöster“ Seed nach dem Wechsel der Exit-Distanz eine andere Aufgabe beschreibt; der Pool wird jetzt bei jedem Phasenstart geleert. (Der A/B-Test wurde noch am selben Tag im Zuge der Fehlersuche durchgeführt, siehe Abschnitt 10.6: Der Erfolgs-Swarm hatte auf Phase 1 keinen messbaren Effekt.)

10.6 Etappe 6 (07.07.2026, nachmittags/abends): Systematische Fehlersuche vom Eval-Artefakt zur Batch-Größen-Korrektur

Die Stagnation der v11-Läufe wurde am Nachmittag des 07.07. in einer mehrstufigen, hypothesengetriebenen Fehlersuche aufgeklärt. Die folgende Darstellung enthält auch die verworfenen Hypothesen, da erst deren Ausschluss die jeweils nächste Maßnahme bestimmte. Zwei der Befunde, das Eval-Cap-Artefakt und die Wirkung der Batch-Größe, haben Tragweite über das konkrete Problem hinaus.

Schritt 1: Messartefakt entdeckt, Eval-Episoden-Caps. Ausgangspunkt war ein Widerspruch: Der Vergleichslauf Seed 1 stagnierte in Phase 1 bei 8–16 %, auf demselben Niveau, auf dem sich Seed 0 den Großteil seiner Phase 1 bewegt hatte, obwohl die Batch-Validierung derselben Konfiguration 52 % erreicht hatte. Eine Nachmessung des besten Seed-0-Checkpoints deckte die Ursache auf: Die am 06.07. zur Eval-Beschleunigung eingeführten Episoden-Caps (Phase 1: 600 statt 4000 Schritte) unterschätzen die Kompetenz erheblich, weil erfolgreiche LSTM-Episoden ohne Orakel lange Suchwege enthalten (Median 477, p95 $\approx 1\,900$, Maximum 3\,239 Schritte). Bemerkenswert ist dabei, dass Phase 1 auf Exit-Distanzen von nur 5–12 Feldern evaluiert (Tabelle 10): Ein Median von 477 Schritten entspricht dort einer Pfadeffizienz von $\eta \approx 0,02$. Bereits auf der leichtesten Curriculum-Stufe löst der Agent die Aufgabe also durch ausgedehnte Suche und nicht durch gerichtete Navigation, ein früher Hinweis auf den in Abschnitt 7.4 systematisch belegten Befund. Dasselbe Modell misst sich bei Cap 600 als 48-%-Agent, bei Cap 4000 als 86-%-Agent (Abbildung 19). Damit war das 85-%-Phasenziel unter Cap 600 strukturell unerreichbar, und die Bestmodell-Selektion bevorzugte schnelle statt kompetente Politiken. Da die volle Messung nur ≈ 21 s kostet, wurden alle Caps auf 4000 (= Episoden-Limit der Umgebung = finales Eval-Protokoll) zurückgesetzt. *Lehre: Ein Eval-Protokoll-Detail (Cap) kann Gating, Modellselektion und Fortschrittswahrnehmung gleichzeitig verfälschen.*

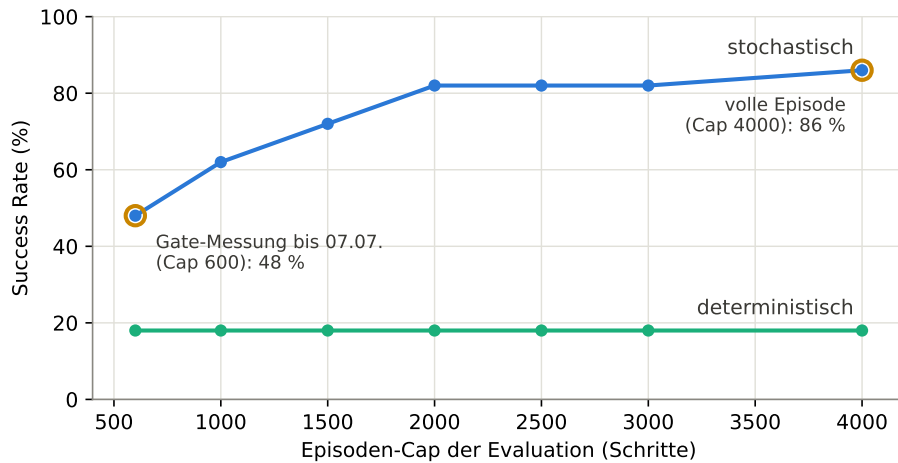


Abbildung 19: Erfolgsquote *desselben Modells* (bester Phase-1-Checkpoint, Seed 0) in Abhängigkeit des Episodenlimits der Evaluation, 50 Val-Seeds. Erzeugt mit `scripts/plot_etappe6_figures.py`.

Schritt 2: Neustart unter korrekter Messung, Stagnation ist real. Seed 1 wurde mit korrigierten Caps neu gestartet. Ergebnis: weiterhin Stagnation (10–34 % über 300 k Schritte ohne erkennbaren Trend, `explained_variance` ≈ 0.1); der zweite stagnierende Seed in Folge. Damit war ein systematisches Problem belegt und eine Verdächtigenliste abzuarbeiten.

Schritt 3: Hypothesen nacheinander eliminiert (A/B-Läufe, Seed fixiert). Drei Komponenten des Trainings-Stacks wurden durch kontrollierte Vergleichsläufe mit identi-

schem Seed geprüft:

- *Wand-/Loop-Penalties (Maßnahme B7)?* Entlastet ohne neuen Lauf: Die Batch-Validierung lief bereits auf der penalty-freien v11-Umgebung und lernte.
- *Erfolgs-Swarm (Maßnahme B8)?* Ein `--no-swarm`-Arm verlief deckungsgleich mit dem Kontrollarm; entlastet.
- *Streaming-Wrapper des Live-Monitorings?* Code-Inspektion (rein lesende Zugriffe) und ein Lauf ohne Wrapper: ebenfalls deckungsgleich; entlastet. (Nebenbefund behoben: Der Wrapper wurde bisher auch bei deaktivierter Live Map angelegt.)

Parallel dazu wurde die Batch-Validierung vom 06.07. *exakt reproduziert* (nackter RecurrentPPO ohne Curriculum-Stack, 150 k Schritte, batch=64): Sie erreichte am Endpunkt fast dieselben Werte wie damals (48%/22% vs. 52%/22%); der *Verlauf* dazwischen schwankte stark und unregelmäßig (74 → 44 → 66 → 24 → 50%, Abbildung 20 oben). Zusätzlich wurde ausgeschlossen, dass sich Curriculum-Pfad und nackter Pfad überhaupt funktional unterscheiden: Gewichtsinitialisierung, Weltseed-Folgen und die ersten 8192 Trainingsschritte (Rewards, Aktionen, Values) sind zwischen beiden *bit-identisch*.

Schritt 4: Diagnose, hochvariable Lerndynamik statt defekter Code. Die Gesamtschau aller Läufe ergab: Mit batch=64 ist die Lerndynamik hochvolatil (SR-Sprünge zwischen 10 und 74% ohne Konvergenz), der Critic erreicht nur vereinzelte, nicht anhaltende **explained_variance**-Hochphasen. Die „Validierung“ der Batch-Größe am 06.07. beruhte auf einem günstigen Einzel-Messpunkt dieses Prozesses: einem Messwert aus einer stark schwankenden Kurve, nicht auf deren Verlauf. *Lehre: Hyperparameter-Entscheidungen brauchen Eval-Kurven, keine Endpunkte.*

Schritt 5: Hypothesentest Batch-Größe, bestätigt. Der einzige verbliebene Unterschied zum funktionierenden v2/v10-Rezept war `batch_size=64` statt 8 (bei batch=8 erfolgen pro Rollout ≈ 8 -mal mehr Gradientenschritte auf denselben Daten; die v10-Reproduktion hatte damit EV bis 0.94 erreicht). Ein ansonsten identischer Lauf mit batch=8 zeigte ein qualitativ anderes Bild (Abbildung 20): stetiger Anstieg auf **84–88% stochastische SR** bei 125–150 k Schritten, die besten Phase-1-Werte des Projekts, und erstmals über **50% deterministische SR** in Phase 1 (batch=64: nie über 26%). Ab ≈ 100 k Schritten hält der Critic anhaltende EV-Hochphasen (26% der Iterationen > 0.5 , gegenüber 3% bei batch=64). Konsequenz: `batch_size` wurde auf 8 zurückgesetzt; der Geschwindigkeitsvorteil von batch=64 (Faktor 1,9–2,1) war mit instabilem Lernen erkaufte und damit wertlos.

Ergebnis der Etappe. Die v11-Stagnation hatte zwei unabhängige, sich überlagernde Ursachen: ein *Messartefakt* (Eval-Caps) und eine *echte Regression* (batch=64). Trainings-Stack, Seed-Replay, Wrapper und Umgebungsrevision wurden dabei experimentell entlastet, ein für die weitere Arbeit wertvolles Nebenergebnis, da die v11-Umgebung damit als methodisch sauber bestätigt ist. Die Konfiguration für die finalen Mehrfach-Läufe steht damit fest (v11-Umgebung, batch=8, Cap-4000-Evals, kurvenbasierte Bewertung).

10.7 Umgebungsrevision v11 (06.07.2026)

Eine systematische Überprüfung der Umgebung gegen Best Practices des Environment- und Reward-Designs führte zu zehn dokumentierten Änderungen. Die folgenden sieben betreffen Umgebung, Reward und Evaluation; die übrigen drei (Device- und Batch-Benchmark, Konsistenz- und Usability-Bereinigung) sind in Abschnitt 10.8 beschrieben.

1. **Tote Features entfernt:** Energie und Inventar waren konstant; Observation 231 → 229 Dimensionen (Tabelle 8).
2. **Straf-Stacking entschärft:** Wand-Penalty entfernt, Loop-Penalty $-0.15 \rightarrow -0.05$ (Tabelle 9).

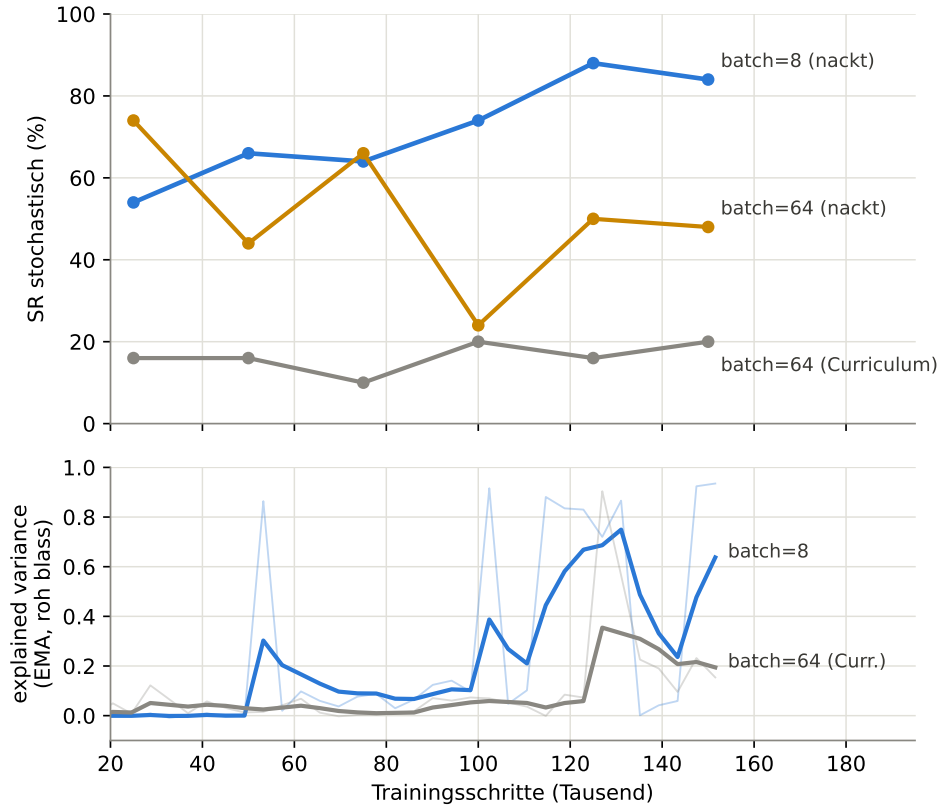


Abbildung 20: Batch-Größen-Forensik (alle Läufe Seed 1, Phase-1-Setup, 50 Val-Seeds, Limit 4000). Oben: stochastische Erfolgsquote. Unten: `explained_variance` des Critics, EMA-geglättet mit blassen Rohkurven. Erzeugt mit `scripts/plot_etappe6_figures.py`.

3. **Exit-Platzierung nach BFS-Laufweg:** Zuvor wurden Kandidaten nach *Luftlinie* gewählt; der reale Laufweg bei nominell „35–45“ streute auf 42–75 Tiles ($\varnothing$ 55). Nach dem Fix liegen alle 150 Eval-Seeds exakt im Sollbereich (Abbildung 21); das Curriculum steuert die Schwierigkeit damit erstmals präzise.
4. **Prozess-globales Config-Leck behoben:** Die Weltgenerierungs-Konfiguration ist im C++-Kern global; parallel existierende Umgebungen überschrieben sich gegenseitig (u. a. verschob der Eval-Callback die Phase-3-Trainingsverteilung unbemerkt). Jede Umgebung stempelt ihre Konfiguration nun bei jedem Reset neu.
5. **Mining/Bauen/Kampf aus dem RL-Pfad entfernt** (Binding erlaubt nur noch Aktionen 0–3; tote Reward-Terme gestrichen).
6. **Entropie-Annealing korrigiert:** stetig 0.05 $\rightarrow$ 0.001 statt Sprung 0.05 $\rightarrow$ 0.01 beim Phasenwechsel.
7. **Eval-Callback:** phasengerechtes Gating (Tabelle 10), Episoden-Caps pro Phase, Det+Stoch-Doppelmessung.

Alle Änderungen wurden einzeln verifiziert (Lösbarkeit 150/150, Oracle-Agent 10/10 mit exakt BFS-Distanz Schritten, Obs-Layout-Konsistenz, Ablehnung unzulässiger Aktionen). Da sich die Aufgabenverteilung dadurch ändert, sind Ergebnisse ab v11 als neue Baseline zu behandeln.

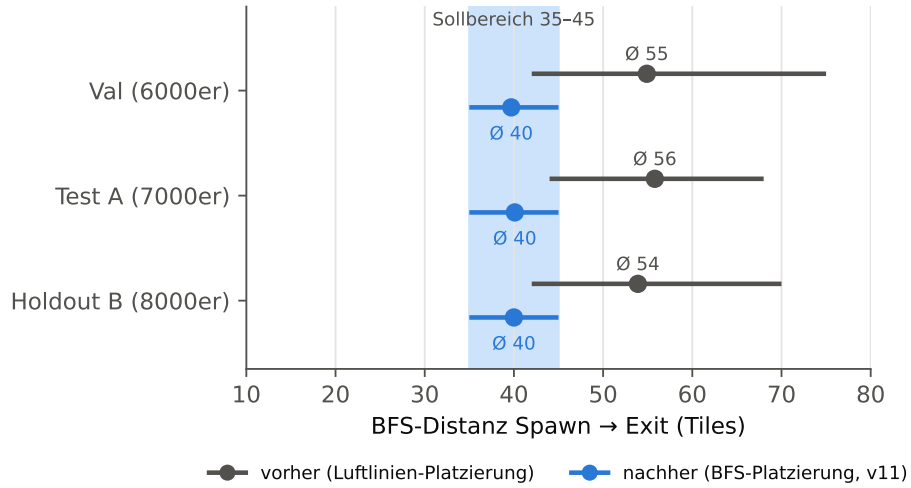


Abbildung 21: Wirkung der BFS-basierten Exit-Platzierung: tatsächliche Laufweg-Distanzen (min/Ø/max über je 50 Seeds) vor und nach dem Fix.

10.8 Performance-Analyse: Training beschleunigen

10.8.1 CPU schlägt Apple-GPU (MPS)

Sämtliche Läufe dieser Arbeit entstanden auf einem Notebook der Apple-M-Serie (M1 Pro, 8 Performance-Kerne, macOS arm64); als Rechen-Device stand damit neben der CPU die integrierte GPU über das MPS-Backend zur Verfügung. Ein kontrollierter Benchmark (identisches Trainings-Setup, 8192 Steps) zeigt: Die GPU ist für dieses kleine Netz (LSTM 256, 229 Inputs) in *jeder* Konfiguration langsamer als die CPU; der Kernel-Dispatch-Overhead pro Mini-Update übersteigt den Rechengewinn (Abbildung 22). Dies deckt sich mit der Empfehlung der SB3-Dokumentation, PPO ohne CNN-Politik primär auf der CPU zu betreiben [37].

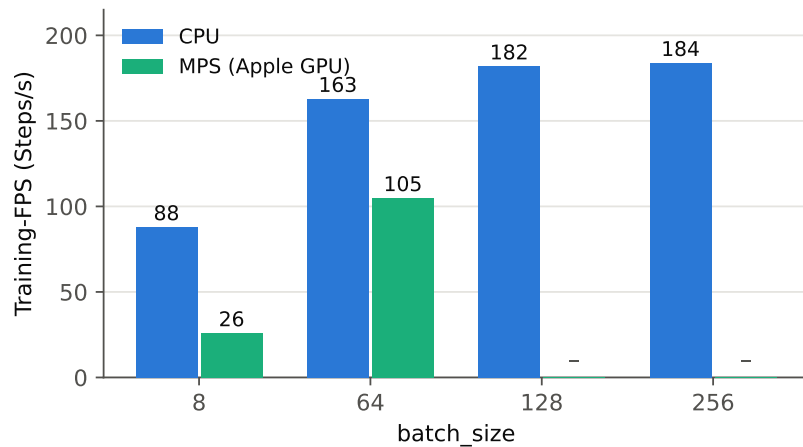


Abbildung 22: Training-FPS nach Device und Batch-Größe (RecurrentPPO, 16 Umgebungen, Apple M-Serie). MPS wurde für batch ≥ 128 nicht weiter gemessen, da bereits bei 64 unterlegen.

10.8.2 Batch-Größe: vermeintliche Validierung (06.07.), am Folgetag widerlegt

Mit `batch_size=8` fallen pro Rollout 5 120 Gradient-Updates an, mehr Optimizer-Schritte als Umgebungsschritte. Da die Root-Cause-Analyse (Tabelle 22) die Batch-Größe nie isoliert getestet hatte, wurde ein Validierungslauf durchgeführt: 150k Phase-1-Steps mit `batch_size=64` (Abbildung 23). Ergebnis: gesunde Lernmetriken (`approx_kl` 0.01–0.06, `explained_variance` bis 0.72) und **52 % stochastische / 22 % deterministische SR** nach nur 150k Steps, bei **163–187 statt 88 FPS** (Abbildung 22; der Durchsatz von `batch=64` schwankt über den Benchmark, `batch=8` liegt stabil bei 88). `batch_size=64` wurde daraufhin übernommen.

Diese Schlussfolgerung hielt der Überprüfung nicht stand. Die Forensik vom 07.07. (Abschnitt 10.6) zeigte per exakter Reproduktion, dass der Validierungslauf ein günstiger Einzel-Schnappschuss einer stark schwankenden Lerndynamik war: Der Endpunkt (52 %) war reproduzierbar, der Verlauf dorthin sprang jedoch regellos zwischen 24 und 74 %, und in vollen Curriculum-Läufen stagnierte `batch=64` bei 16–28 %. Mit `batch_size=8` lernt dieselbe Konfiguration stetig auf 84–88 % (Abbildung 20). Die Batch-Größe *war* also doch ein kritischer v2-Erfolgsfaktor; sie wurde auf 8 zurückgesetzt. Die Lehre daraus: *Einzel-Messpunkte volatiler Prozesse validieren nichts.*

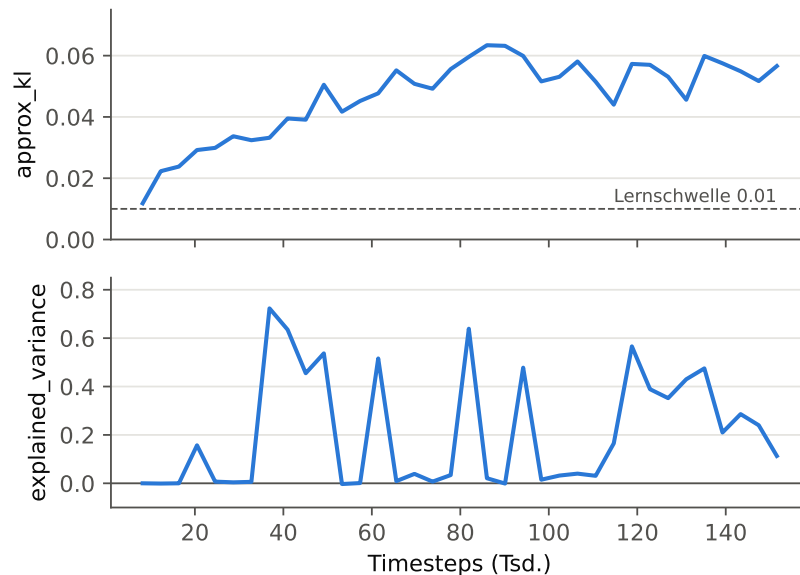


Abbildung 23: Validierungslauf `batch_size=64` (Phase-1-Setup, Env v11), Stand 06.07.: `approx_kl` und `explained_variance` über der Zeit. Zur späteren Widerlegung dieses Befunds siehe Abschnitt 10.6.

10.9 Zeitleiste und Lessons Learned

Tabelle 23 fasst den Verlauf der vorangegangenen Abschnitte chronologisch zusammen; die anschließende Liste hält die übertragbaren Erkenntnisse fest.

Lessons Learned (negative Ergebnisse mit Erkenntniswert):

- *Visited-Mask in der Observation* destabilisiert den Argmax (jede 1-Bit-Änderung kann die Richtungspräferenz kippen).
- *Naiver Curriculum-Transfer* zerstört die Vorgänger-Politik; Mixed-Distribution bzw. leistungsbasierte Phasenwechsel sind nötig.

Datum	Meilenstein	Ergebnis (Testset A)
12.05.	Spiel-Engine & Weltgenerierung spielbar	–
16.05.	DQN-Fehlstart → PPO; BFS-Kraftfeld-Bug gefixt	Oracle 0/3 → 3/3
16.05.	Curriculum P1; naiver Transfer kollabiert → Mixed-Dist.	56 % → 86 % stoch
16.05.	Phase 4 (Reward-Tuning), 3-Läufe-Protokoll	98 % stoch / 2 % det
18.05.	Delta-BFS-Kodierung + Temperature-Sampling	100 % stoch / 42 % det
07./08.06.	RecurrentPPO ohne BFS-Orakel (Kernbeitrag) ; Lauf gestartet 07.06., ausgewertet 08.06.	86 % / 36 %
08.06.	POMDP-These; Callback-Bugs; Swarm/Live-Map	–
11.06.	Data-Leakage behoben (Val-Set 6000er); Lösbarkeit 100 %	–
13.–15.06.	v6–v9 scheitern → Root-Cause (ent_coef/batch/n_steps)	0–12 %
25.06.	v10-Reproduktion abgeschlossen (Referenzmodell)	86 % (Val)
06.07.	Env-Revision v11 (10 Änderungen); batch=64 „validiert“ (am 07.07. widerlegt)	52 % stoch @150k (P1)
07.07.	v11-Läufe Seed 0/1 stagnieren; Monitoring-Redesign; Swarm-Analyse (B8) + Pool-Fix	10–34 % stoch (P1, Val)
07.07.	Forensik (Etappe 6) : Eval-Cap-Artefakt gefixt; Swarm/Wrapper/Stack entlastet; batch 64 → 8	84–88 % stoch / 52 % det (P1, Val)
08.07.	v12-Läufe Seeds 1–3 (batch=8) ; Gap-Experimente E1–E3 + Temperatur-Sweep	73,3 % / 80,0 % (A/B, stoch, $n = 3$)
17.07.	Aufstockung auf $n = 7$ (Seeds 4–7); Alternativklärungen ausgeschlossen	65,7 % / 66,9 % (A/B, stoch, $n = 7$)

Tabelle 23: Zeitleiste der RL-Entwicklung (Auswahl; vollständig in `CHANGELOG.md`). SR-Angaben je nach Etappe mit unterschiedlichem Protokoll; Werte vor dem 11.06. ohne strikte Val/Test-Trennung.

- *Feature-Kodierung schlägt Feature-Auswahl*: Der BFS-Gradient war nie „zu schwach“, sondern falsch kodiert (Delta-Fix: Signal $32\times$ stärker).
- *Straf-Stacking* (Wand-, Loop-, Stagnations-Penalties) macht Stillstand lokal optimal; Information gehört in die Observation, nicht in den Reward.
- *Hyperparameter dominieren Features*: Alle v7-Feature-Ideen scheiterten an verstellten Basis-Hyperparametern; **ent_coef** war der entscheidende Faktor. Ohne das Changelog wäre die Root-Cause-Analyse (v2-Reverse-Engineering) nicht möglich gewesen.
- *Eval-Protokoll ist Teil des Experiments*: Data-Leakage über den Callback und das Gating auf einer signal-losen Metrik (det. SR bei hoher Entropie) verzerrten Selektion und Laufzeit unbemerkt.
- *Eval-Episoden-Caps verfälschen die Messung*: Dasselbe Modell misst sich bei Cap 600 als 48-%-, bei Cap 4000 als 86-%-Agent; eine als harmlos gedachte Beschleunigung machte das Phasenziel unerreichbar (Abschnitt 10.6).
- *Einzel-Snapshots sind keine Validierung*: Die batch=64-Freigabe beruhte auf einem Messpunkt einer stark schwankenden Kurve; die exakte Reproduktion traf den Endpunkt, zeigte aber, dass der Verlauf dazwischen volatil war. Hyperparameter-Entscheidungen nur auf ganzen Eval-Kurven.
- *Negative Kontrollen lohnen sich*: Der bit-genaue Vergleich von Curriculum-Pfad und nacktem Trainingspfad schloss eine ganze Verdächtige-Klasse (Stack-Bugs) in Minuten

aus und lenkte die Suche auf die Hyperparameter.

Literatur

- [1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. <https://www.nature.com/articles/nature14236>.
- [2] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International conference on machine learning*, pages 2048–2056. PMLR, 2020. <https://arxiv.org/abs/1912.01588>.
- [3] Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research (JAIR)*, 76:201–264, 2023. <https://arxiv.org/abs/2111.09794>.
- [4] Sebastian Risi and Julian Togelius. Increasing generality in machine learning through procedural content generation. *Nature Machine Intelligence*, 2(8):428–436, 2020. <https://www.nature.com/articles/s42256-020-0208-5>.
- [5] OpenAI. Procgen benchmark: Procedurally-generated game-like gym environments. Quellcode-Repository, 2020. <https://github.com/openai/procgen>.
- [6] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. <https://arxiv.org/abs/1709.06560>.
- [7] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018. <http://incompleteideas.net/book/the-book-2nd.html>.
- [8] Noor Shaker, Julian Togelius, and Mark J. Nelson. *Procedural Content Generation in Games*. Computational Synthesis and Creative Systems. Springer, 2016.
- [9] Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186, 2011. <https://ieeexplore.ieee.org/document/5963131>.
- [10] Ken Perlin. An image synthesizer. In *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 287–296. ACM, 1985.
- [11] Lawrence Johnson, Georgios N. Yannakakis, and Julian Togelius. Cellular automata for real-time generation of infinite cave levels. In *Proceedings of the 2010 Workshop on Procedural Content Generation in Games (PCGames)*, pages 1–4. ACM, 2010.
- [12] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [13] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.
- [14] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. *arXiv preprint arXiv:1507.06527*, 2015. <https://arxiv.org/abs/1507.06527>.

- [15] Satinder P Singh, Tommi Jaakkola, and Michael I Jordan. Learning without state-estimation in partially observable markovian decision processes. In *Proceedings of the Eleventh International Conference on Machine Learning (ICML)*, pages 284–292, 1994. <https://www.cs.utexas.edu/~shivaram/readings/b2hd-SinghJJ1994.html>.
- [16] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016. <https://arxiv.org/abs/1506.02438>.
- [17] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 1928–1937. PMLR, 2016. <https://arxiv.org/abs/1602.01783>.
- [18] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>.
- [19] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. <http://jmlr.org/papers/v22/20-1364.html>.
- [20] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [21] Steven Kapturowski, Georg Ostrovski, John Quan, Rémi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2019.
- [22] Dibya Ghosh, Jad Rahme, Aviral Kumar, Amy Zhang, Ryan P Adams, and Sergey Levine. Why generalization in RL is difficult: Epistemic POMDPs and implicit partial observability. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021. <https://arxiv.org/abs/2107.06277>.
- [23] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- [24] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 41–48, 2009. <https://dl.acm.org/doi/10.1145/1553374.1553380>.
- [25] Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning (ICML)*, pages 4940–4950. PMLR, 2021. <https://arxiv.org/abs/2010.03934>.
- [26] Norbert L. Kerr. HARKing: Hypothesizing after the results are known. *Personality and Social Psychology Review*, 2(3):196–217, 1998. https://journals.sagepub.com/doi/10.1207/s15327957pspr0203_4.

- [27] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021. <https://arxiv.org/abs/2003.12206>.
- [28] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, et al. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. *Advances in Neural Information Processing Systems*, 36, 2023. <https://arxiv.org/abs/2306.13831>.
- [29] Danijar Hafner. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*, 2021. <https://arxiv.org/abs/2109.06780>.
- [30] Marco Pleines, Matthias Pallasch, Frank Zimmer, and Mike Preuss. Generalization, mayhems and limits in recurrent proximal policy optimization. *arXiv preprint arXiv:2205.11104*, 2022. <https://arxiv.org/abs/2205.11104>.
- [31] Steven Morad, Ryan Kortvelesy, Matteo Bettini, Stephan Liwicki, and Amanda Prorok. POPGym: Benchmarking partially observable reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2023. <https://arxiv.org/abs/2303.01859>.
- [32] Jake Grigsby, Linxi Fan, and Yuke Zhu. AMAGO: Scalable in-context reinforcement learning for adaptive agents. In *International Conference on Learning Representations (ICLR)*, 2024. <https://arxiv.org/abs/2310.09971>.
- [33] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pages 2778–2787. PMLR, 2017. <https://arxiv.org/abs/1705.05363>.
- [34] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*, 2019. Preprint 2018, <https://arxiv.org/abs/1810.12894>.
- [35] Guy L. Steele, Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 453–472. ACM, 2014.
- [36] Junhyuk Oh, Yijie Guo, Satinder Singh, and Honglak Lee. Self-imitation learning. In *International Conference on Machine Learning (ICML)*, pages 3878–3887. PMLR, 2018. <https://arxiv.org/abs/1806.05635>.
- [37] Stable-Baselines3 Contributors. Stable-baselines3 documentation: PPO, 2025. <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html>, abgerufen am 25.07.2026: „PPO is meant to be run primarily on the CPU, especially when you are not using a CNN“.